Week 2 — Assembly, Memory Hierarchy, and the C Abstract Machine

Format: seven daily sections collected into one weekly module

Suggested workload: approximately 21 focused hours, adjustable to mastery

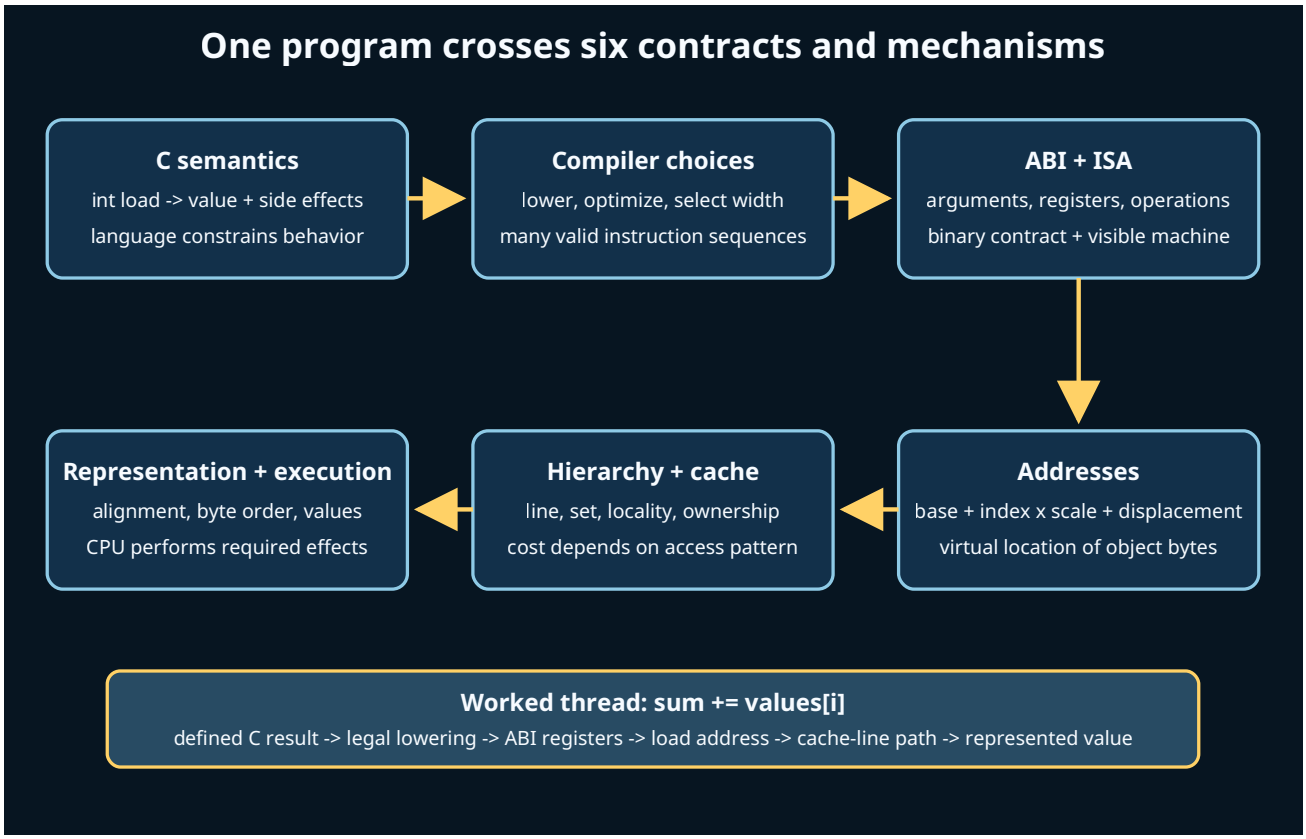
> Last week built upward from physical state to CPU control. This week begins at the software-visible machine: read its instructions, follow a function call, compare two ISAs, discover why memory speed depends on access pattern, and finally explain how C's typed abstract machine becomes operations on that hardware.

“Week” names a mastery module, not a seven-day deadline. A difficult lab may take two sessions; a familiar section may take less. The completion criterion is integrated evidence, not elapsed calendar time.

Why these seven days belong together

These topics can look like separate courses—assembly, caches, data layout, and C—but a running program crosses all of them:

```
C types, objects, and expressions
        ↓ compiler preserves required behavior
ISA instructions and ABI contracts
        ↓ name registers, operations, calls, and addresses
virtual and physical memory paths
        ↓ hierarchy exploits temporal and spatial locality
cache lines, sets, placement, and coherence
        ↓ concrete bytes have alignment and byte order
CPU executes generated operations on representations
```



Each boundary changes the vocabulary, but preserves a causal obligation from the layer above.

The causal order matters. Assembly makes architectural state visible. Calling conventions show that interoperability depends on contracts beyond the ISA. RISC-V comparison separates universal machine problems from x86-64's particular solutions. The memory hierarchy then explains why two correct instruction sequences can have very different costs. Alignment and byte order make object representation concrete. C's abstract machine finally explains which of those details the language specifies, which the implementation chooses, and which an optimizer may transform away.

Daily roadmap and exact files

Day 1 — x86-64 Assembly Foundations

Read registers, subregisters, memory operands, address calculations, flags, arithmetic, and branches. Hand-trace state and use GDB to step one instruction at a time.

Lesson:

```
02-assembly-memory-and-c/sections/day-001-x86-64-assembly-foundations.md
```

Challenges:

```
02-assembly-memory-and-c/challenges/day-001-routines.s
02-assembly-memory-and-c/challenges/day-001-driver.c
```

Evidence: three small routines, one instruction trace with registers and flags, and an annotated compiler-generated loop.

Day 2 — Stack Frames and Calling Conventions

Follow `call` and `ret`, draw activation records, apply the System V AMD64 ABI, distinguish caller- and callee-saved registers, and connect recursion to finite user and kernel stacks.

Lesson:

```
02-assembly-memory-and-c/sections/day-002-stack-frames-and-calling-conventions.md
```

Challenges:

```
02-assembly-memory-and-c/challenges/day-002-stack-frames-lab.c
02-assembly-memory-and-c/challenges/day-002-abi-probe.s
```

Evidence: a labelled stack frame, GDB backtrace/frame inspection, a detected ABI violation, and a measured stack-exhaustion experiment.

Day 3 — RISC-V Assembly and ISA Comparison

Read a load/store ISA, compare register and instruction conventions with x86-64, and distinguish architectural simplicity from the performance and compatibility of a full implementation.

Lesson:

```
02-assembly-memory-and-c/sections/day-003-risc-v-assembly-and-isa-comparison.md
```

Challenge:

```
02-assembly-memory-and-c/challenges/day-003-riscv-decoder.py
```

Evidence: the same small computation explained in x86-64 and RISC-V, decoded instruction fields, and a concise comparison grounded in actual operations rather than “RISC good, CISC bad.”

Day 4 — RAM, Locality, and the Memory Wall

Explain why registers, caches, DRAM, and storage form a hierarchy; separate latency from bandwidth; and measure sequential, strided, and random access without treating one benchmark as a universal hardware truth.

Lesson:

```
02-assembly-memory-and-c/sections/day-004-ram-locality-and-memory-wall.md
```

Challenges:

```
02-assembly-memory-and-c/challenges/day-004-memory-wall-lab.c
02-assembly-memory-and-c/challenges/day-004-plot.py
```

Evidence: reproducible benchmark parameters and output, a runtime versus working-set plot, and a written explanation of where locality and the memory wall appear.

Day 5 — Cache Organization

Calculate tag, set, and offset; distinguish lines, sets, and ways; classify misses; understand write policies; and observe false sharing and performance counter limitations.

Lesson:

```
02-assembly-memory-and-c/sections/day-005-cache-organization.md
```

Challenge:

```
02-assembly-memory-and-c/challenges/day-005-cache-lab.c
```

Evidence: one hand-derived mapping checked by code, compact-versus-padded measurements, and `perf` evidence or the exact environmental reason PMU counters were unavailable.

Day 6 — Alignment, Endianness, and NUMA

Inspect alignment and structure padding, distinguish object layout from wire format, reconstruct byte order safely, and relate memory-controller topology and NUMA placement to access cost.

Lesson:

```
02-assembly-memory-and-c/sections/day-006-alignment-endianness-and-numa.md
```

Challenges:

```
02-assembly-memory-and-c/challenges/day-006-layout-endian-lab.c
02-assembly-memory-and-c/challenges/day-006-topology-probe.sh
```

Evidence: predicted and observed structure offsets, a safe byte-order decode, host or VM topology observations, and an explanation of how layout affects correctness, ABI compatibility, and performance.

Day 7 — C's Execution Model and Data Types

Use C's abstract machine to distinguish translation from execution and objects from representations. Reason about integer types, rank, promotions, conversions, character bytes, floating-point properties, expression evaluation, and the first boundaries among scope, storage duration, and lifetime.

Lesson:

```
02-assembly-memory-and-c/sections/day-007-c-execution-model-and-data-types.md
```

Challenge:

```
02-assembly-memory-and-c/challenges/day-007-c-machine-model-lab.c
```

Evidence: predicted and observed type/conversion results, raw object-byte observations with appropriate scope, annotated `-O0`/`-O2` assembly, and an explanation of how type changes operation width, extension, or comparison.

Workload guide

The nominal budget is about 21 focused hours:

```
Day 1   3 h   assembly reading, routines, GDB, compiler loop
Day 2   3 h   calls, ABI, frames, unwinding, stack experiment
Day 3   3 h   RISC-V model, decoding, cross-ISA comparison
Day 4   3 h   hierarchy/locality model, benchmark, plot
Day 5   3 h   cache geometry, false sharing, counters
Day 6   3 h   layout, endianness, topology/NUMA
Day 7   3 h   C machine model, conversions, assembly comparison
```

This is a planning estimate, not a pace requirement. Adjust it:

- split GDB or benchmarking work across sessions if observation quality starts dropping;
- repeat a lab when prediction and result disagree;
- shorten familiar exposition, but still produce mastery evidence;
- preserve failed `perf`, topology, or cross-toolchain attempts when the failure accurately documents an environmental limitation;
- do not mark a day complete merely because its lesson was read.

Integrated weekly evidence

Keep each artifact small enough to review, but connect it to the same running model.

Explain

Without notes, tell this story:

1. an ISA defines software-visible state and operations;
2. an ABI adds calling and interoperability contracts;
3. x86-64 and RISC-V solve the same machine problems with different visible instruction styles;
4. loads and stores encounter a hierarchy whose cost depends on locality;
5. caches locate lines through tag, set, and offset and coordinate ownership at line granularity;
6. alignment, padding, and byte order shape stored representations;
7. C types constrain values and observable behavior while leaving many machine-level choices to the implementation and compiler.

Draw

Produce one connected diagram containing:

- a C expression with operand types;
- compiler lowering to a plausible instruction sequence;
- ABI argument/result locations;

- register and memory state;
- virtual address to cache-line path;
- line offset/set/tag;
- object bytes, alignment boundary, and byte order;
- labels distinguishing language rule, ABI contract, ISA rule, microarchitectural choice, and OS policy.

Also include a separate stack-frame drawing with return address, saved registers, arguments, locals, `rsp`, and optional `rbp`.

Observe

Preserve a compact evidence bundle:

- one GDB instruction trace and one frame/backtrace inspection;
- one x86-64/RISC-V comparison from source or decoded instructions;
- memory-wall benchmark data and plot;
- one cache mapping and repeated false-sharing measurements;
- structure offsets, object bytes, and observed endianness;
- topology/NUMA output with environment caveats;
- C limits/conversion output plus annotated unoptimized and optimized assembly;
- compiler, kernel, CPU/VM, and relevant tool versions.

An unavailable PMU, cross-compiler, or NUMA interface is valid evidence only when the exact failure and environment are recorded. It is not evidence for the hardware behavior the tool could not observe.

Build

Complete or extend the challenge work so that the final code demonstrates:

- one correct hand-written assembly routine and its C caller;
- one mechanically explained function call across the ABI;
- one decoded or modified RISC-V instruction;
- one reproducible memory-access experiment;
- one cache-address calculation or false-sharing variation;
- one safe layout/byte-order conversion;
- two C functions where a type change causes an explainable generated-code difference.

Extensions should preserve a prediction before execution and a written explanation afterwards.

Integrated weekly mastery checkpoint

Start with this C function:

```
long sum_positive(const int *values, size_t count);
```

Then, without relying on “the compiler handles it,” explain the complete route:

1. What do the parameter and result types require on this C implementation?

2. Where do the two arguments and return value travel under the System V AMD64 ABI?
3. Give a plausible x86-64 loop and identify its registers, load width, signed comparison, branch, and address calculation.
4. Give the equivalent RISC-V shape and explain its explicit load/store behavior.
5. Draw one function activation and the return path.
6. Explain which accesses have spatial or temporal locality.
7. For a stated cache geometry, split one array-element address into tag, set, and offset.
8. Explain how element alignment and host byte order affect representation without changing the mathematical sum.
9. Identify at least three places where another correct compiler could emit different instructions.
10. Explain why none of this supports “one C line equals one instruction.”

Mastery is demonstrated when the explanation crosses every boundary explicitly:

```
C semantics → compiler choice → ABI → ISA → addresses/layout
            → cache/hierarchy → observed execution
```

If an arrow is justified only by “the machine knows,” return to that day's lesson and identify the missing contract or mechanism.

End-of-module reflection

Record, in your own words:

- strongest understanding;
- remaining confusion;
- connection to Linux, kernel, virtualization, or current work;
- one result that contradicted your prediction;
- next revision action;
- whether to advance, repeat one lab, or extend the module.

Do not auto-fill confidence, completion, hours, or mastery. Those remain the learner's evidence-based assessment.

Day 1 — x86-64 Assembly Foundations

Curriculum alignment: tracker topic 8, *x86-64 Assembly Foundations*

Target time: 2–3 hours: 70 minutes concept and tracing · 60 minutes assembly/GDB lab · 30 minutes compiler annotation · 20 minutes mastery work

> Last week ended with instruction bytes changing architectural state. Today we stop looking at "the CPU" from a distance. If `RIP` points at one x86-64 instruction, which state can that instruction read, what can it change, and how do we prove it one instruction at a time?

Tracker contract

By the end you should be able to:

- read basic x86-64 assembly and track register values;
- explain memory operands and effective addresses;
- use GDB for instruction stepping;
- recognize general-purpose registers, moves, arithmetic, logic, flags, and basic branches;
- write three small routines, step through them, and annotate compiler-generated assembly.

The mastery checkpoint is exact:

> **Translate a simple C loop into a plausible assembly sequence.**

"Plausible" matters. A C statement has no unique assembly translation.

---

1. Why assembly exists between bytes and source

Machine-code bytes are excellent for a decoder and hostile to human memory. Early programmers entered numeric operation codes directly, then created *assemblers*: programs that map names such as `add`, labels such as `.loop`, and register spellings such as `%rax` into encodings.

```
human intent → C → compiler → assembly text → assembler → machine bytes
                                    ↑
                          today's working level
```

Assembly is not what the CPU reads. It is a textual notation for a particular ISA plus assembler features. Labels disappear into addresses or relocations; comments disappear; mnemonics become bytes.

Why descend to this level?

- A debugger stops at an instruction address even when source is unavailable.
- The kernel's entry paths and context switches must manipulate registers explicitly.

- A compiler bug, ABI violation, crash dump, or performance mystery is often visible only after source has been lowered.
- Virtualization saves and restores architectural state—the same state named here.

The goal is not to memorize the x86 manual. It is to learn to execute a small sequence on paper and verify the model on Linux.

2. Architectural state: the visible machine

General-purpose registers

x86-64 exposes sixteen 64-bit general-purpose registers:

```
rax rbx rcx rdx   rsi rdi rbp rsp   r8 r9 r10 r11 r12 r13 r14 r15
```

They are not physically guaranteed to be sixteen simple storage boxes inside a modern core. Register renaming may map them onto a larger internal file. They are *architectural names*: software-visible state the implementation must preserve.

Conventional roles come from instructions and the ABI:

- `rsp` is the architectural stack pointer used implicitly by `push`, `pop`, `call`, and `ret`;
- `rbp` can be a frame pointer, but is otherwise a general register;
- `rax` often carries integer return values;
- `rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9` carry the first six integer/pointer arguments under the System V AMD64 ABI.

Only `rsp`'s special instruction behavior belongs to the ISA. Argument and return roles are ABI agreements, studied tomorrow.

Subregisters and the zero-extension rule

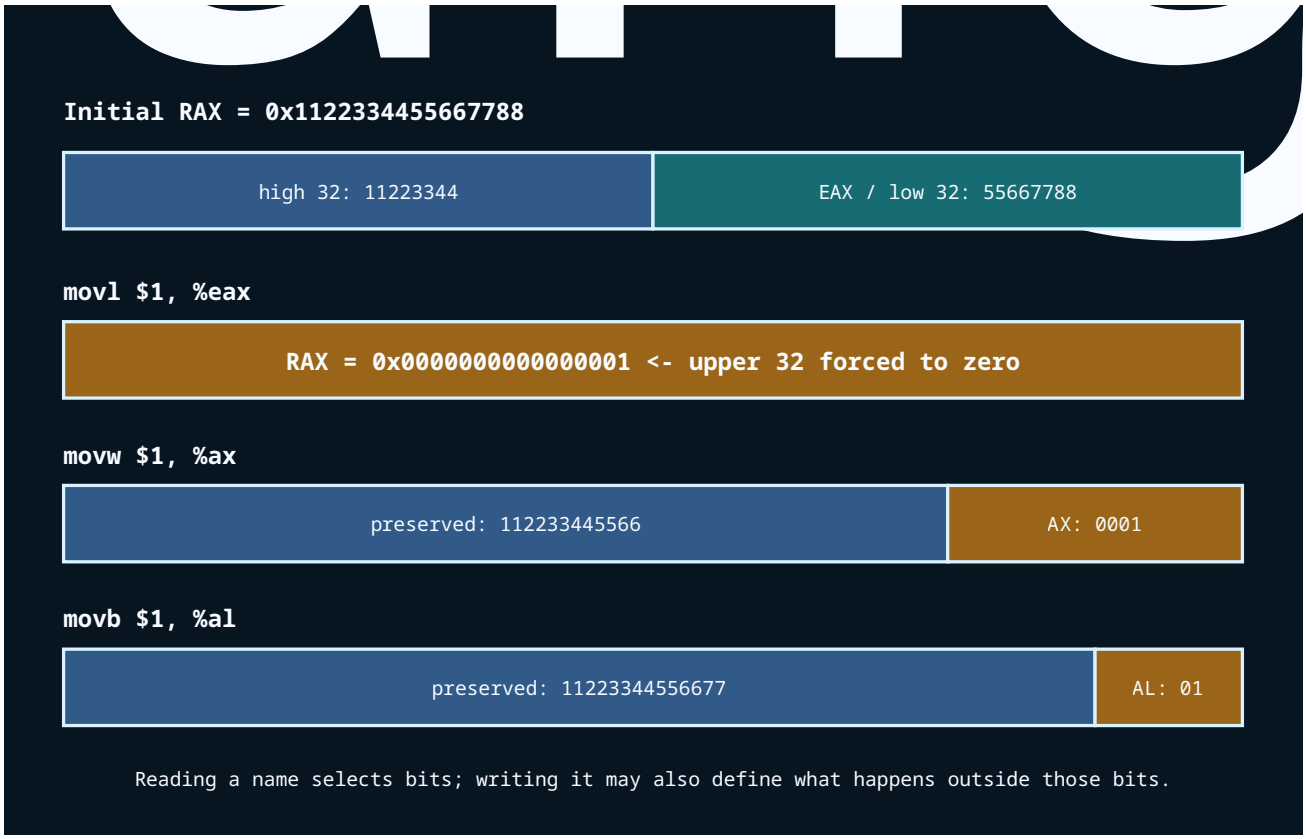
Parts of a register have historical names:

```
rax  64 bits
eax  low 32 bits
 ax  low 16 bits
 al  low  8 bits
 ah  bits 8–15 (legacy; avoid in new 64-bit examples)
```

Writing a 32-bit register zeros its upper half:

```
movl $1, %eax       # RAX becomes 0x0000000000000001
```

Writing `ax` or `al` preserves the other bits. Predicting this distinction is part of register tracking.



The destination spelling determines both the written slice and whether untouched high bits survive.

`RIP`: where fetch continues

`RIP` names the next architectural instruction location. Ordinary execution advances it by the current variable-length instruction's size. A branch, call, return, exception, interrupt, or system call replaces that ordinary next value.

You normally cannot use `mov %rax, %rip`. Control-transfer instructions update it according to defined rules. x86-64 also supports RIP-relative memory addressing, vital for position-independent code:

```
mov value(%rip), %eax
```

`RFLAGS`: recorded conditions

Arithmetic and logical instructions update selected status bits. The most useful today are:

- `ZF`: result was zero;
- `SF`: high result bit was one;
- `CF`: unsigned carry/borrow condition;
- `OF`: signed overflow;
- `PF` and `AF`: historically useful conditions that still exist.

Flags are state, not a hidden Boolean attached permanently to a value. Another flag-writing instruction replaces them.

Architecture lens — x86-64, AArch64, and RISC-V

x86-64 exposes sixteen general-purpose registers and variable-length instructions that may combine an arithmetic operation with one memory operand. AArch64 exposes thirty-one general-purpose registers in A64 state and fixed 32-bit load/store instructions; RISC-V exposes 32 integer registers including fixed-zero `x0`, with 32-bit base instructions and optional 16-bit compressed encodings. AArch64 has `NZCV` condition flags, while base RISC-V integer branches compare registers directly rather than reading an x86-like flags register.

Those register files, encodings, and instruction semantics belong to each ISA. Names such as x86-64 `rdi`, AArch64 `x0`, or RISC-V `a0` becoming the first argument are ABI choices; register renaming, decode strategy, and execution width are implementation choices. See Architecture Comparison.

3. Reading AT&T syntax without guessing

GNU tools often display AT&T syntax:

```
mnemonic source, destination
movq     %rsi, %rax
addq     $7, %rax
```

Conventions:

- `%` introduces a register;
- `$` introduces an immediate constant;
- suffix `b/w/l/q` means 1/2/4/8-byte operation;
- source precedes destination;
- parentheses denote memory addressed through registers.

Intel syntax commonly puts destination first and omits `%`/`$`. Both are textual views of the same possible bytes. Always state which one you use. This week uses AT&T in source and requests Intel flavor when helpful in GDB.

4. Data movement and memory operands

`mov` copies bits; it does not “move” an object

```
movq %rsi, %rax       # copy 64 bits RSI → RAX
movl (%rdi), %eax     # read 4 bytes at address RDI; zero upper RAX
movq %rax, 8(%rdi)    # write 8 bytes at address RDI+8
```

The source remains unchanged. A load is memory → register; a store is register → memory. Ordinary x86 `mov` does not permit both operands to be arbitrary memory locations.

Effective addresses

The general integer address form is:

```
displacement(base, index, scale)
effective address = displacement + base + index × scale
scale ∈ {1,2,4,8}
```

Example:

```
movl 12(%rdi,%rsi,4), %eax
```

If `rdi=0x1000`, `rsi=3`, the effective address is `0x1000 + 3×4 + 12 = 0x1018`. Four bytes are loaded from that address.

Do not conflate calculation with access. `lea 12(%rdi,%rsi,4), %rax` calculates `0x1018` and performs **no load**. Translation and permissions are consulted only when an actual memory access occurs.

Draw the path

```
instruction fields + base/index register values
                    ↓ address-generation logic
             virtual effective address
                    ↓ translation + permission + hierarchy
                bytes or fault
```

That last arrow connects assembly to Linux: a user load can trigger a page fault; the kernel may map a page and restart the same instruction.

5. Arithmetic and logic

```
addq %rsi, %rax       # rax = rax + rsi
subq $1, %rcx         # rcx = rcx - 1
imulq %rdx, %rax      # low 64-bit signed product into rax
andq $0xff, %rax      # preserve low byte
orq  %rsi, %rax
xorl %eax, %eax       # eax=0, therefore rax=0
shlq $3, %rax         # shift left three positions
```

Most have a read-modify-write destination and update flags. The width is part of the operation. A 32-bit `addl` wraps modulo (2^{32}) and zeros the upper half of its destination register; `addq` works modulo (2^{64}).

Signedness is not stored in a register. `add` produces the same bit sum for signed and unsigned interpretations; later code chooses `OF`-based or `CF`-based conditions.

6. Compare, test, and branch

`cmpq %rsi, %rdi` computes `rdi-rsi` for flags and discards the result.

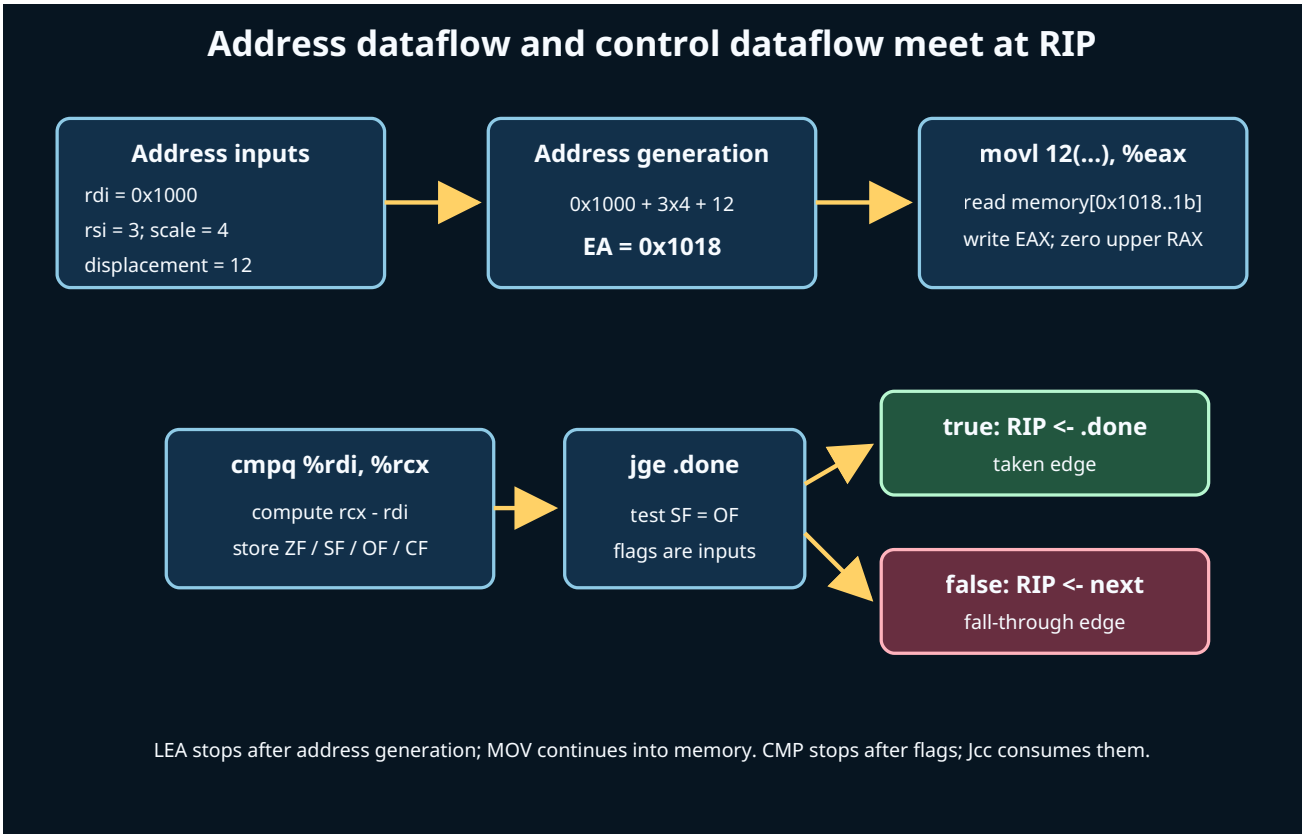
```
cmpq %rsi, %rdi
je   .equal           # ZF=1
jl   .signed_less     # SF != OF
jb   .unsigned_below  # CF=1
```

`testq %rax, %rax` computes an AND for flags without storing it. It is a common zero/sign test.

Trace this loop with `n=3` in `rdi`:

```
xorl %eax, %eax        # sum = 0
xorl %ecx, %ecx        # i = 0
.loop:
cmpq %rdi, %rcx        # i - n
jge  .done             # signed i >= n?
addq %rcx, %rax        # sum += i
incq %rcx              # i++
jmp  .loop
.done:
ret
```

Create a table with columns `RIP`, `RAX`, `RCX`, `RDI`, `ZF/SF/OF`, and “next instruction.” A loop is not one event. It is repeated state transitions and a backward next-`RIP` choice.



Address generation feeds a possible memory access; flag generation feeds a later control-flow decision.

7. Lab — write, predict, step

Use:

```
02-assembly-memory-and-c/challenges/day-001-routines.s
02-assembly-memory-and-c/challenges/day-001-driver.c
```

Build safely on Fedora x86-64:

```
cd /home/jd/Desktop/masterclass
gcc -std=c17 -Wall -Wextra -Wpedantic -Werror -O0 -g \
  -fno-omit-frame-pointer \
  02-assembly-memory-and-c/challenges/day-001-driver.c \
  02-assembly-memory-and-c/challenges/day-001-routines.s \
  -o /tmp/week2-day1
/tmp/week2-day1
```

Before running, hand-trace:

1. `asm_sum_to(4)`;
2. `asm_scale_add(10, 3, -2)`;
3. `asm_count_byte(bytes, count, 0x2a)`.

The files contain TODO prompts, not solutions to the extension tasks.

GDB instruction stepping

```
gdb -q /tmp/week2-day1
```

```
set disassembly-flavor intel
break asm_sum_to
run
display/i $pc
display/x $rax
display/x $rcx
display/x $rdi
si
info registers rip rax rcx rdi eflags
x/8i $pc
```

Use `si` for one machine instruction; `ni` steps over calls. Stop immediately before and after `cmp`, then before and after its conditional branch. Record the relevant values and whether `RIP` became the target or fall-through.

For the byte-count routine:

```
break asm_count_byte
run
x/12bx $rdi
si
```

When a load executes, compute its address before asking GDB:

```
p/x $rdi+$rcx
x/1bx $rdi+$rcx
```

Annotate compiler-generated assembly

The driver contains `c_sum_to`. Generate assembly at two optimization levels:

```
gcc -std=c17 -Wall -Wextra -O0 -S \
  02-assembly-memory-and-c/challenges/day-001-driver.c -o /tmp/day1-O0.s
gcc -std=c17 -Wall -Wextra -O2 -S \
  02-assembly-memory-and-c/challenges/day-001-driver.c -o /tmp/day1-O2.s
gcc -std=c17 -Wall -Wextra -O2 -g \
  02-assembly-memory-and-c/challenges/day-001-driver.c \
  02-assembly-memory-and-c/challenges/day-001-routines.s -o /tmp/day1-O2
objdump -d -Mintel --disassemble=c_sum_to /tmp/day1-O2
```

Annotate input register, loop state, compare, backward edge, and return register. If `-O2` transforms the loop, explain equivalence rather than searching for a one-to-one source line.

8. Kernel and virtualization connection

On exception or syscall entry, Linux assembly saves architectural registers into a kernel-defined frame. On return it restores enough state for user execution. A scheduler context switch preserves task state; KVM's vCPU state similarly models guest registers such as `RIP`, `RSP`, and `RFLAGS`.

This does **not** mean every internal CPU register is saved. Rename maps, reorder-buffer entries, and speculative work are microarchitectural. The architectural boundary is exactly why an OS or hypervisor can reason about the machine.

9. Mastery — Explain · Draw · Observe · Build

Explain

Without notes:

- register versus subregister, including a 32-bit write;
- `RIP`, `RSP`, `RBP`, and `RFLAGS`;
- load versus store versus `lea`;
- effective address calculation;
- why signed and unsigned branches differ;
- why assembly text is not machine code.

Draw

Draw one loop iteration:

```
RIP → fetch/decode → register reads → optional address/memory
    → ALU result/flags → register write → sequential or branch next RIP
```

Observe

Capture one GDB trace with:

- instruction bytes/text;
- before/after registers;
- effective address of one load;
- flags after `cmp`;
- branch's next `RIP`.

Build

Without looking at compiler output first:

1. write a plausible assembly sequence for:

```
c long sum_positive(const int *a, size_t n);
```

1. state its register/width assumptions;
2. assemble and call it from C;
3. test empty, mixed, and all-negative arrays;
4. only then compare with `gcc -O2 -S`.

That completes the tracker's “three routines,” debugger stepping, compiler annotation, and loop-translation evidence.

Why? notebook

1. Why does writing `eax` clear the upper half of `rax`?
2. Why is `rbp` sometimes a frame pointer and sometimes ordinary storage?
3. Why does `lea` use memory-address syntax without accessing memory?
4. Why can a valid load still enter the kernel?
5. Why does `cmp a,b` become `b-a` in AT&T syntax?
6. Why can two compilers emit different correct loops?

End model

```
assembly names an ISA operation
  → instruction reads architectural registers/immediates
  → optional effective address identifies memory
  → arithmetic/logical work changes result and flags
  → ordinary sequencing or a branch chooses next RIP
  → GDB lets us freeze and inspect each boundary
```

References used selectively

Local primary companion:

- Bryant and O'Hallaron, *Computer Systems: A Programmer's Perspective*, 3rd ed. global edition, Chapter 3: §§3.2–3.6, book pages 205–273 (encodings, registers/operands, data movement, arithmetic, condition codes, jumps, loops). Local PDF: `references/library/books/01-foundations-and-architecture/computer-systems-programmers-perspective-bryant-ohallaron-3e-global.pdf`

Current official references:

- Intel, *Intel® 64 and IA-32 Architectures Software Developer's Manual*, Vol. 1 Chapters 3 and 7, and Vol. 2 instruction reference: https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html
- GDB current manual, “Registers,” “Machine Code,” and “Stopping and Continuing”: https://sourceware.org/gdb/current/onlinedocs/gdb/

- Linux x86 entry documentation, connecting architectural registers to kernel entry: https://docs.kernel.org/arch/x86/entry_64.html

Next bridge: instructions define what `call` and `ret` do, but not where arguments arrive or which registers survive. Those promises belong to an ABI.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Driver

Format: C lab

What you will do: Use this driver exercise to test the section's model.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
gcc -std=c17 -Wall -Wextra -O0 -g -pthread 02-assembly-memory-and-c/challenges/day-001-driver.c 02-assembly-memory-and-c/challenges/day-001-routines.s -lm -o /tmp/day-001-driver && /tmp/day-001-driver
```

- View lab source on GitHub

Put it into practice — Routines

Format: assembly lab

What you will do: Week 2 Day 1: three small System V AMD64 routines.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
gcc -c -g 02-assembly-memory-and-c/challenges/day-001-routines.s -o /tmp/day-001-routines.o
```

- View lab source on GitHub

Day 2 — Frames and Contracts

Curriculum alignment: Stage 3 — Assembly, ABI, and Program Representation · Stack Frames and Calling Conventions

Target time: approximately 3 hours

- The problem that forced a stack to exist: about 25 minutes
- call, ret, and the anatomy of a frame: about 45 minutes
- The System V AMD64 ABI as a contract: about 40 minutes
- Labs: frames, the seventh argument, a broken promise, exhaustion: about 55 minutes
- Unwinding, kernel stacks, and mastery work: about 35 minutes

> Yesterday's routines took arguments in `rdi` and `rsi` and returned in `rax`. Nothing in the instruction set said they had to.
>
> `call` pushes a return address and jumps. `ret` pops and jumps. That is the entire architectural contribution to function calls. Everything else — which register holds argument one, who is allowed to destroy `rbx`, where local variables live, how a debugger reconstructs a backtrace — is agreement, not hardware.
>
> Today is about that agreement, and about what happens when someone breaks it.

---

1. Why a subroutine needs somewhere to put the return address

Start where the hardware starts. Tiny-8 had no `call`. If you wanted the same sequence of instructions used from two places, you copied it. That works, and it was genuinely how early programs were written, but it wastes storage and it means fixing a bug twice.

The fix seems obvious: jump to the shared code, and jump back afterwards. The difficulty is the word *back*. A jump instruction contains a fixed target. The shared routine does not know which of its callers is running.

The first widely used answer stored the return address *inside* the routine. On the EDSAC in the late 1940s, David Wheeler's technique — still called the Wheeler jump — had the caller place a return-jump instruction into a slot at the end of the subroutine before transferring control. The routine literally modified itself to know where to go home to. Some later machines gave this hardware support: a call instruction that stored the return address into a fixed location associated with the routine.

That design has one fatal property. Consider:

```
main calls compute
compute calls compute          (directly or through helper)
```

The second call overwrites the return address slot with an address inside `compute`. When the inner call returns, control comes back correctly. When the outer call tries to return, the slot no longer holds `main`'s address. The program returns into itself, forever.

So the requirement is sharper than "store the return address." It is:

> Store a return address **per activation**, not per routine, and reclaim that storage automatically when the activation ends.

Activations nest perfectly: the most recently started call is always the first to finish. A structure whose last item in is the first item out is a stack. This is not a coincidence or an implementation convenience — the nesting discipline of function calls *is* stack discipline, and the data structure follows from it.

Algol 60 made this unavoidable by specifying recursion as a language feature, and by the mid-1960s a hardware-supported stack with a dedicated stack pointer register was normal. x86-64 inherits that directly: `rsp`, `push`, `pop`, `call`, `ret`.

The same stack solves three problems at once

Once you have per-activation storage, you get more than return addresses:

```
per-activation return address    →  nested and recursive calls work
per-activation local variables   →  each call gets its own copies
per-activation saved registers   →  a callee can borrow a register and
                                    give it back
```

All three live in the same region, and together they are called a **stack frame** or **activation record**.

---

2. What `call` and `ret` actually do

These two instructions are simpler than their reputation.

```
call target       rsp ← rsp - 8
                  memory[rsp] ← address of the instruction after the call
                  rip ← target

ret               rip ← memory[rsp]
                  rsp ← rsp + 8
```

That is all. `call` is a push followed by a jump. `ret` is a pop into the program counter.

Two consequences follow immediately, and both matter.

First: `ret` does not verify anything. It jumps to whatever eight bytes sit at `rsp`. If your code wrote over those bytes, `ret` transfers control there with complete confidence. This is the mechanism behind stack-smashing attacks, and it is also why a corrupted stack produces a crash at an address with no relationship to the bug.

Second: the stack pointer is the only thing tying an activation to its storage. There is no header, no length field, no allocator metadata. The discipline is entirely in the generated code.

The stack grows downward

`push` *decreases* `rsp`. New frames occupy lower addresses than their callers.

```
higher addresses
    ┌──────────────────────────┐
    │  environment, argv       │
    ├──────────────────────────┤
    │  main's frame            │  ← started here
    ├──────────────────────────┤
    │  descend(5) frame        │
    ├──────────────────────────┤
    │  descend(4) frame        │
    ├──────────────────────────┤
    │  descend(3) frame        │  ← rsp points into the newest frame
    ├──────────────────────────┤
    │                          │
    │  unmapped                │  ← the kernel maps more on demand,
    │                          │     up to RLIMIT_STACK
lower addresses
```

Why downward? Historically, because putting the stack at the top of the address space and the heap just above the program's data let the two grow toward each other and share whatever was left, without either needing to know the other's size in advance. On a machine with megabytes of address space that mattered. On a 64-bit machine with a 47-bit user address space it no longer does, but the convention is baked into instruction semantics — `push` subtracts, and that is not something you can change.

You can confirm the direction in the lab. `descend(5)` prints a frame address for each recursion level, and every level is 96 bytes *below* the last:

```
  depth  5  frame 0x7ffd58a01ca0  return 0x000000400ad0  delta    -
  depth  4  frame 0x7ffd58a01c40  return 0x000000400663  delta   96
  depth  3  frame 0x7ffd58a01be0  return 0x000000400663  delta   96
  depth  2  frame 0x7ffd58a01b80  return 0x000000400663  delta   96
  depth  1  frame 0x7ffd58a01b20  return 0x000000400663  delta   96
```

Notice the return addresses, too. The outermost activation was called from `main` at `0x400ad0`; the other four were all called from the same recursive call site at `0x400663`. Four activations, one piece of code.

---

3. What is inside a frame

Here is `descend`'s prologue exactly as GCC 16 generated it at `-O0` with frame pointers on this machine:

```
descend:
  push   rbp                  ; save the caller's frame pointer
  mov    rbp,rsp              ; establish ours
  push   rbx                  ; borrow a callee-saved register
  sub    rsp,0x48             ; carve out 72 bytes of locals
```

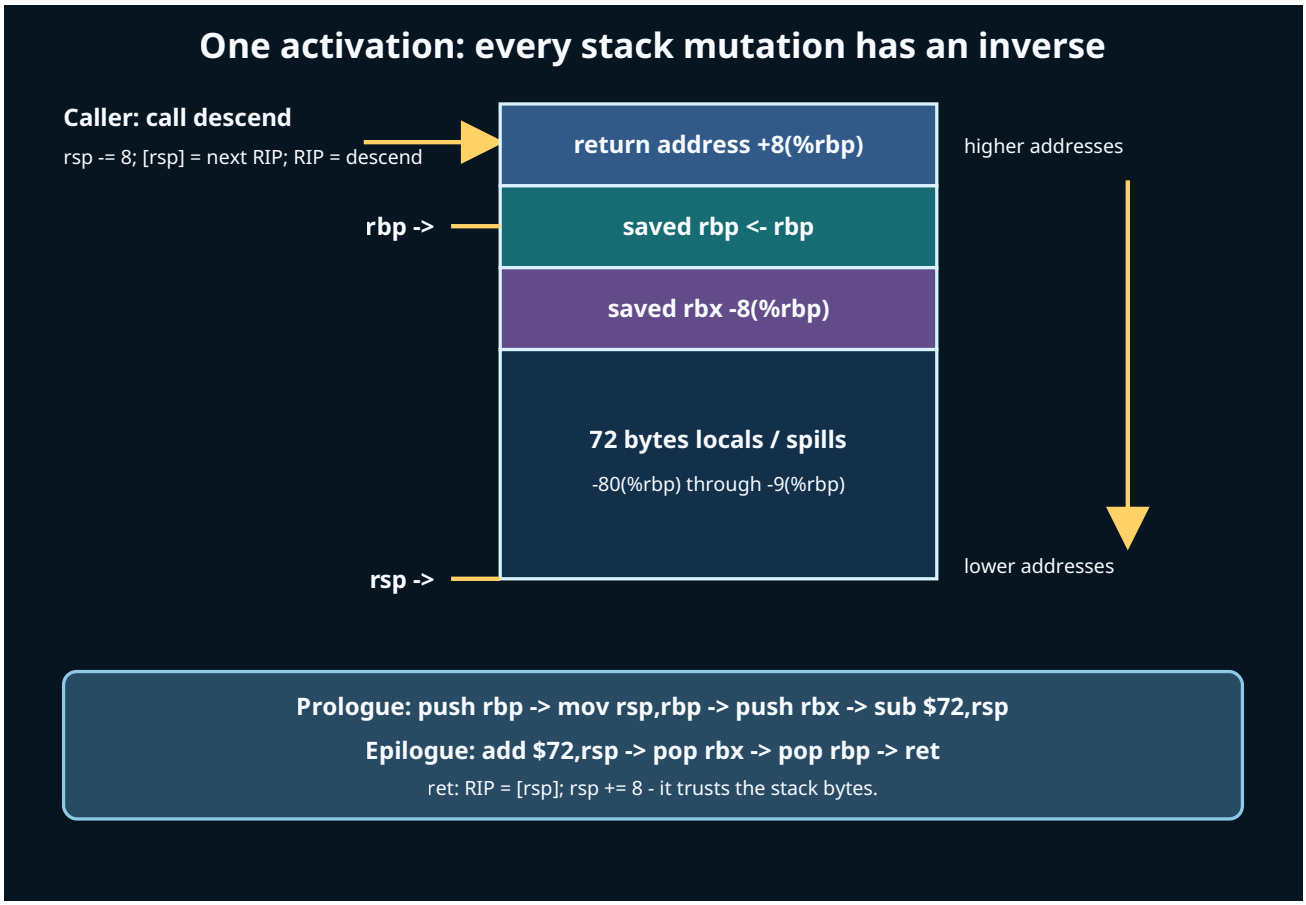
Add it up, from one `rbp` to the caller's `rbp`:

```
8   the return address pushed by call
8   the saved rbp
8   the saved rbx
72  locals and spill space (0x48)
--
96  bytes -- exactly the delta the lab measured
```

The general shape, drawn with higher addresses at the top:

```
                              higher addresses
    ┌──────────────────────────┐
    │  argument 8              │  pushed by the caller, if any
    │  argument 7              │
    ├──────────────────────────┤
    │  return address          │  ← pushed by call
    ├──────────────────────────┤
    │  saved rbp               │  ← rbp points here
    ├──────────────────────────┤
    │  saved rbx, r12..r15     │  only the ones this function uses
    ├──────────────────────────┤
    │  local variables         │
    │  spilled temporaries     │
    │  outgoing argument area  │
    └──────────────────────────┘  ← rsp
                              lower addresses
```

With `rbp` established, every local has a fixed negative offset from `rbp` and every stack argument has a fixed positive offset, regardless of how `rsp` moves afterwards. That is the entire point of a frame pointer.



The prologue creates one activation record; the epilogue reverses those mutations before `ret` consumes the saved address.

Reading it in GDB

```
gdb -q /tmp/day-002-stack-frames-lab
(gdb) break descend
(gdb) run
(gdb) continue 3
(gdb) backtrace
(gdb) info frame
```

On this machine that produces:

```
#0  descend (depth=2) at day-002-stack-frames-lab.c:61
#1  0x0000000000400663 in descend (depth=3) at day-002-stack-frames-lab.c:81
#2  0x0000000000400663 in descend (depth=4) at day-002-stack-frames-lab.c:81
#3  0x0000000000400663 in descend (depth=5) at day-002-stack-frames-lab.c:81
#4  0x0000000000400ad0 in main (argc=1, ...) at day-002-stack-frames-lab.c:249

Stack level 0, frame at 0x7fffffffdb70:
 rip = 0x40059c in descend; saved rip = 0x400663
 called by frame at 0x7fffffffdbd0
 Arglist at 0x7fffffffdb60, args: depth=2
 Locals at 0x7fffffffdb60, Previous frame's sp is 0x7fffffffdb70
 Saved registers:
  rbx at 0x7fffffffdb58, rbp at 0x7fffffffdb60, rip at 0x7fffffffdb68
```

Read that last block carefully. GDB is telling you the exact byte addresses where this activation stored the caller's `rbx`, the caller's `rbp`, and the return address. Three consecutive eight-byte slots, in the order the prologue pushed them. This is not GDB being clever — it is GDB reading a table the compiler emitted, which section 8 comes back to.

---

4. The ABI: a contract nobody can enforce

The instruction set says nothing about argument passing. So why does a function compiled by GCC work when called from code compiled by Clang, or from a library shipped as a binary five years ago, or from Rust, or from a hand-written `.s` file?

Because they all follow the same written agreement: the **System V AMD64 ABI**. An Application Binary Interface is the set of conventions that make separately compiled code interoperable. It is the reason `libc` is a single file rather than a per-compiler build.

The parts you need today:

Integer and pointer arguments

```
argument 1  rdi
argument 2  rsi
argument 3  rdx
argument 4  rcx
argument 5  r8
argument 6  r9
argument 7+ pushed on the stack, rightmost first, so that argument 7
            is at 8(%rsp) on entry to the callee
```

Floating-point arguments go in `xmm0`–`xmm7`, counted separately. A function can therefore have six integer and eight floating-point arguments all in registers.

Return values

```
integer or pointer      rax  (and rdx for a 128-bit value)
floating point          xmm0
large structs           the caller passes a hidden pointer in rdi to
                        space it allocated; the callee fills it in
```

Who owns which register

This is the part people get wrong, so state it precisely.

```
callee-saved ("the callee must give these back unchanged")
    rbx  rbp  r12  r13  r14  r15  rsp

caller-saved ("a call may destroy these; save them yourself if you care")
    rax  rcx  rdx  rsi  rdi  r8  r9  r10  r11
```

The split is an economic compromise. If every register were callee-saved, a tiny leaf function would pay to preserve registers nobody was using. If every register were caller-saved, a function with a long-lived value in a register would pay at every call. Splitting them lets the compiler put short-lived values in caller-saved registers and long-lived ones in callee-saved registers, and spill only what is genuinely live across a call.

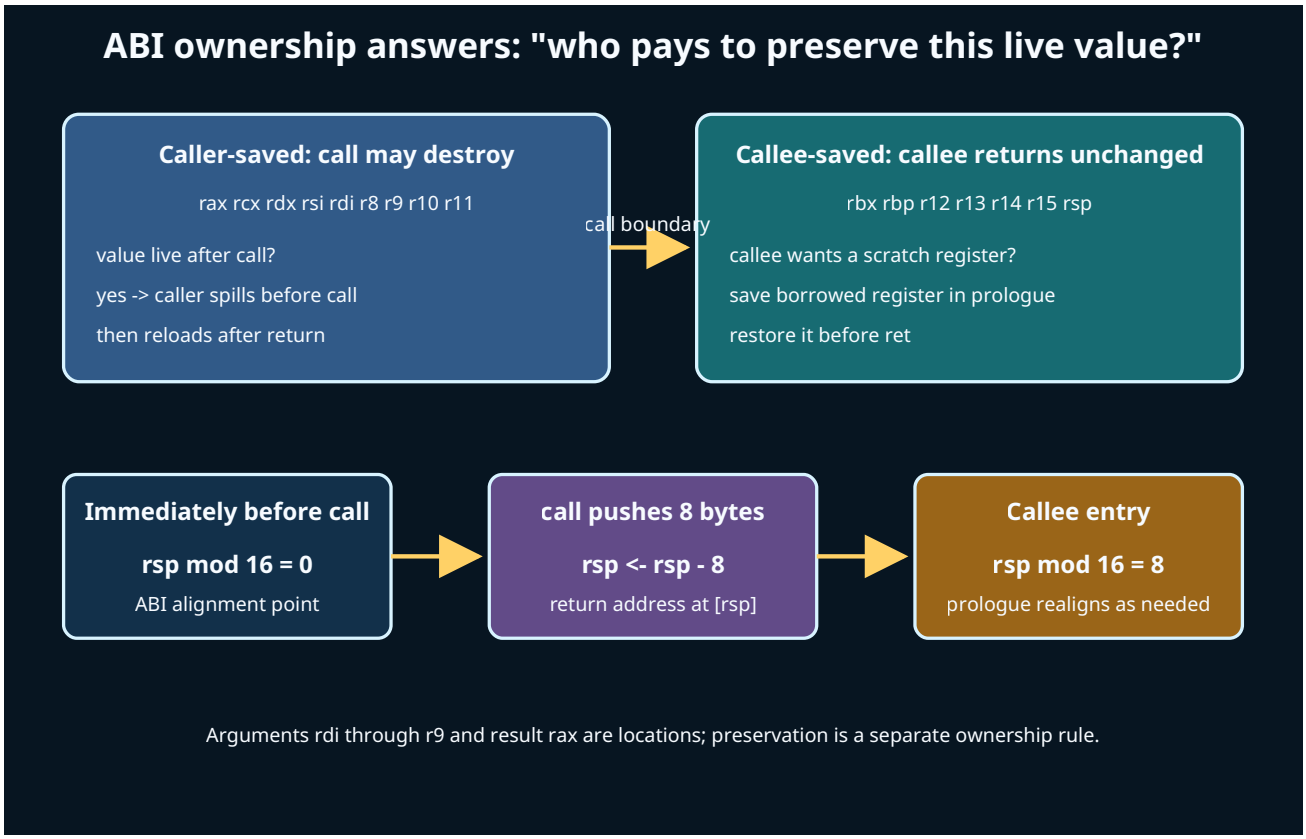
Stack alignment

`rsp` must be a multiple of 16 **immediately before** a `call` executes. Since `call` pushes eight bytes, the callee sees `rsp ≡ 8 (mod 16)` on entry.

The lab confirms this directly with a two-instruction routine:

```
  rsp inside asm_read_rsp: 0x7ffd58a01ce8
  rsp % 16 = 8
```

Why require alignment at all? Because SSE instructions that load sixteen bytes at once were historically much faster — and in some forms required — when the address was sixteen-byte aligned. Making it an ABI rule means a function can spill a vector register to its own frame without checking anything at runtime. It is a small tax paid everywhere to remove a check from the hot path.



Preservation responsibility and stack alignment are separate ABI promises that meet at the call boundary.

This is also the single most common way hand-written assembly breaks mysteriously. Push an odd number of registers before a `call` and the callee may fault inside a library function that has nothing to do with your code. The lab's `asm_probe_rbx` pushes exactly three registers for this reason, and the comment says so.

The red zone

A leaf function — one that calls nothing — may use the 128 bytes *below* `rsp` without adjusting `rsp` at all. The ABI guarantees that signal handlers and interrupts will not clobber that region.

This saves two instructions in small functions and is a nice example of an ABI making a promise that costs the kernel something: Linux's signal delivery code must skip the red zone when it builds a signal frame on the user stack. Kernel code itself is compiled with `-mno-red-zone`, because an interrupt arriving in kernel mode would happily scribble on it.

Architecture lens — x86-64, AArch64, and RISC-V

The ISA-level call primitive differs: x86-64 `call` pushes a return address, AArch64 `bl` writes link register `x30`, and RISC-V `jal` writes a selected destination register, conventionally `ra`. AArch64 and RISC-V non-leaf functions commonly spill the link register when another call would overwrite it; all three still need per-activation storage when values cannot remain in registers.

Argument registers, callee-saved sets, stack alignment, red zones, frame layout, and unwind rules are ABI contracts—System V AMD64, AAPCS64, and the RISC-V psABI make different choices. Whether a compiler keeps a frame pointer or spills a particular value is an implementation decision within that contract. See Architecture Comparison.

---

5. Lab A — frames, arguments, and a promise broken on purpose

Two files:

```
02-assembly-memory-and-c/challenges/day-002-abi-probe.s
02-assembly-memory-and-c/challenges/day-002-stack-frames-lab.c
```

Build with frame pointers so part 4 has a chain to follow:

```
cd 02-assembly-memory-and-c/challenges

gcc -std=c17 -Wall -Wextra -O0 -g -fno-omit-frame-pointer \
  day-002-abi-probe.s day-002-stack-frames-lab.c \
  -o /tmp/day-002-stack-frames-lab

/tmp/day-002-stack-frames-lab
```

Predict before running:

1. Will successive `descend` frame addresses increase or decrease?
2. What will `rsp % 16` be inside `asm_read_rsp`?
3. `asm_seven` reads its seventh argument from `8(%rsp)`. What is at `0(%rsp)`, and why must the routine not push anything before that read?
4. `asm_sum_squares` and `asm_bad_sum_squares` compute the same sum. Which observable difference will the probe find?

The one that matters

Part 5 is the heart of the lab. Two routines, identical arithmetic, and one of them omits `push %rbx` / `pop %rbx`. Both return the right answer:

```
  asm_sum_squares(5)     = 55
  asm_bad_sum_squares(5) = 55
  rbx after the well-behaved callee: 0x5a5a5a5a5a5a5a5a (marker intact)
  rbx after the careless callee:     0x0000000000000006 (CLOBBERED)
```

Sit with that. The broken routine is *correct* by any test that looks at return values. It produces the right number every time. It breaks its caller, silently, and only if the caller happened to be keeping something in `rbx`.

This is what ABI violations look like in practice. Not a crash at the point of the mistake — a wrong value somewhere else, in code that did nothing wrong, possibly only in an optimized build where the compiler chose to use `rbx`, possibly only on one compiler version. The whole reason to learn the callee-saved list is so you never generate this bug.

Then break it yourself

Add a fifth routine to the `.s` file that uses `r12` as a scratch register without saving it, and call it through `asm_probe_rbx` after modifying the probe to plant a marker in `r12` instead. Predict what you will see before you run it.

---

6. Lab B — running out of stack

Run the second mode:

```
/tmp/day-002-stack-frames-lab --overflow
```

A child process recurses until the kernel stops it. On this machine:

```
  RLIMIT_STACK soft limit: 8388608 bytes (8192 KiB)
  child reached depth 27557
  first frame 0x7ffcd5df44d0, last frame 0x7ffcd55f7210
  stack consumed 8377024 bytes, about 303 bytes per frame
  child terminated by signal 11 (Segmentation fault)
```

Check the arithmetic yourself: 27557 × 303 ≈ 8.35 MB, against a limit of 8.39 MB. The numbers close.

What the kernel actually did

The main thread's stack is not 8 MiB of physical memory sitting there waiting. It is a single virtual memory area that the kernel extends as needed:

1. The process touches an address just below the current stack VMA.
2. The MMU finds no mapping and raises a page fault — a synchronous exception, exactly the mechanism Week 1 Day 7 described.
3. The kernel's fault handler recognizes the address as a plausible stack extension and maps another page.
4. Execution resumes at the faulting instruction.

That loop runs tens of thousands of times in this demonstration and you never see it. Step 3 has a limit, though: `RLIMIT_STACK`. Once the VMA has grown to that size, the next fault is not a growth request, it is an error, and the kernel delivers `SIGSEGV`.

Note what did *not* happen: the stack did not silently run into the heap. Modern kernels keep a guard gap below the stack VMA so that a large stack allocation cannot jump over the guard and land in unrelated mapped memory — a real class of vulnerability, patched in 2017 and known as Stack Clash. This is also why GCC has `-fstack-clash-protection`, which forces large stack allocations to touch each page as they grow so the guard cannot be skipped.

Try the variations

```
(ulimit -s 1024; /tmp/day-002-stack-frames-lab --overflow)
(ulimit -s unlimited; /tmp/day-002-stack-frames-lab --overflow)
```

Predict the depth each time before you run it. Then check whether the kernel logged anything:

```
dmesg | tail
```

and compare against a `coredumpctl list` entry if core dumps are enabled on your system.

---

7. Recursion is not the only thing that consumes stack

A frame's size is set at compile time by the largest set of simultaneously live locals, spills, and outgoing arguments. Three things make frames large enough to matter:

- **Large local arrays.** `char buffer[65536];` inside a function allocates 64 KiB per activation. Eight nested calls like that exhaust a default thread stack.
- **Variable-length arrays and `alloca`.** These move `rsp` by a value the compiler does not know. If the length comes from untrusted input, the program can move `rsp` past the guard page in one step. This is why the Linux kernel removed VLAs from its own source tree.
- **Inlining.** A function that inlines three callees may need a frame as large as all four combined.

The kernel side is stricter still. A Linux kernel stack on x86-64 is `THREAD_SIZE` — 16 KiB, four pages — and it **does not grow**. There is no fault handler that can extend it, because the fault would arrive with no stack to handle it on. That single fact explains a family of kernel coding rules: no recursion, no large stack buffers, no VLAs, and separate stacks for interrupt and exception handling so a deep call chain interrupted at the wrong moment cannot overflow. `CONFIG_VMAP_STACK` adds a guard page below each kernel stack so an overflow faults instead of quietly corrupting a neighbouring task's data.

When you eventually write kernel code, "how much stack does this path use" becomes a question you have to be able to answer. Today's lab is where the instinct starts.

---

8. How a backtrace is reconstructed

The lab's part 4 walks the stack by hand, following saved `rbp` values:

```
manual walk from walk_level_two:
  level 0  rbp 0x7fffa2bc9640  return 0x000000400728
  level 1  rbp 0x7fffa2bc9650  return 0x000000400734
  level 2  rbp 0x7fffa2bc9660  return 0x000000400bb1
  level 3  rbp 0x7fffa2bc96a0  return 0x7f83884ab681
  level 4  rbp 0x7fffa2bc9750  return 0x7f83884ab798
  level 5  rbp 0x7fffa2bc97b0  return 0x000000400455
  chain ends here
```

Each frame's saved `rbp` points at the caller's `rbp`, and the return address sits eight bytes above it. That is a linked list through the stack, and walking it is twelve lines of C.

It only works if every function on the path maintained a frame pointer. Modern compilers default to `-fomit-frame-pointer` at `-O1` and above, because `rbp` is then available as a seventeenth general-purpose register — worth a few percent on register-hungry code. The lab is built with `-fno-omit-frame-pointer` specifically so the walk succeeds; try removing that flag and watch it stop after one level.

So how does GDB produce a backtrace for optimized code with no frame pointers? From a table. The compiler emits **DWARF Call Frame Information** into the `.eh_frame` section: for every instruction address in the function, a description of where the return address and each saved register can be found relative to the current `rsp` or `rbp`. Look at it:

```
readelf --debug-dump=frames-interp /tmp/day-002-stack-frames-lab | head -40
objdump --dwarf=frames-interp /tmp/day-002-stack-frames-lab | head -40
```

You will see rows that say, in effect, "from this address onward, the canonical frame address is `rsp+16`, and `rbx` is at CFA−24." That is the same information the frame-pointer chain encodes, except it is a table instead of runtime data, it is accurate at every instruction including in the middle of a prologue, and it costs zero registers.

The trade-off is that reading it is expensive. This is the live argument behind Linux's `CONFIG_UNWINDER_FRAME_POINTER` versus `CONFIG_UNWINDER_ORC`: the kernel wanted reliable stack traces for `perf` and for live patching without paying the frame-pointer register cost, so it generates its own compact unwind format (ORC) at build time rather than parsing DWARF in the kernel. And it is why `perf record --call-graph fp` and `--call-graph dwarf` give you different results on the same binary.

---

9. The return address is data, and that has consequences

One more implication of section 2 worth stating explicitly, because it connects this lesson to a large part of systems security.

The return address sits on the stack, immediately above the local variables, at a position the compiler chose. A local array is written forward, toward higher addresses. If code writes past the end of that array, the next things it reaches are the saved registers and then the return address.

```
┌──────────────────────┐
│  return address      │  ← ret jumps here, no questions asked
├──────────────────────┤
│  saved rbp           │
├──────────────────────┤
│  char buffer[64]     │  ← overflows upward, into the two above
└──────────────────────┘
```

The mitigations you will meet are all attempts to break one link in that chain:

- **Stack canaries** (`-fstack-protector-strong`) place a random value between the locals and the saved registers, and check it before `ret`.
- **Non-executable stack** (the `.note.GNU-stack` marker in the lab's `.s` files, and `PT_GNU_STACK` in the ELF header) stops the overwritten address from pointing at attacker-supplied instructions on the stack.
- **ASLR** randomizes the stack base so the attacker does not know the address to write. You can see this working: run the lab twice and compare the frame addresses.
- **Shadow stacks** (Intel CET, and `arch_prctl` support in recent Linux) keep a second, protected copy of return addresses and compare on `ret`.

Note that a plain `gcc` invocation on this machine enables none of these by default:

```
gcc -Q --help=common | grep stack-protector
nm /tmp/day-002-stack-frames-lab | grep -i stack_chk
```

Fedora's *package* builds add them through distribution build flags, which is a useful distinction to understand: hardening is a policy applied by whoever invokes the compiler, not a property of the compiler.

Rebuild the lab with `-fstack-protector-strong` and diff the disassembly of `descend`. You will find two new instructions in the prologue and three in the epilogue, and now `nm` shows `__stack_chk_fail`.

---

10. Mastery checkpoint

The tracker checkpoint is:

> **Explain function calls mechanically, identify arguments, return values, and preserved registers, draw a stack frame, and explain recursion and stack overflow.**

Explain

Without notes:

- why storing a return address per *routine* fails, and per *activation* works;
- exactly what `call` and `ret` do to `rsp` and `rip`;
- why the stack grows toward lower addresses, and whether that is a hardware fact or a convention;
- the difference between caller-saved and callee-saved, and why the ABI has both instead of one rule;
- what `rsp % 16` is on entry to a callee, and why the ABI demands alignment at all;
- how a frame pointer chain differs from DWARF call-frame information, and what each costs;
- why a Linux kernel stack cannot grow the way a user stack does.

Draw

Draw `descend`'s frame with byte offsets, labelling:

- the return address and which instruction placed it;
- the saved `rbp` and which instruction placed it;
- the saved `rbx`;
- the 72 bytes of locals;
- where `rbp` points and where `rsp` points;
- the arrow from this frame's saved `rbp` to the caller's `rbp`.

Then draw the same function's frame as it would look if compiled with `-fomit-frame-pointer`, and mark what a manual walker loses.

Observe

1. In GDB, break on `descend`, `continue 3`, and run `backtrace`, `info frame`, `frame 2`, `info args`. Confirm that GDB's "Saved registers" addresses match what the manual walk printed.
2. `x/6gx $rbp` and identify, by eye, the saved `rbp` and the return address.
3. Run the program twice and compare stack addresses. Explain the difference.
4. Run `--overflow` under three different `ulimit -s` values and show that depth scales with the limit.
5. `readelf --debug-dump=frames-interp` on the binary; find the entry for `descend` and match one row against the prologue instructions.

Build

1. Add `asm_clobbers_r12` to the probe file and detect it.
2. Write `asm_seven` yourself, from scratch, without looking at the existing version, and get it passing.
3. Write a hand-written assembly function that calls `printf`. Getting the alignment right is the exercise; getting it wrong is instructive.
4. Modify `descend` to declare `char pad[4096];` and re-measure the frame delta. Predict the new number first.
5. Rebuild with `-fstack-protector-strong` and identify every added instruction.

Why? notebook

1. Why does the Wheeler-jump approach break under recursion but work fine for non-recursive programs?
2. Why is `rsp` in the callee-saved list when no function explicitly saves it?
3. Why does the seventh argument live at `8(%rsp)` and not `0(%rsp)`?
4. Why is the red zone safe from signal handlers but not from kernel interrupts?
5. Why does a broken callee-saved register produce a bug far from its cause?
6. Why does `-O2` usually omit the frame pointer, and what did the kernel do instead of accepting that trade-off?
7. Why can a VLA with an untrusted length skip a guard page when a fixed array cannot?
8. Why does the lab's overflow demonstration need a child process?
9. If `ret` jumps wherever `rsp` points, why do programs not crash constantly?

Mental model at the end

```
caller has a value it needs after the call
        ↓ is it in a caller-saved register?
        ↓ yes → caller spills it to its own frame first
caller places arguments: rdi rsi rdx rcx r8 r9, then the stack
        ↓ rsp is a multiple of 16 at this instant
call
        ↓ pushes the return address, rsp now ≡ 8 (mod 16)
callee prologue
        ↓ push rbp; mov rbp,rsp        (if it keeps a frame pointer)
        ↓ push any callee-saved register it intends to use
        ↓ sub rsp, N                   (locals, spills, outgoing args)
callee body
        ↓ locals at negative offsets from rbp
        ↓ stack arguments at positive offsets from rbp
callee epilogue
        ↓ restores rsp, pops the callee-saved registers it pushed
ret
        ↓ pops the return address into rip
caller continues, and every callee-saved register holds what it did before
```

The instruction set contributes two lines of that diagram. Everything else is a contract, written down in a document, obeyed by every compiler, and enforced by nothing at all.

That is worth carrying forward. Tomorrow you meet a different instruction set with a different register file and a different ABI — and you will find that the *shape* of that diagram is nearly identical, because the problem it solves has not changed.

References used selectively

- *System V Application Binary Interface, AMD64 Architecture Processor Supplement*, sections 3.2 (function calling sequence), 3.2.2 (the stack frame), 3.2.3 (parameter passing), and Figure 3.4 (the register-usage table). This is the authoritative document for everything in section 4: https://gitlab.com/x86-psABIs/x86-64-ABI
- Bryant and O'Hallaron, *Computer Systems: A Programmer's Perspective*, 3rd ed., section 3.7, "Procedures," for the runtime stack, transferring control, argument passing, local storage, and recursion. Local PDF: `references/library/books/01-foundations-and-architecture/computer-systems-a-programmers-perspective-3rd-edition.pdf`
- Michael Kerrisk, *The Linux Programming Interface*, Chapter 6, for the process memory layout and the relationship between the stack, `RLIMIT_STACK`, and virtual memory areas.
- Linux kernel documentation and source for `THREAD_SIZE`, `CONFIG_VMAP_STACK`, and the ORC unwinder (`Documentation/arch/x86/orc-unwinder.rst`), read for orientation.
- DWARF Debugging Information Format, Version 5, Section 6.4, "Call Frame Information," for what `.eh_frame` contains: https://dwarfstd.org/

Every address, frame size, recursion depth, and GDB transcript quoted above was captured on this machine with GCC 16.2.1 and GDB 17.2. Stack addresses change on every run because of ASLR; frame *sizes* will match if you use the same compiler and flags.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Abi Probe

Format: assembly lab

What you will do: Day 2 challenge A — hand-written routines that expose the ABI contract.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
gcc -c -g 02-assembly-memory-and-c/challenges/day-002-abi-probe.s -o /tmp/day-002-abi-probe.o
```

- View lab source on GitHub

Put it into practice — Stack Frames Lab

Format: C lab

What you will do: Day 2 challenge B — watch frames appear, watch the ABI hold, watch it

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
gcc -std=c17 -Wall -Wextra -O0 -g -pthread 02-assembly-memory-and-c/challenges/day-002-stack-frames-lab.c 02-assembly-memory-and-c/challenges/day-002-abi-probe.s -lm -o /tmp/day-002-stack-frames-lab && /tmp/day-002-stack-frames-lab
```

- View lab source on GitHub

Day 3 — RISC-V, Seen Through a Decoder

Curriculum alignment: tracker Week 10, *RISC-V Assembly and ISA Comparison*

Target time: 2–3 hours: 65 minutes building the RV64I model · 40 minutes comparing equivalent sequences · 50 minutes decoder/trace laboratory · 20 minutes mastery evidence

> Yesterday, x86-64 `call` placed a return address on the stack and `ret` removed it. Today a RISC-V function call will place the return address in a register, and `ret` will turn out not to be a separate base instruction at all.
>
> The software problem did not change. The architectural choices did.

Tracker contract

By the end, you should be able to:

- read basic RV64I assembly and track integer registers;
- explain the special behavior of `x0`;
- distinguish register arithmetic from explicit loads and stores;
- recognize the common R, I, S, B, U, and J 32-bit forms;
- reconstruct signed immediates, including a branch immediate whose bits are not adjacent in the instruction;
- compare RISC-V and x86-64 without reducing either to “simple” or “complex”;
- relate ISA choices to compiler output, kernel entry, and virtual CPU state;
- decode and execute a small RISC-V loop without requiring a cross-toolchain.

The tracker mastery checkpoint says:

> **Explain why RISC-V is easier to reason about while x86 retains compatibility advantages.**

That sentence is a useful starting claim, not a verdict. We will make “easier,” “compatibility,” and the costs on both sides precise.

---

1. The same problem, a deliberately different contract

An ISA must let software:

```
name working state
    → transform values
    → move values between registers and memory
    → choose a next instruction
    → enter and return from functions
```

x86-64 and RISC-V both solve that list. They allocate encoding space, architectural state, and implementation work differently.

RISC-V began at UC Berkeley in 2010, after decades of experience with earlier RISC designs. Its designers were not trying to rediscover the smallest possible CPU. They wanted an open, extensible ISA suitable for research, teaching, tiny embedded implementations, and high-performance systems. The base integer ISA is intentionally small; multiplication, atomics, floating-point, vectors, compressed encodings, and other facilities are organized as extensions.

"RISC-V" therefore does not name one fixed collection of every possible instruction. `RV64I` means:

```
RV       RISC-V
64       64-bit integer register width and address-space model
I        base integer instruction set
```

A real Linux-capable machine supports additional extensions. The lesson starts with RV64I because it exposes the architectural skeleton clearly.

Three objects that must stay separate

```
RISC-V specification   defines encodings and architectural behavior
instruction word       32 bits fetched from one particular program
RISC-V processor       hardware or a software model implementing the contract
```

The Python challenge is in the third category: it is a deliberately partial software implementation. It can demonstrate decode and architectural state transitions. It says nothing about how many pipeline stages, execution units, or cycles a physical processor uses.

---

2. Thirty-two integer registers, and one that refuses to change

RV64I has 32 integer registers, each 64 bits wide:

```
x0 x1 x2 ... x31
```

Five bits can name one of 32 registers because (2^5=32). That fact will become visible in every common 32-bit instruction format.

All 32 names are architectural. A high-performance implementation may use register renaming and many more physical registers internally, just as an x86-64 implementation may. The ISA promises the behavior of `x0`–`x31`, not one literal bank of 32 flip-flop arrays.

`x0`: read zero, discard writes

`x0` always reads as zero. A write to `x0` has no architectural effect:

```
addi x5, x0, 12      # x5 = 0 + 12
add  x0, x5, x6      # addition occurs architecturally; result is discarded
```

The second instruction does **not** permanently make `x0` nonzero. The next read still yields zero.

Why spend one of 32 encodings on a fixed value?

It lets ordinary instruction forms express several common operations without dedicated opcodes:

```
addi x5, x0, 7       # load small constant 7
sub  x6, x0, x5      # negate x5
addi x7, x5, 0       # copy x5
jal  x0, target      # jump, discarding the link address
jalr x0, 0(x1)       # return through x1, discarding a new link
beq  x5, x0, empty   # compare with zero
```

Assemblers provide pseudoinstructions such as `li`, `mv`, `neg`, `j`, and `ret` for these patterns. A pseudoinstruction is assembler vocabulary, not necessarily an extra opcode in the ISA.

There is a cost: only 31 registers can retain arbitrary values. The design judgment is that a permanently available zero simplifies enough common sequences and hardware operand selection to justify that cost.

Two names for the same register

RISC-V assembly normally uses ABI names:

```
x0  zero              constant zero
x1  ra                return address
x2  sp                stack pointer
x5–x7, x28–x31 t0–t6  caller-saved temporaries
x8–x9, x18–x27 s0–s11 callee-saved values
x10–x17 a0–a7         arguments; a0/a1 also return values
```

`x8` is called `s0` and may also be displayed as `fp` when used as a frame pointer. The names do not create extra storage:

```
x10 = a0
x11 = a1
x1  = ra
```

The ISA assigns the number. The ELF psABI assigns the conventional role. RISC-V hardware does not reject a program for putting an argument in `t3`; separately compiled functions simply will not interoperate unless they obey the same ABI.

Retrieval check

Without looking back:

1. How many bits are needed to name one integer register?
2. What value does `x0` return after an attempted write?
3. Is `a0` a second register in addition to `x10`?
4. Which facts above belong to the ISA, and which belong to the ABI?

---

3. Load/store means arithmetic names registers, not memory

RV64I integer arithmetic uses register operands. Memory is reached through explicit load and store instructions.

```
ld   t0, 0(a0)       # read 8 bytes at address a0+0 into t0
addi t0, t0, 7       # t0 = t0 + 7
sd   t0, 0(a0)       # write low 8 bytes of t0 to address a0+0
```

That sequence implements the essential work of:

```
*p += 7;
```

The address form is:

```
effective address = integer register rs1 + sign-extended 12-bit immediate
```

The immediate is measured in bytes. `24(sp)` means an address 24 bytes above the current stack pointer; it does not mean register 24.

Load width and extension are part of the operation

RV64I has several integer loads:

```
lb   load 1 byte, sign-extend to 64 bits
lbu  load 1 byte, zero-extend to 64 bits
lh   load 2 bytes, sign-extend
lhu  load 2 bytes, zero-extend
lw   load 4 bytes, sign-extend
lwu  load 4 bytes, zero-extend
ld   load 8 bytes
```

Stores `sb`, `sh`, `sw`, and `sd` write the low 1, 2, 4, or 8 bytes of the source register. A store does not first sign-extend the source. It selects low bits and transfers those bytes.

Predict:

```
memory[0x100] = 0xff
a0 = 0x100
```

After `lb t0, 0(a0)`, `t0` is `0xffffffffffffffff`. After `lbu t1, 0(a0)`, `t1` is `0x00000000000000ff`.

The memory byte is identical. The instruction tells the machine how to widen it.

Address construction is visible

Suppose `a0` is a `uint64_t *base` and `a1` is an array index:

```
slli t0, a1, 3       # byte offset = index × 8
add  t0, a0, t0      # address = base + byte offset
ld   a0, 0(t0)       # load the element
```

The load itself has only base-plus-12-bit-displacement addressing. Scaling and adding the index happen in earlier register operations.

x86-64 can put this calculation in one memory operand:

```
movq (%rdi,%rsi,8), %rax
```

The RISC-V sequence makes intermediate values explicit. The x86 instruction makes a common address calculation compact. Neither observation by itself tells us the number of cycles. A RISC-V implementation can execute the shift and add efficiently; an x86 implementation can decode its instruction into internal work. Cache behavior can dominate both.

A load is not “just copying bytes”

Architecturally, the load describes an effective virtual address and result. In a Linux process:

```
rs1 + immediate
    → virtual address
    → address translation and permission check
    → cache/memory access, or a page/access/alignment fault
    → destination register updated only if the instruction completes
```

This is the same causal path studied on x86-64. The syntax changed. The need for translation, protection, restartable faults, and precise architectural state did not.

---

4. Six common 32-bit forms

In the base ISA, the ordinary instruction length is 32 bits. The low seven bits hold the major opcode. Register fields stay in stable locations where a format uses them:

```
rd   bits 11:7
rs1  bits 19:15
rs2  bits 24:20
```

That regularity lets a decoder begin extracting register numbers before it has resolved every operation detail.

The optional compressed `C` extension adds 16-bit encodings. Longer instruction encodings are also defined by RISC-V's variable-length framework. Therefore “every RISC-V instruction is 32 bits” is false for RISC-V in general. It is accurate for the base RV64I forms studied here when compressed and other variable-length extensions are excluded.

R-type — three register operands

```
31       25 24    20 19    15 14  12 11     7 6       0
+----------+--------+--------+------+---------+---------+
| funct7   | rs2    | rs1    |funct3| rd      | opcode  |
+----------+--------+--------+------+---------+---------+
```

Examples:

```
add t0, t1, t2       # t0 = t1 + t2
sub a0, a0, a1
and t3, t3, t4
slt a0, a0, a1      # signed less-than produces 0 or 1
```

The seven-bit major opcode identifies an operation family. `funct3` and `funct7` distinguish operations within that family. “Opcode” is therefore often used informally for more than one encoded field.

I-type — one register, one signed immediate

```
31                20 19    15 14  12 11     7 6       0
+-------------------+--------+------+---------+---------+
| imm[11:0]         | rs1    |funct3| rd      | opcode  |
+-------------------+--------+------+---------+---------+
```

Examples:

```
addi sp, sp, -32     # allocate 32 stack bytes
andi a0, a0, 255
ld   ra, 24(sp)      # loads use an I-shaped encoding
jalr x0, 0(ra)       # indirect jump also uses I-type fields
```

For `addi`, the 12-bit immediate is sign-extended to the register width. Its range is -2048 through +2047. `addi a0, zero, 4096` cannot be encoded as one base instruction.

Shift-immediate operations reuse some upper immediate bits to distinguish logical from arithmetic shifts. A field diagram is a family pattern, not a license to treat every bit as data.

S-type — store, with a split immediate

```
31       25 24    20 19    15 14  12 11      7 6       0
+----------+--------+--------+------+----------+---------+
|imm[11:5] | rs2    | rs1    |funct3| imm[4:0] | opcode  |
+----------+--------+--------+------+----------+---------+
```

Example:

```
sd ra, 24(sp)
```

A store needs:

- `rs1`: address base;
- `rs2`: value to write;
- immediate: address displacement.

It does not need `rd`, because a store has no integer destination register. The five bits normally occupied by `rd` become part of the immediate. This is encoding budgeting made visible.

B-type — compare two registers and branch

```
31      30:25 24:20 19:15 14:12 11:8    7 6:0
+---------+-----+-----+-----+------+-------+---+-------+
|imm[12]  |10:5 | rs2 | rs1 |funct3| imm[4:1]|11|opcode|
+---------+-----+-----+-----+------+-------+---+-------+
```

The exact drawing is crowded because the immediate is rearranged:

```
imm[12]   comes from instruction bit 31
imm[10:5] comes from bits 30:25
imm[4:1]  comes from bits 11:8
imm[11]   comes from bit 7
imm[0]    is implicitly zero
```

Examples:

```
beq  t0, zero, done
bne  a0, a1, retry
blt  t0, t1, loop       # signed comparison
bltu t0, t1, loop       # unsigned comparison
```

Why scramble the immediate instead of placing it in one neat range? The register fields and `funct3` remain in the same positions as related formats. The immediate bits are arranged so hardware can reconstruct the displacement while preserving those stable positions. The low target bit is known to be zero under the base alignment model, so it need not consume an encoded bit.

This is not beautiful for a human decoder. It is a trade: regular register field placement and encoding overlap versus contiguous immediate notation.



Stable register fields and reconstructed immediates make the encoding budget visible in a worked decode.

U-type — twenty upper bits

```
31                                    12 11     7 6:0
+----------------------------------------+---------+-------+
| immediate[31:12]                        | rd      |opcode |
+----------------------------------------+---------+-------+
```

`lui` places the 20 encoded bits into bits 31:12 and zeros the low 12 bits. On RV64, the 32-bit result is sign-extended to 64 bits.

`auipc` adds that upper immediate to the current PC. Combined with another instruction, it supports PC-relative address formation over a much larger range than a 12-bit immediate.

J-type — jump and link

`jal rd, offset` writes `pc+4` to `rd` and transfers to `pc+offset`. Like the branch immediate, its encoded displacement is rearranged and has an implicit zero low bit.

```
jal ra, function      # ordinary direct call: save return PC in ra
jal x0, loop          # jump: discard return PC
```

The link destination is explicit. `jal` is not inherently “a call” or “a jump”; the destination register and software convention give it that role.

Why fixed 32-bit forms are attractive

For this subset, a front end knows:

```
next sequential PC = current PC + 4
```

Field positions are regular, and several candidate values can be extracted in parallel. This can simplify teaching, implementation, verification, and binary rewriting.

But fixed width does not make design constraints disappear:

- every simple operation still consumes four bytes;
- three 5-bit registers consume 15 of the 32 bits;
- large constants require more than one instruction;
- a large address calculation often takes several instructions;
- extensions need reserved encoding space or longer/alternative forms.

The compressed extension exists partly because code density matters to instruction-cache capacity, fetch bandwidth, memory footprint, and energy.

---

5. Immediates: small constants and displaced control flow

An immediate is a bit field inside the instruction, not a miniature memory object. The decoder reconstructs it according to that instruction's format.

Sign extension

For:

```
addi a0, a0, -1
```

the immediate field contains:

```
12-bit representation: 111111111111
sign-extended to 64:    111...111111111111
numeric value:          -1
```

The ALU adds that 64-bit pattern modulo (2^{64}). Registers do not carry a permanent “signed” tag.

Why one constant may need two instructions

A common conceptual sequence for a larger constant is:

```
lui  t0, upper20
addi t0, t0, lower12
```

The exact split must account for `addi` sign extension. If bit 11 of the low part is set, the assembler may round the upper part and use a negative lower immediate. This is why `li t0, 0x12345678` is a pseudoinstruction: the assembler chooses a legal sequence rather than replacing one universally defined `li` opcode.

For arbitrary 64-bit constants, more instructions may be needed.

Branch targets

Suppose:

```
branch PC = 0x0000000000000040
decoded immediate = -12
condition = true
```

Then:

```
target = branch PC + (-12) = 0x34
```

The immediate is not an absolute address. It is a signed PC-relative byte displacement. Its encoded low bit is implicit.

Do not silently substitute x86's branch rules. Both architectures commonly use PC-relative control flow, but their encodings and defined PC bases must be read from their own specifications.

---

6. Branches without an integer flags register

x86-64 often separates comparison from conditional transfer:

```
cmpq %rsi, %rdi
jl   .less
```

`cmp` writes condition state into `RFLAGS`; `jl` reads the relevant flags.

RV64I integer branches compare registers as part of the branch:

```
blt a0, a1, .less
```

The base integer architecture has no x86-like general condition-code register for this sequence.

Why might a design choose each model?

Flags can be useful because:

- one comparison can feed a following condition;
- arithmetic instructions can naturally record carry, zero, sign, and overflow conditions;
- an existing architecture and its software already rely on them.

Compare-and-branch can be useful because:

- the data dependencies are explicit in one instruction;
- unrelated arithmetic does not accidentally overwrite an earlier condition;
- hardware and compilers need not track a shared integer flags register for ordinary branches.

But do not leap from this to “RISC-V branches are always faster.” A microarchitecture chooses its own branch units, prediction, fusion, and scheduling. Modern x86 implementations may macro-fuse common compare/branch pairs. Architectural style influences implementation; it does not dictate a single performance result.

Signed and unsigned are selected by the instruction

Let:

```
t0 = 0xffffffffffffffff
t1 = 0x0000000000000001
```

Then:

```
blt  t0, t1, target    # taken: signed -1 < signed 1
bltu t0, t1, target    # not taken: UINT64_MAX < 1 is false
```

The register bits did not change. The operation selected their interpretation.

---

7. Calls: same nesting problem, different architectural primitive

RISC-V direct calls normally use:

```
jal ra, function
```

Architecturally:

```
ra (x1) ← address of the instruction after jal
pc      ← current pc + signed displacement
```

An indirect call can use:

```
jalr ra, 0(t0)
```

A return is conventionally:

```
jalr x0, 0(ra)
```

which the assembler displays through the `ret` pseudoinstruction.

Compare yesterday's x86-64 mechanism:

```
x86 call    pushes return address to memory at the stack
RISC-V jal  writes return address to an explicitly named register
```

The RISC-V choice makes a leaf call possible without touching memory for its return address. But a non-leaf function will call another function, and that new call overwrites `ra`. The current function must preserve its incoming return address, commonly in its stack frame:

```
example:
    addi sp, sp, -16
    sd   ra, 8(sp)
    sd   s0, 0(sp)
    # ... body, perhaps another call ...
    ld   s0, 0(sp)
    ld   ra, 8(sp)
    addi sp, sp, 16
    ret
```

The exact frame is compiler- and function-dependent. The point is mechanical: nesting still requires one preserved return address per live activation. RISC-V starts that address in `ra`; software spills it when necessary. x86-64 starts it on the stack.

Calling-convention overview

Under the RISC-V ELF psABI:

- `a0`–`a7` carry integer/pointer arguments;
- `a0` and `a1` carry integer return values;
- `t0`–`t6`, `a0`–`a7`, and `ra` are not preserved across calls;
- `s0`–`s11` are callee-saved;
- `sp` must be restored and follows ABI alignment rules;
- `s0` may be a frame pointer, but a frame pointer is optional.

This resembles the System V AMD64 convention in purpose, not in exact register names or counts. Both conventions answer the same economic question: which side should pay to preserve a live value across a call?

---

8. Equivalent sequences, compared carefully

Use AT&T syntax for x86-64 below because Days 1–2 did. RISC-V syntax writes the destination first. Count instructions, but do not confuse the count with time.

A. Return `*p + 7`

```
long add_seven(const long *p);
```

x86-64 System V:

```
movq (%rdi), %rax
addq $7, %rax
ret
```

RV64I ELF psABI:

```
ld   a0, 0(a0)
addi a0, a0, 7
ret                      # jalr x0, 0(ra)
```

Both make the load and arithmetic distinct here. The argument and return roles differ by ABI. The RISC-V `ret` spelling hides a base instruction pattern; x86 `ret` is an architectural instruction with implicit `rsp` behavior.

B. Increment through a pointer

x86-64:

```
addq $1, (%rdi)
ret
```

RV64I:

```
ld   t0, 0(a0)
addi t0, t0, 1
sd   t0, 0(a0)
ret
```

The one x86 instruction specifies a memory read-modify-write. That does not make it automatically atomic between cores; x86 has separate locking semantics for atomic read-modify-write behavior. The three RISC-V instructions make data movement explicit but likewise do not implement an atomic C update. RISC-V atomics belong to an extension beyond RV64I.

C. Return `base[index]`

x86-64:

```
movq (%rdi,%rsi,8), %rax
ret
```

RV64I:

```
slli a1, a1, 3
add  a0, a0, a1
ld   a0, 0(a0)
ret
```

x86 spends encoding and address-generation capability on a scaled indexed memory form. RISC-V composes the calculation from register operations. The compiler may schedule independent work around the explicit operations, while the x86 form is compact in architectural instruction count and often in bytes.

D. A counted sum loop

C intent:

```
long sum(const long *a, long n)
{
    long total = 0;
    for (long i = 0; i < n; ++i)
        total += a[i];
    return total;
}
```

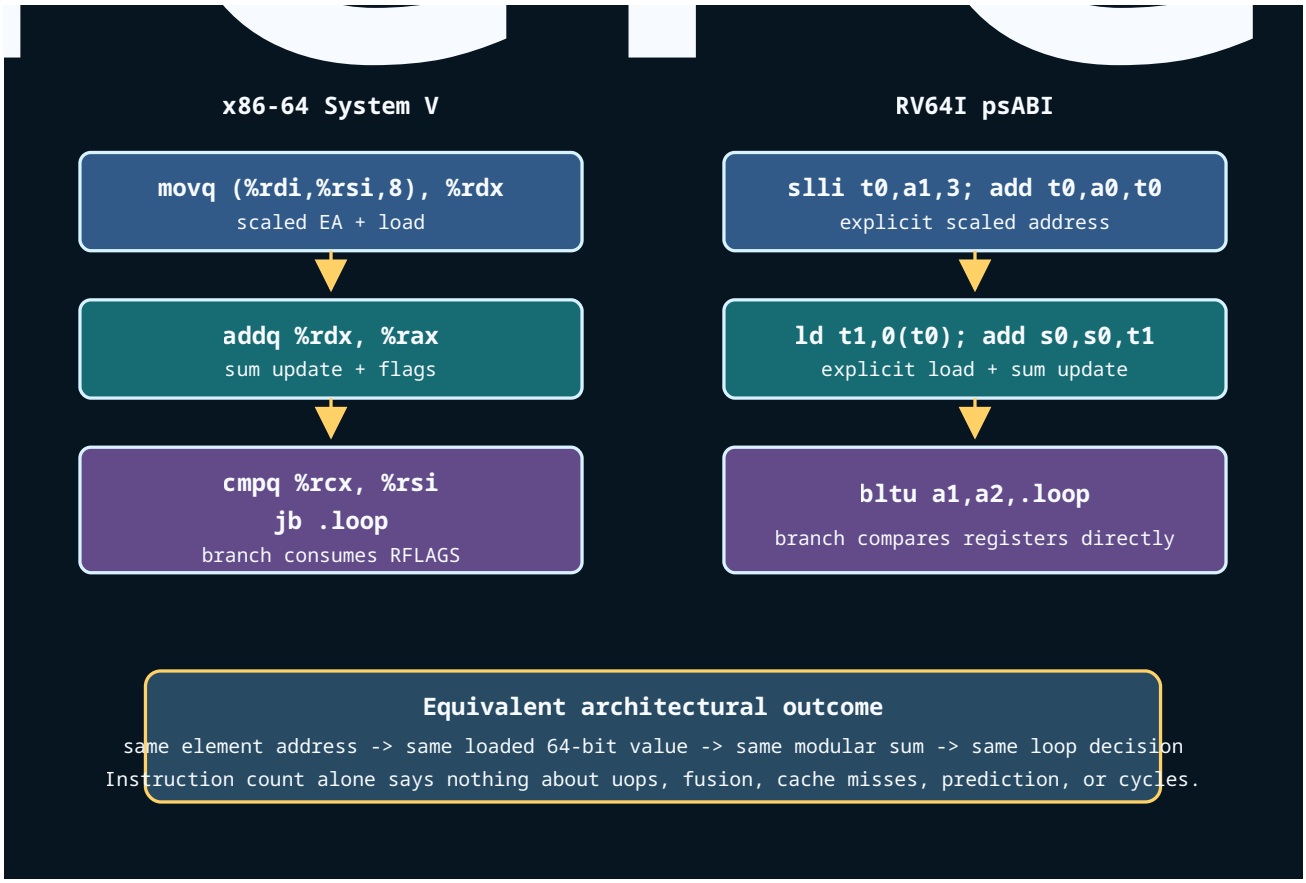
Plausible RV64I body:

```
    addi t0, zero, 0       # i
    addi t1, zero, 0       # total
.loop:
    bge  t0, a1, .done
    slli t2, t0, 3
    add  t3, a0, t2
    ld   t4, 0(t3)
    add  t1, t1, t4
    addi t0, t0, 1
    jal  zero, .loop
.done:
    addi a0, t1, 0
    ret
```

A compiler can choose a pointer-walking loop instead:

```
    slli a1, a1, 3
    add  a1, a0, a1       # end pointer
    addi t0, zero, 0
.loop:
    bgeu a0, a1, .done
    ld   t1, 0(a0)
    add  t0, t0, t1
    addi a0, a0, 8
    jal  zero, .loop
.done:
    addi a0, t0, 0
    ret
```

Both are plausible. A real compiler may transform more aggressively when C overflow, aliasing, vector extensions, and optimization settings permit. Assembly comparison is about preserved semantics, not matching one canonical answer.



The architectures expose different decompositions of the same required work; instruction count alone is not a performance result.

Comparison worksheet

For each pair above, record:

1. which instructions can access data memory;
2. how the effective address is formed;
3. where the immediate resides;
4. whether condition state is explicit in registers, implicit in flags, or consumed in a combined compare-and-branch;
5. which register role comes from the ABI;
6. which spelling is a pseudoinstruction;
7. what instruction count cannot prove.

---

9. RISC and CISC without the cartoon

The historical labels describe clusters of design choices, not two laws of nature.

RISC-V's base integer ISA visibly favors:

- regular base encodings;
- a load/store operand model;
- many general-purpose registers;
- explicit destination registers;

- a small base with named extensions;
- instructions that compose larger operations.

x86-64 visibly preserves:

- variable-length encodings from one through fifteen bytes;
- arithmetic forms that can use one memory operand;
- rich effective-address calculations;
- implicit operands and architectural flags;
- compatibility with decades of deployed binaries and operating systems.

Now remove the caricatures.

“RISC means one cycle per instruction” is false. Loads miss caches; division and vector operations take varying work; implementations may be superscalar, out of order, speculative, and internally complex.

“CISC means one instruction performs an entire program” is false. Common x86 compiler output is full of loads, stores, adds, compares, and branches. Many implementations decode architectural instructions into internal micro-operations and schedule them aggressively.

“Modern x86 is really RISC inside” is too imprecise to teach from. Internal operations are implementation-specific, not public RISC-V instructions. Intel and AMD may implement the same x86 instruction differently while preserving its architectural result.

“RISC-V is automatically faster or lower power” is false. ISA regularity can reduce some front-end and verification costs, especially in small implementations. Performance and energy also depend on process technology, cache hierarchy, prediction, execution width, physical design, software, and workload.

Why make different choices?

RISC-V had the opportunity to define a modular ISA with no obligation to run old x86 binaries. Regular fields, explicit operands, and extensions improve teachability, implementation freedom, and the ability to build anything from a small core to a large one under an open specification.

x86's compatibility is not an accidental embarrassment. Existing operating systems, applications, firmware, compilers, expertise, and validated platforms are enormous economic assets. Preserving old architectural behavior lets new processors run old binaries. The cost appears in encoding constraints, front-end complexity, verification burden, and architectural rules that cannot simply be redesigned.

The responsible comparison is:

```
RISC-V: freedom from a particular legacy + regular base contract
        ↔ software ecosystem maturity and extension/profile choices

x86-64: extraordinary deployed compatibility + dense expressive encodings
        ↔ decode/verification complexity and inherited constraints
```

This is why RISC-V is often easier to reason about at the base-ISA level while x86 retains compatibility advantages. “Easier to reason about” does not mean “easy to build a competitive server CPU,” and “compatibility” does not mean “no innovation is possible.”

Architecture lens — x86-64, AArch64, and RISC-V

AArch64 is a useful check against treating “RISC” and “CISC” as two complete design packages: like RISC-V it is a fixed-width load/store ISA in its main 64-bit state, but it has 31 general-purpose registers, an `NZCV` flags register, and a conventional link register. RISC-V instead has 32 integer registers including `x0` and direct integer compare-and-branch instructions; x86-64 combines variable-length encoding, sixteen general-purpose registers, flags, and many register-or-memory forms.

Those are independent ISA choices, not performance rankings. AAPCS64, the RISC-V psABI, and System V AMD64 separately assign argument, return, and preserved-register roles; pipeline depth, µops, fusion, and execution width belong to implementations. See Architecture Comparison.

---

10. CPU, kernel, and virtualization connections

What a context switch must preserve

Linux can switch tasks only because each ISA defines architectural state. For RISC-V that includes the program counter, integer registers, and relevant privileged/control state; enabled extensions add state such as floating-point or vector registers. The kernel does not save an implementation's reorder buffer or branch-predictor table as if those were process registers.

On a trap from userspace, architecture-specific entry code saves a register frame so C kernel code can inspect or modify the interrupted context. The RISC-V Linux `pt_regs` layout is not the x86-64 `pt_regs` layout because the architectural registers and trap mechanisms differ. The kernel's abstract jobs—preserve state, identify the cause, service it, and return—remain.

Why `x0` still matters in privileged code

The zero register is not only a convenience for application arithmetic. Control-transfer forms can discard links by selecting `x0`; comparisons can use it directly; privileged assembly can clear or initialize values without first dedicating a register to zero. Its semantics remain architectural at every privilege level.

Virtualization

A hypervisor or emulator presents a virtual architectural machine. KVM's RISC-V support must expose guest-visible registers and virtualize privileged state according to the relevant specifications and extensions. QEMU can decode guest RISC-V instruction words in software and update a modeled machine even when the host CPU is x86-64.

Today's Python decoder uses the same high-level idea at toy scale:

```
fetch 32-bit word from a PC-indexed map
    → inspect opcode/funct/register/immediate fields
    → read modeled registers or bytes
    → compute the required result
    → update modeled architectural state
```

It is not virtualization: there is no guest operating system, privilege model, device model, translation cache, or complete ISA. But it reveals why cross-architecture emulation is possible. The host need not physically implement the guest ISA if software can reproduce its required visible behavior.

---

11. Laboratory — decode and trace without a cross-toolchain

Use:

```
02-assembly-memory-and-c/challenges/day-003-riscv-decoder.py
```

The script uses only Python's standard library. It decodes a substantial teaching subset of RV64I and simulates its architectural effects.

From the repository root:

```
python3 02-assembly-memory-and-c/challenges/day-003-riscv-decoder.py demo
python3 02-assembly-memory-and-c/challenges/day-003-riscv-decoder.py selftest
```

The self-test checks representative R/I/S/B/U/J encodings, immediate edges, the fixed-zero invariant, and the complete sum trace.

Lab A — expose the fields

Start with:

```
python3 02-assembly-memory-and-c/challenges/day-003-riscv-decoder.py \
  fields 0x00b50633
```

Before reading the rendered decode:

1. mark bits 6:0, 11:7, 14:12, 19:15, 24:20, and 31:25;
2. convert each five-bit register field to decimal;
3. identify the format from the major opcode;
4. only then identify the operation.

Decode several words at once:

```
python3 02-assembly-memory-and-c/challenges/day-003-riscv-decoder.py decode \
  0xfff50513 0x00b50633 0x00513423
```

Add `--x-registers` to remove ABI aliases. Explain why the instruction has not changed when `a0` becomes `x10`.

Lab B — immediate reconstruction

```
python3 02-assembly-memory-and-c/challenges/day-003-riscv-decoder.py exercise
```

For exercises A–E, draw the appropriate format and reconstruct the immediate by hand. For the branch:

```
instruction bits → imm[12|10:5|4:1|11|0] → sign extension → PC addition
```

Then reveal:

```
python3 02-assembly-memory-and-c/challenges/day-003-riscv-decoder.py \
  exercise --answers
```

The answers intentionally show both `xN` and ABI register names.

Lab C — trace signed versus unsigned branches

```
python3 02-assembly-memory-and-c/challenges/day-003-riscv-decoder.py trace branch
```

Before running, predict:

- the 64-bit pattern produced by `addi t0, zero, -1`;
- whether `blt t0,t1` is taken;
- whether `bltu t0,t1` is taken;
- final `a0` and `a1`;
- whether any instruction changes `x0`.

The trace reports PC before and after each instruction and every changed register. Explain the result using bit interpretation, not “the register became unsigned.”

Lab D — trace a load/store loop

Run only the first eight dynamic instructions:

```
python3 02-assembly-memory-and-c/challenges/day-003-riscv-decoder.py \
  trace sum --limit 8
```

At the stop, record `t0`–`t4`, `t1`'s sum role, and the next PC. Then trace to completion:

```
python3 02-assembly-memory-and-c/challenges/day-003-riscv-decoder.py trace sum
```

The mapped array is `[7, -2, 13, 5]`, so predict `a0` before running. For each loop iteration annotate:

```
index → byte offset → effective address → loaded element → accumulated sum
```

Only `ld` accesses data memory in this trace. Instruction fetch is also a memory-system activity in a real CPU, but the lesson's phrase “data memory access” distinguishes the explicit program operand from instruction fetch.

Lab E — modify one architectural choice

Make one local experimental edit, then restore it:

1. remove the `if number != 0` condition in `write_reg`;
2. run `selftest`;
3. inspect the first failure;
4. explain which later sequences would become unreliable if `x0` could retain a value.

Or choose the encoding route:

1. add one new decode-only instruction to an unused pattern in this teaching model;
2. state which fields select it;
3. decide whether it needs `rd`, `rs1`, `rs2`, or an immediate;
4. explain what encoding resource you consumed.

The goal is not to invent a superior ISA in ten minutes. It is to discover that every convenience needs fields, semantics, tool support, tests, and compatibility rules.

Optional real toolchain observation

If a RISC-V GCC or LLVM target happens to be installed, verify the examples. Do not install one merely to complete this lesson.

Possible checks:

```
command -v riscv64-linux-gnu-gcc
clang --print-targets | grep -i riscv
```

If supported:

```
clang --target=riscv64-linux-gnu -O2 -S example.c -o /tmp/example-rv64.s
```

Sysroot or headers may be absent even when the backend exists. A freestanding function with no includes is the simplest test. The Python simulator is the required laboratory precisely because host toolchain availability should not block architectural reasoning.

---

12. Mastery — Explain · Draw · Observe · Build

Explain

Without notes, answer:

> Why is the RV64I base easier to decode on paper than an arbitrary x86-64 instruction, and why does that not prove a RISC-V program is faster?

A complete answer should include:

1. common 32-bit base width and stable register-field positions;
2. explicit load/store traffic and three-register arithmetic;
3. fewer implicit integer operands and no general integer flags dependency;

4. x86 variable length, prefix/opcode/ModR/M/SIB/displacement/immediate possibilities, and inherited compatibility;
5. the distinction between architectural instruction count and microarchitectural execution;
6. caches, prediction, scheduling, code density, extensions, and workload.

Also explain:

- why `x0` is useful even though it cannot store a changing value;
- why an S-type instruction has no `rd`;
- why B-type immediate bits are rearranged;
- why `ret` can be a pseudoinstruction;
- why `ra` must be saved by a non-leaf function;
- why ABI register names are not ISA-mandated argument behavior.

Draw

Draw one 32-bit word for each format:

```
R  I  S  B  U  J
```

Color or mark:

- major opcode;
- operation-selecting function bits;
- each register;
- immediate bits;
- implicit low zero for branch/jump displacement.

Then draw:

```
ELF/text bytes → PC fetch → 32-bit instruction → field extraction
    → register reads → ALU/address/branch → register or memory update
    → next PC
```

Beside it, draw an x86-64 fetch window containing instructions of unequal length. The diagram should show a tradeoff, not a winner.

Observe

Save evidence from the simulator:

1. one R-type field decode;
2. one negative I-type immediate;
3. one reconstructed S- or B-type split immediate;
4. the signed/unsigned branch trace;
5. one complete sum-loop iteration with effective address;
6. final `x0`, proving a write was discarded;
7. one invalid or unsupported word and the decoder's error.

For the last item, try a word with major opcode zero:

```
python3 02-assembly-memory-and-c/challenges/day-003-riscv-decoder.py decode 0x00000000
```

A decoder must reject unsupported/reserved encodings. It must not invent a plausible instruction for every 32-bit pattern.

Build

Complete all three:

1. Change the sum array and predict the trace's result before running it.
2. Add a second small program using `sd` followed by `ld`; prove byte addresses and little-endian storage in the simulator.
3. Write equivalent plausible x86-64 and RV64I sequences for:

`c long max(long a, long b); long sum4(const long *p); void replace(long *p, long old, long new_value);`

For each, document instruction count, explicit memory instructions, condition mechanism, and ABI registers. Do **not** rank performance from source-line count.

This satisfies the tracker's hands-on requirement to implement the same function conceptually in both ISAs, compare instruction count and clarity, and document key differences even when a RISC-V assembler is unavailable.

Why? notebook

1. Why does a fixed zero register save opcodes but cost storage capacity?
2. Why does a load/store ISA expose more address-calculation instructions?
3. Why can `lw` and `lwu` read the same four bytes and produce different 64-bit values?
4. Why is a 12-bit immediate sign-extended instead of always zero-extended?
5. Why are store and branch immediates split across fields?
6. Why does a branch displacement have an implicit zero bit?
7. Why can RISC-V call a function without immediately touching the stack?
8. Why must a non-leaf function still preserve `ra`?
9. Why are `a0` and `x10` two names for one architectural register?
10. Why can one x86 instruction and three RISC-V instructions perform the same architectural task without implying a 3:1 runtime ratio?
11. Why is x86 compatibility an engineering benefit rather than merely “legacy baggage”?
12. Why can QEMU execute RISC-V guest instructions on an x86-64 host?

End model

```
software problem: compute, access memory, choose control flow, call
        ↓
RV64I provides 32 integer registers, with x0 fixed at zero
        ↓
32-bit base forms allocate fields among opcode/function/register/immediate
        ↓
loads and stores alone name data memory
        ↓
branches compare registers and add a signed displacement to PC
        ↓
jal/jalr write an explicit link register; the ABI makes x1 the return address
        ↓
compiler and ABI compose these primitives into functions
        ↓
CPU, kernel, emulator, or hypervisor preserves the specified architectural
behavior while choosing very different internal mechanisms
```

The mastery sentence, made precise:

> RV64I is comparatively easy to reason about because its base formats, register roles in each instruction, and load/store data movement are regular and explicit. x86-64 preserves a richer, variable-length architectural contract so decades of binaries and systems continue to work. Those facts describe software-visible contracts and engineering costs; they do not determine performance by themselves.

References used selectively

Primary RISC-V specifications:

- RISC-V International, *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture*, current ratified specification index. See “RV32I Base Integer Instruction Set” for register state, instruction formats, integer computation, control transfer, and loads/stores; and “RV64I Base Integer Instruction Set” for RV64 differences: https://docs.riscv.org/reference/isa/unpriv/unpriv-index.html
- RISC-V International, *RV32I Base Integer Instruction Set*, especially “Programmers' Model for Base Integer ISA,” “Base Instruction Formats,” “Immediate Encoding Variants,” “Integer Register-Immediate Instructions,” “Control Transfer Instructions,” and “Load and Store Instructions”: https://docs.riscv.org/reference/isa/unpriv/rv32.html
- RISC-V International, *RV64I Base Integer Instruction Set*, including sign-extension conventions and RV64 load forms: https://docs.riscv.org/reference/isa/unpriv/rv64.html
- RISC-V non-ISA specifications, *RISC-V ABIs Specification*, current `riscv-elf-psabi-doc`, especially “Integer Register Convention,” “Frame Pointer Convention,” “Procedure Calling Convention,” and stack alignment: https://riscv-non-isa.github.io/riscv-elf-psabi-doc/

Official x86 comparison sources:

- Intel, *Intel® 64 and IA-32 Architectures Software Developer's Manual*, Volume 1 for the programming environment and Volume 2, Chapter 2 for instruction format and instruction

reference: https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html

- AMD, *AMD64 Architecture Programmer's Manual, Volume 1: Application Programming* and Volume 3: *General-Purpose and System Instructions* (publication 24594), for the programmer-visible model, instruction encoding, addressing, flags, and control transfer: https://www.amd.com/en/search/documentation/hub.html#sortCriteria=%40amd_release_date%20descending&f-amd_archive_status=Active&f-amd_document_type=Programmer%20References

Kernel and virtualization connection:

- Linux kernel documentation, RISC-V architecture index: https://docs.kernel.org/arch/riscv/index.html
- Linux KVM documentation, including architecture-specific API material: https://docs.kernel.org/virt/kvm/index.html

The official manuals are authoritative for architectural behavior. The Python challenge intentionally implements only the subset named at the top of the file and should not be used as a compliance oracle.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Riscv Decoder

Format: Python lab

What you will do: Day 3 — RV64I decoder and trace laboratory.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
python3 02-assembly-memory-and-c/challenges/day-003-riscv-decoder.py trace sum
```

- View lab source on GitHub

Day 4 — RAM, Locality, and the Memory Wall

Target time: approximately 2–3 hours

- First-principles lesson and predictions: 70–85 minutes
- Fedora benchmark and plot: 45–60 minutes
- Evidence, explanation, and review: 30–40 minutes

> A CPU can finish an arithmetic operation in a handful of cycles. What happens when the operand has not arrived yet?

That question created much of the modern memory system. Today's path is:

```
one ideal memory
    → incompatible speed, capacity, cost, and persistence goals
    → registers + caches + DRAM + storage
    → locality makes the hierarchy useful
    → experiments reveal where the illusion becomes expensive
```

This lesson connects the CPU datapath from earlier lessons to Linux virtual memory and, eventually, NUMA. It deliberately stops short of cache indexing, replacement, and coherence; those belong to the next lesson.

---

1. Why not build one enormous, instant memory?

A useful computer wants memory to be all of these:

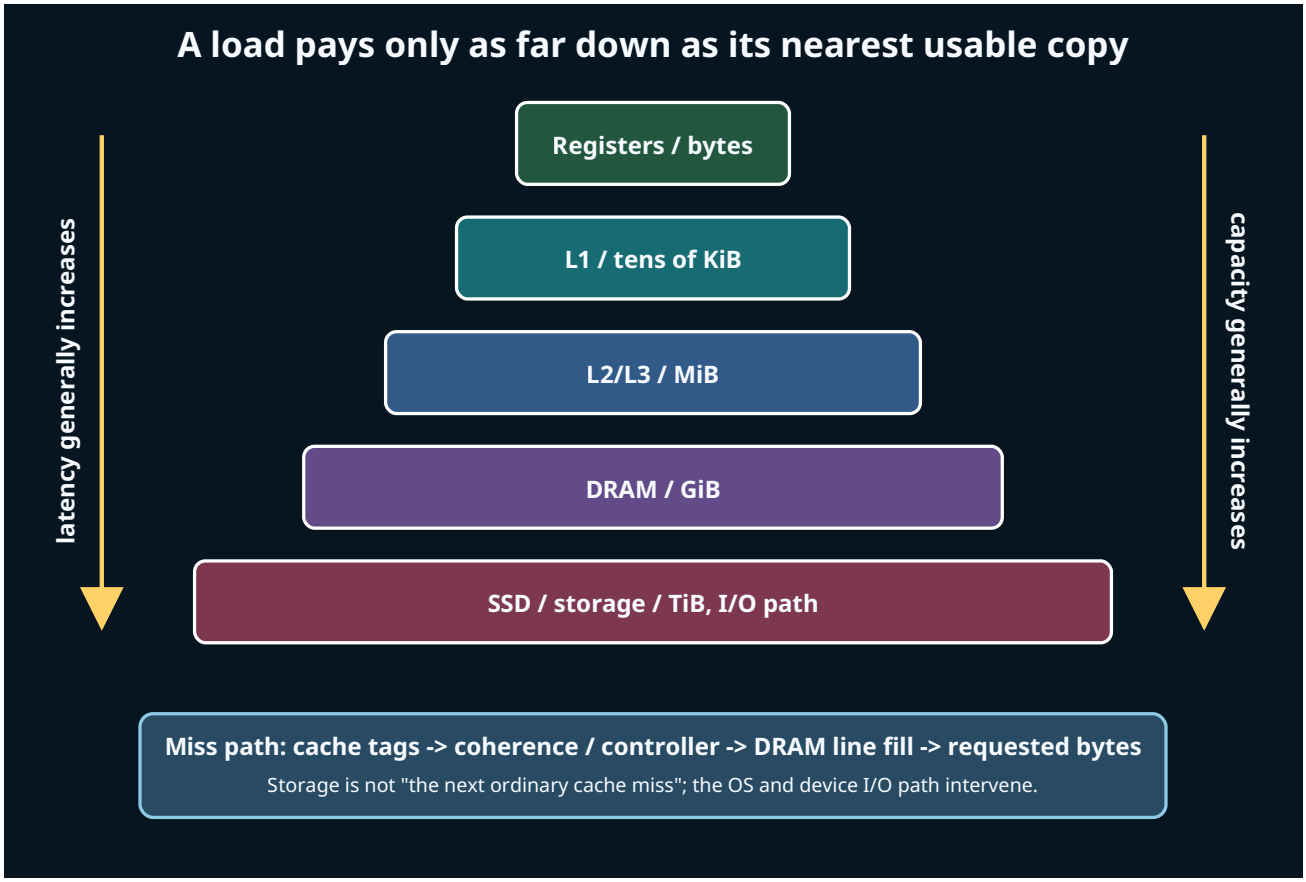
- as fast as the CPU;
- large enough for every active program and file;
- cheap enough to manufacture;
- low-power;
- persistent after power loss.

No current technology gives all of those properties at once. Fast storage usually spends more area and energy per bit. Dense storage usually takes longer to access. Persistent storage has different physical constraints again. Distance also matters: a signal must travel through wires, controllers, and devices; large structures take time to select and drive.

The resulting hierarchy is not arbitrary decoration:

```
                  smaller, faster, costlier per byte
CPU registers                tens to hundreds of bytes per core
L1 / L2 / L3 caches          KiB to many MiB
DRAM                          GiB
SSD / other storage          hundreds of GiB to TiB
                  larger, slower, cheaper per byte
```

The exact sizes and timings are machine-specific. Even the diagram is an abstraction: some caches are private, some shared; storage is reached through I/O rather than ordinary CPU loads; and virtual memory may make storage back some pages. The durable idea is that each level tries to make the next, slower level less visible.



Increasing capacity generally increases access distance, while each hit prevents a trip to lower levels.

The historical problem path

Early machines repeatedly changed storage technology because each design hit a different limit. Registers and flip-flop-like storage were fast but expensive. Delay lines, magnetic drums, and magnetic core offered useful capacity with different access constraints. Semiconductor memory made random-access main memory practical at scale. As processors became faster through integration, pipelining, and parallel execution, DRAM latency improved much more slowly in processor-cycle terms.

Caches were an engineering response: keep recently or nearby used data in a small fast structure. Multiple cache levels appeared because one cache could not simultaneously be tiny enough for the fastest access and large enough to catch broad working sets. This is the historical core of the **memory wall**: increasing CPU execution capability does not ensure proportional application speedup when data delivery is the limiting path.

---

2. Where a loaded byte actually travels

For a normal userspace load whose address is already translated:

```
instruction
  → CPU load machinery
  → search cache hierarchy
  → on a last-level miss, memory controller
  → DRAM channel/rank/bank/row
  → cache-line-sized transfer back toward the core
  → requested bytes delivered to a register
```

That is a conceptual path, not a universal timing diagram. Translation may need the TLB or a page walk. Hardware prefetchers may request lines before the load. Several misses may overlap. Dirty evictions can create more traffic.

Linux adds another layer of meaning. `malloc` normally reserves virtual address space; physical pages are commonly supplied on first touch. The kernel chooses physical pages, handles faults, may reclaim or migrate pages, and on a NUMA host applies placement policy. Once mappings exist, ordinary cacheable loads do not trap into the kernel. Hardware translates and accesses them.

On a NUMA machine, the memory controller nearest one CPU package may reach some DRAM with lower cost than remote DRAM attached to another package. Linux exposes topology and placement controls (`lscpu`, `numactl --hardware`, `numactl` policies), but NUMA measurement is a later focused lab. For today, remember that “DRAM latency” is not one context-free number.

---

3. SRAM and DRAM at the right depth

SRAM: retain state while powered

A typical SRAM bit cell uses a small bistable transistor network plus access transistors. Its state can be read without periodically refreshing every bit. It is fast, but the cell and supporting circuitry consume substantial silicon area per bit. SRAM is therefore a natural fit for caches and similar small, fast on-chip arrays.

“Static” does **not** mean persistent: remove power and the state is lost. It also does not mean no control circuitry or no energy use.

DRAM: store charge densely

A conceptual DRAM cell stores a bit as charge in a tiny capacitor selected by a transistor. Charge leaks, so rows must be refreshed. Reading involves sense amplifiers and is effectively destructive at the cell level, requiring the value to be restored. DRAM accesses involve opening a row, operating on columns, and sometimes closing/precharging before another row.

The cell is dense, making large main memories economical, but access has more latency and management overhead than SRAM. Real DRAM organizations include channels, ranks, banks, rows, bursts, queues, and scheduling. We do not need their electrical timings today to understand why access order can matter.

Question	SRAM	DRAM
Typical role	CPU caches	Main memory
Conceptual storage	bistable circuit state	charge sensed and restored

Question	SRAM	DRAM
Refresh	no periodic per-bit refresh	required
Density/cost per bit	lower density, higher cost	higher density, lower cost
Relative access	generally faster	generally slower
Persistent without power	no	no

These are architectural tendencies, not promises that every product called SRAM or DRAM has one fixed latency.

---

4. Latency and bandwidth are different questions

Latency asks how long one operation takes from request to useful result.

Bandwidth asks how much data can be transferred per unit time once work is flowing.

A delivery truck has high latency for one package but can have high aggregate bandwidth when full. Memory behaves similarly: a dependent pointer chase must wait for each address before issuing the next useful load, exposing latency. A sequential scan presents predictable, independent work; prefetching, wide transfers, cache lines, multiple outstanding misses, and vectorization can raise throughput.

Do not convert one into the other carelessly. `ns/access` from a dependency chain is a latency-oriented observation. `GB/s` from a large sequential scan is throughput-oriented. A cache line fetch also transfers neighboring bytes, so “one C load” is not necessarily “one DRAM transaction.”

---

5. Locality is the reason caching usually works

Programs do not normally choose each next byte uniformly from all available memory.

Temporal locality means recently used data is likely to be used again:

```
for (...) {
    total += frequently_used_counter;
}
```

Keeping the relevant cache line nearby helps repeated access.

Spatial locality means an address near a recently used address is likely to be used soon:

```
for (size_t i = 0; i < n; ++i) {
    total += array[i];
}
```

Memory moves data in blocks—typically cache lines between DRAM and caches—so one miss can bring neighboring elements needed by later iterations.

A row-major C matrix makes the consequence concrete. For:

```
int a[rows][cols];
```

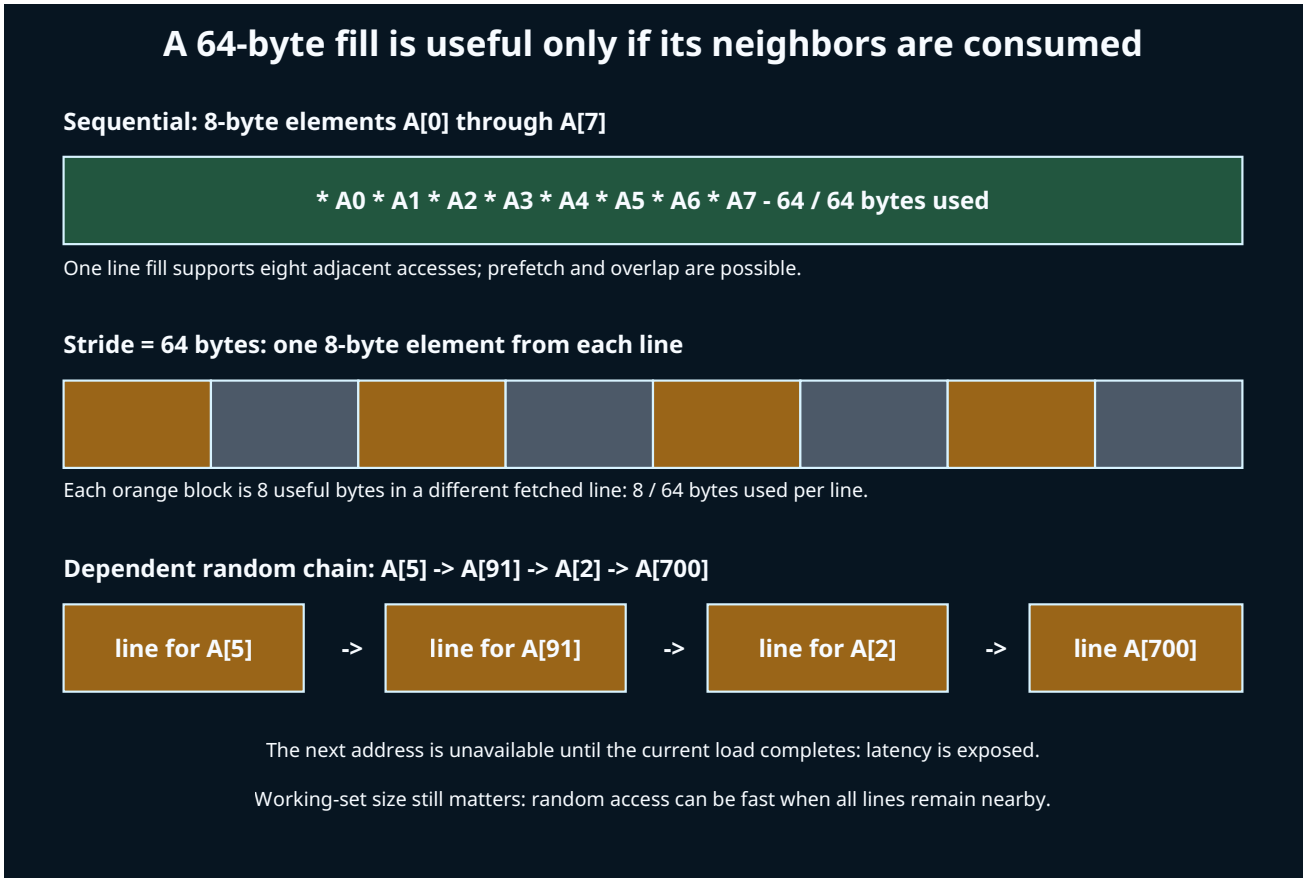
`a[r][0]`, `a[r][1]`, and `a[r][2]` are adjacent. Iterating columns in the inner loop follows layout; iterating rows in the inner loop jumps by an entire row. Both compute the same mathematical set of elements, but they present very different address streams.

Sequential versus random is not magic vocabulary

Sequential access tends to:

- use most bytes fetched in each cache line;
- be predictable to hardware prefetchers;
- permit several loads to overlap;
- make vectorization easier.

Random access tends to waste neighboring fetched bytes and defeat simple prefetching. A **dependent** random chain additionally prevents the next address from being known until the previous load completes.



Access order determines both how much of each fetched line is useful and how much latency can overlap.

But random does not automatically mean slow. A small random working set can remain in cache. Conversely, a sequential stream larger than cache still consumes memory bandwidth. Size, reuse, dependency, layout, and concurrency all matter.

---

6. Working sets and visible boundaries

A program's **working set** is the data it actively needs during an interval. As that set grows, it may cease to fit in a nearby cache level. More accesses then reach farther levels.

One might expect a perfect staircase at L1, L2, L3, and DRAM capacities. Measurements are rarely that clean:

- caches are shared with code, page tables, and other data;
- set mapping and associativity affect usable capacity;
- hardware prefetching changes sequential behavior;
- replacement is dynamic;
- the OS and hypervisor interrupt or deschedule the process;
- frequency, power, and thermal control change CPU timing;
- first touch causes page faults;
- transparent huge pages and page placement may differ;
- a virtual machine adds host scheduling and a virtualized topology.

Look for ranges, slopes, and repeatable transitions—not an invented claim that one point is “the exact L2 latency.”

---

7. Lab: observe the wall without fooling yourself

The lab has two kernels:

1. **Sequential scan** sums every `uint64_t` in order. It reports elapsed nanoseconds per C-level access and is throughput-friendly.
2. **Random pointer chase** follows a shuffled cycle. Each next index depends on the preceding load, making it latency-sensitive.

They answer related but different questions. Their ratio is **not** a pure measurement of “random versus sequential DRAM.” The instruction streams, dependency structure, available memory-level parallelism, and extra index array differ.

Compile warning-clean on Fedora

From the repository root:

```
gcc -std=c17 -O2 -Wall -Wextra -Wpedantic -Wconversion \
  02-assembly-memory-and-c/challenges/day-004-memory-wall-lab.c \
  -o /tmp/day-004-memory-wall-lab
```

`-O2` is necessary for a representative optimized loop. The program consumes results through a volatile sink and output, so the compiler cannot simply delete the measured work. This does not freeze every optimization; inspect assembly if you need to characterize exactly what GCC generated.

Run with a 64 MiB maximum working set (or choose another safe limit):

```
/tmp/day-004-memory-wall-lab 64 | tee /tmp/day-004-memory-wall.csv
```

The program allocates roughly three times the requested maximum because it keeps data, permutation, and link arrays. Do not request a size that pressures the host into swapping. Close heavy applications if appropriate.

Plot:

```
python3 02-assembly-memory-and-c/challenges/day-004-plot.py \
  /tmp/day-004-memory-wall.csv
```

If Matplotlib is absent, the script prints the Fedora package to install. Installing it is optional; the CSV itself is valid evidence, and a spreadsheet can plot `bytes` against `ns_per_access`.

Predict before running

Write down:

1. At what working-set ranges do you expect changes?
2. Which curve should rise more visibly as the set exceeds caches?
3. Why might the sequential curve remain comparatively flat?
4. What result would make you rerun rather than tell a dramatic story?

Run protocol

1. Record `lscpu` and whether this is bare metal or a VM.
2. Run the complete benchmark at least three times.
3. Do not average first. Plot or compare the runs and identify variability.
4. Repeat once while the system is busy, if safe, to see environmental noise.
5. Record compiler version and command.
6. Optionally inspect generated code with:

`bash objdump -d -Mintel /tmp/day-004-memory-wall-lab | less`

The code performs a warm-up, but warm-up does not make a benchmark pure. Initial page faults are reduced because initialization touches memory; cache state still evolves, and larger sets evict earlier data. `CLOCK_MONOTONIC` measures wall time, including descheduling. That is honest for a simple lab, provided we report the noise.

Interpret carefully

For each run, ask:

- Are transitions in similar size ranges across repeats?
- Does random-chase cost increase as the working set grows?
- Does sequential behavior look more throughput-efficient?
- Are there isolated spikes likely caused by scheduling?
- Is the largest set actually below, near, or above the host's last-level cache?
- Could a VM be hiding or sharing the physical cache topology?

You may say, “The repeatable increase between these ranges is consistent with more accesses being served by a farther hierarchy level.” Do not say, “This point proves my L3 is exactly X MiB and DRAM is exactly Y ns” from this lab alone.

For deeper work, CPU performance counters can add evidence, but event names, availability, privilege policy, multiplexing, and interpretation are processor-specific. `perf stat` is not a universal truth machine.

---

8. Optimization lessons that survive the benchmark

Useful optimization is about reducing expensive data movement or hiding it, not merely changing syntax.

1. **Lay out data for the traversal.** Contiguous arrays often beat scattered objects when the operation naturally visits everything.
2. **Reuse while data is nearby.** Loop tiling/blocking reorganizes work so a cache-sized tile receives several operations before eviction.
3. **Avoid needless footprint.** Smaller representations can fit more useful objects in each cache and cache line.
4. **Separate hot and cold fields.** Frequently used data need not drag rare metadata through caches.
5. **Allow parallel requests where semantics permit.** Independent loads can overlap; dependent pointer chains cannot.
6. **Measure the real workload.** A synthetic benchmark diagnoses mechanisms, not end-to-end application value.

Optimization can shift cost elsewhere. Compression reduces bytes but spends compute. Copying into a contiguous buffer helps repeated traversal but costs a copy. Prefetching too early evicts useful data; too late does not hide latency. NUMA placement that helps one thread can hurt another.

---

9. CPU, controller, Linux, and NUMA bridge

An application says `sum += a[i]`. Several layers cooperate:

```
C object and loop
  → optimized machine loads
  → virtual address translation
  → cache lookup and hardware prefetch policy
  → memory-controller queues and DRAM commands on misses
  → physical page chosen under Linux policy
  → possibly local or remote NUMA memory
```

This bridge prevents two common errors:

- “Linux handles every memory access.” It establishes and manages mappings; hardware normally services mapped loads without entering the kernel.
- “The CPU directly asks a DRAM chip for one variable.” Caches, line transfers, translation, controllers, queues, and DRAM organization intervene.

Later modules will sharpen cache geometry, page allocation, virtual memory, and NUMA policy. Today's model is enough to explain why a faster execution engine can spend cycles waiting for its data path.

---

10. Evidence: Explain, Draw, Observe, Build

Explain

Without notes:

1. Why can no single practical memory level satisfy speed, capacity, cost, and persistence simultaneously?
2. Why is SRAM used for caches and DRAM for main memory?
3. Distinguish latency from bandwidth with one program example.
4. Define temporal and spatial locality without using the word “cache.”
5. Why can a sequential scan outperform a dependent random traversal?
6. Why can a faster CPU still wait on memory?

Draw

Draw one complete load path:

```
register ← caches ← memory controller ← DRAM
                    ↑
virtual address → translation → physical page / NUMA node
```

Add storage below DRAM and explain why reaching it is not an ordinary cache miss. Mark where Linux policy acts and where hardware normally acts.

Observe

Submit:

- three raw CSV runs;
- one plot or a hand-drawn graph from the CSV;
- host/VM, compiler, command, and maximum size;
- two repeatable observations;
- one noisy or ambiguous observation;
- one claim you deliberately refused to make.

Build

The C benchmark and plot are the build evidence. Then change one variable: maximum size, system load, compiler optimization level, or CPU affinity. Predict first, rerun, and explain why the result changed—or why it did not.

---

11. Mastery checkpoint

Close the lesson and answer:

> Why may a CPU with a faster arithmetic pipeline fail to make a memory-intensive program proportionally faster?

A complete answer includes:

- the mismatch among register/cache, DRAM, and storage properties;
- latency versus bandwidth;
- cache lines and locality;
- working-set size;
- dependent versus independent accesses;
- the controller/DRAM path on cache misses;
- Linux page placement and the possibility of NUMA distance;
- why one noisy microbenchmark cannot establish universal hardware constants.

Then make two predictions:

1. a small randomly traversed array versus a huge one;
2. a huge sequential array versus a huge dependent random chain.

If the explanation is only “RAM is slow,” it is not yet complete.

Why? notebook

1. Why does greater DRAM density not automatically make it cache-speed?
2. Why does fetching a cache line reward spatial locality?
3. Why does reuse reward temporal locality?
4. Why can bandwidth be high while dependent-load latency remains painful?
5. Why might a cache-capacity boundary appear as a broad slope?
6. Why does first touch matter on Linux?
7. Why can VM measurements differ between runs?
8. Why is pointer chasing useful for exposing latency but unlike many real application loops?
9. Why might reducing a structure's size improve performance without reducing the number of source-level accesses?
10. Where can NUMA enter the path, even though the C pointer looks unchanged?

Mental model at the end

```
The CPU requests an address, not “a variable from RAM.”
Nearby hierarchy levels are tried first.
Locality makes small fast levels effective.
Misses travel through controllers to denser, slower memory.
Sequential independent work can use bandwidth and overlap.
Dependent random work exposes more latency.
Linux controls mappings and placement; hardware handles ordinary mapped loads.
Measurements are evidence conditioned on code, machine, OS, and environment.
```

References used selectively

- John L. Hennessy and David A. Patterson, *Computer Architecture: A Quantitative Approach*, memory-hierarchy chapters, for locality, latency, bandwidth, and the quantitative hierarchy model.
- Ulrich Drepper, “What Every Programmer Should Know About Memory,” 2007, Sections 2–3. Useful for foundational SRAM/DRAM and cache concepts; specific hardware examples are historical: https://people.freebsd.org/~lstewart/articles/cpumemory.pdf
- Wulf and McKee, “Hitting the Memory Wall: Implications of the Obvious,” *ACM SIGARCH Computer Architecture News* 23(1), 1995: https://doi.org/10.1145/216585.216588
- Linux kernel documentation, “Memory Management”: https://docs.kernel.org/mm/index.html
- Linux kernel documentation, “NUMA Memory Policy”: https://docs.kernel.org/admin-guide/mm/numa_memory_policy.html
- `clock_gettime(3)` and `perf-stat(1)` Fedora/Linux manual pages for the measurement interfaces and their limits.

The benchmark is a teaching instrument synthesized for this course. It is not a hardware certification tool, and its observations must be reported with the compiler, host, topology, and run conditions.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Memory Wall Lab

Format: C lab

What you will do: Warm-up is not a promise of a noise-free or entirely resident run.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
gcc -std=c17 -Wall -Wextra -O0 -g -pthread 02-assembly-memory-and-c/challenges/day-004-memory-wall-lab.c -lm -o /tmp/day-004-memory-wall-lab && /tmp/day-004-memory-wall-lab 64 > /tmp/memory-wall.csv
```

- View lab source on GitHub

Put it into practice — Plot

Format: Python lab

What you will do: Plot CSV emitted by day-004-memory-wall-lab.c.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
python3 02-assembly-memory-and-c/challenges/day-004-plot.py /tmp/memory-wall.csv
```

- View lab source on GitHub

Day 5 — Cache Organization

Curriculum alignment: Stage 4 — Memory Hierarchy and Performance · Cache Organization

Target time: approximately 2 hours 45 minutes

- Build the address-to-cache model: 55 minutes
- Misses, writes, replacement, and coherence: 45 minutes
- Mapping, locality, false-sharing, and `perf` labs: 50 minutes
- Explain · Draw · Observe · Build · Why: 15 minutes

> Yesterday established the memory wall: nearby computation can finish while DRAM is still finding and returning data. A cache narrows that gap—but how does a small structure decide whether it already holds the byte at address `0x12345678`?

The answer was shaped by an engineering conflict. A fully flexible lookup would compare an address against every cached block: fast to describe, expensive to wire, power-hungry, and difficult to scale. A single fixed home for each block is cheap, but unrelated addresses can evict one another. Modern caches compromise by dividing storage into **sets** and giving each set several candidate **ways**.

Today we make that geometry mechanical enough to calculate by hand, then connect it to Linux performance counters and a two-thread false-sharing lab.

Tracker objectives

By the end, you should be able to:

- **Explain cache lines, sets, and ways.**
- **Identify compulsory, capacity, and conflict misses.**
- **Understand write policies.**
- **Reason about false sharing.**

The tracker labs are all present: address-to-cache mapping, a threaded false sharing demonstration, and `perf stat` cache-event observation. The mastery checkpoint is exact:

> **Given a memory address and cache geometry, identify its tag, set, and offset.**

---

1. A cache transfers lines, not isolated C objects

Programs name bytes. Main memory and caches move data in blocks. In cache terminology that block is a **cache line**. A common line size on current x86-64 systems is 64 bytes, but this is an observed property, not a C or Linux guarantee.

If a load needs address `A`, a 64-byte line begins at:

```
line_base = floor(A / 64) × 64
```

and contains addresses `line_base` through `line_base + 63`.

Fetching the neighbors is a wager on spatial locality. If code reads the next array element, those bytes may already be present. If code reads one byte from each 4 KiB page, most bytes fetched in each line may go unused.

Inspect the host's reported geometry:

```
lscpu --caches
getconf LEVEL1_DCACHE_LINESIZE 2>/dev/null || true
for f in /sys/devices/system/cpu/cpu0/cache/index*/{level,type,size,coherency_line_size,ways_of_asso
ciativity,number_of_sets}; do
    printf '%s: ' "$f"; cat "$f" 2>/dev/null
done
```

Containers and virtual machines may expose incomplete, synthetic, or host- filtered topology. Record what the interface reports; do not promote it to a universal hardware fact.

---

2. Sets and ways: restricting the search

A cache is organized as:

```
capacity = line_size × number_of_sets × ways
```

Each memory line maps to exactly one set. Within that set, it may occupy one of the available ways.

```
set 0: [way 0] [way 1] [way 2] [way 3]
set 1: [way 0] [way 1] [way 2] [way 3]
...
```

- **Direct-mapped:** one way per set. One possible home.
- **N-way set associative:** N candidate slots in the selected set.
- **Fully associative:** one set containing every line. Any slot is possible; useful as a limiting model, uncommon for large ordinary data caches.

Associativity buys placement flexibility but costs tag comparisons, multiplexing, power, and metadata. It reduces a class of conflicts; it cannot make a finite cache infinite.

Tag, index, offset

For power-of-two geometry:

```
address = [            tag | set index | line offset ]
```

- **offset** chooses a byte within a line;
- **index** chooses one set;
- **tag** distinguishes all memory lines that map to that set.

The cache stores each resident line's data plus its tag and state bits. A lookup selects a set with the index, compares the requested tag against every valid way in that set, and uses the matching way's bytes at the requested offset.

The arithmetic form works even before drawing bit fields:

```
offset      = address mod line_size
line_number = floor(address / line_size)
set         = line_number mod number_of_sets
tag         = floor(line_number / number_of_sets)
```

Ways do not change these three values. Ways say how many different tags can reside in the selected set simultaneously.

Worked example

Geometry:

```
address width: 32 bits
line size:     64 B = 2^6
sets:          256  = 2^8
ways:          4
```

Therefore:

```
offset bits = 6
index bits  = 8
tag bits    = 32 - 6 - 8 = 18
```

For `0x12345678`:

```
offset = 0x12345678 mod 64             = 56
line   = floor(0x12345678 / 64)        = 0x0048d159
set    = 0x0048d159 mod 256            = 0x59 = 89
tag    = floor(0x0048d159 / 256)       = 0x48d1
```

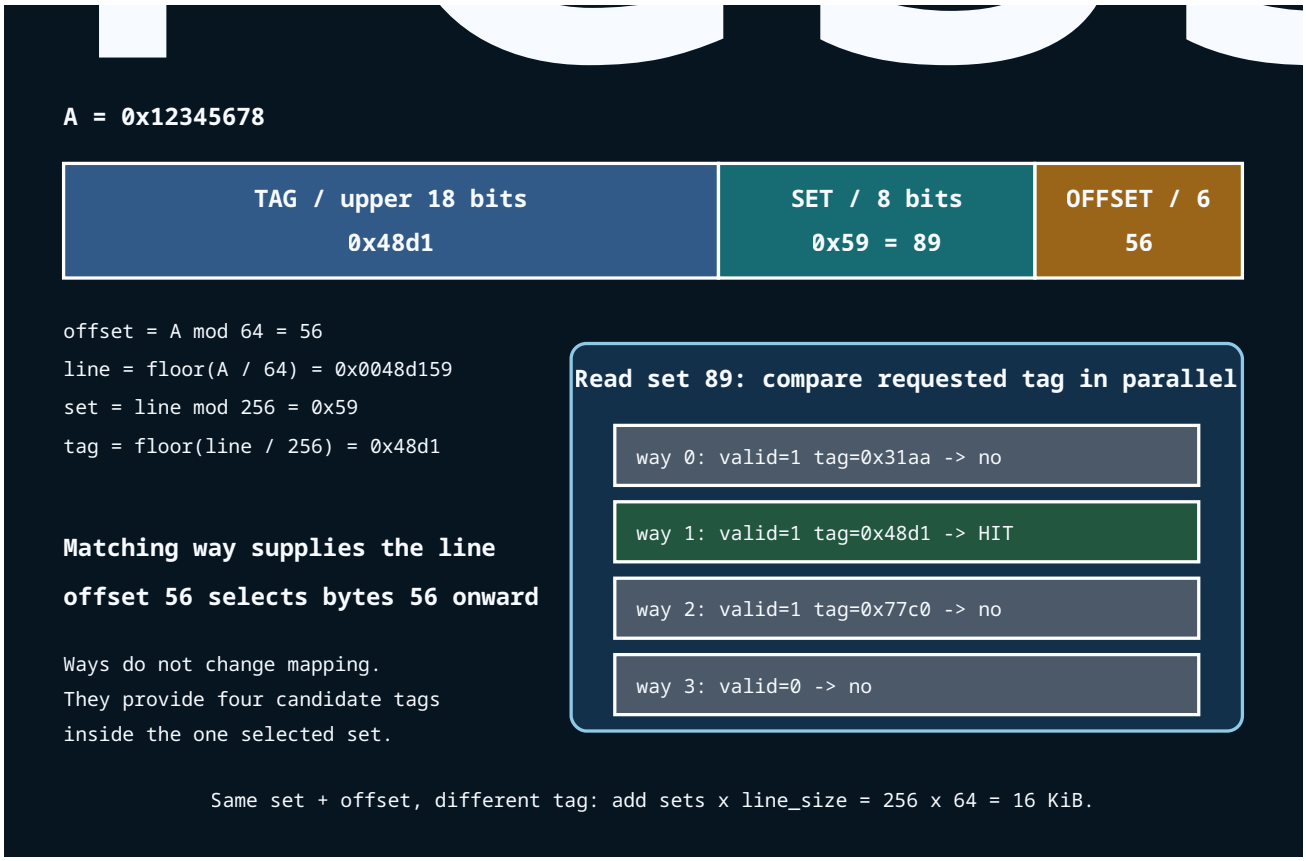
Check it:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Werror -O2 -pthread \
  02-assembly-memory-and-c/challenges/day-005-cache-lab.c -o /tmp/day5-cache
/tmp/day5-cache map 0x12345678 64 256 4
```

Now solve before running:

1. `0x0000abcd`, line 64, 64 sets, 1 way
2. `0xdeadbeef`, line 64, 512 sets, 8 ways
3. two addresses separated by `sets × line_size`: what fields match?

That third spacing is a **cache-sized stride per way**: the offset and index repeat while the tag changes. Enough such addresses can contend for one set.



The index selects one set, the tag identifies a resident line, and the offset selects bytes within that line.

---

3. Hit, fill, eviction

For a read:

```
extract index → read selected set's tags
              → tag matches a valid way?
                  yes: hit; select bytes by offset
                  no:  miss; fetch line from lower level
                       choose/fill a way
                       update tag and state
                       retry or satisfy waiting load
```

Real cores overlap, pipeline, speculate, prefetch, and support multiple outstanding misses. This diagram is an architectural teaching sequence, not a cycle-accurate description.

The lower level may be L2, a shared last-level cache, another core's cache through coherence, or DRAM. “Cache miss” does not mean “go directly to RAM.”

Three useful miss categories

The classic 3C model classifies misses relative to a cache model:

1. **Compulsory (cold):** this line has not yet been brought into the cache.
2. **Capacity:** the active collection of lines exceeds cache capacity; even a fully associative cache of that size would evict something needed later.

3. **Conflict:** the cache has enough total room, but restricted placement forces lines mapping to the same set to evict each other.

Examples:

- First pass over a new array: mostly compulsory demand misses, though hardware prefetch may obscure them in counters.
- Repeated pass over data far larger than the cache: capacity pressure.
- Alternating among five same-set lines in a four-way cache while the rest is empty: conflict pressure.

These categories explain a model; production measurements rarely label each event cleanly. Prefetching, inclusive/exclusive policies, victim structures, TLB misses, coherence, and undocumented replacement details blur the simple story.

---

4. Replacement: which resident line loses?

On a miss to a full set, one way must be replaced.

- **LRU** evicts the least recently used line. Exact LRU metadata becomes expensive as associativity grows.
- **Pseudo-LRU** approximates recency with less state.
- **Random** is simple and avoids some adversarial patterns.
- Practical policies may consider insertion position, reuse prediction, prefetch origin, or implementation-specific heuristics.

Do not assume the CPU uses exact LRU because a textbook exercise says “evict the oldest.” The exercise needs a stated policy; the actual processor needs its vendor documentation or measurement, and even then details may be undocumented.

Kernel page replacement is a different layer. The kernel decides which physical pages remain resident; CPU cache replacement decides which cache lines remain in a hardware cache. Similar vocabulary does not make them the same mechanism.

---

5. Writes create two separate policy questions

Hit policy

- **Write-through:** update the cache and propagate the write toward the next level immediately, usually buffered.
- **Write-back:** update the cached line and mark it dirty; write it to the next level when the line is evicted or otherwise cleaned.

Write-back reduces lower-level traffic when the same line is updated repeatedly, but requires dirty-state tracking and correct coherence.

Miss policy

- **Write-allocate:** fetch the line into the cache, then update it.
- **No-write-allocate (write-around):** send the write downward without filling this cache.

Common write-back caches use write-allocate, while write-through designs may pair with no-write-allocate. These are combinations, not definitions. Also distinguish ordinary cached memory from uncached/device mappings, where the architecture and kernel choose different memory types and ordering rules.

---

6. Coherence and false sharing

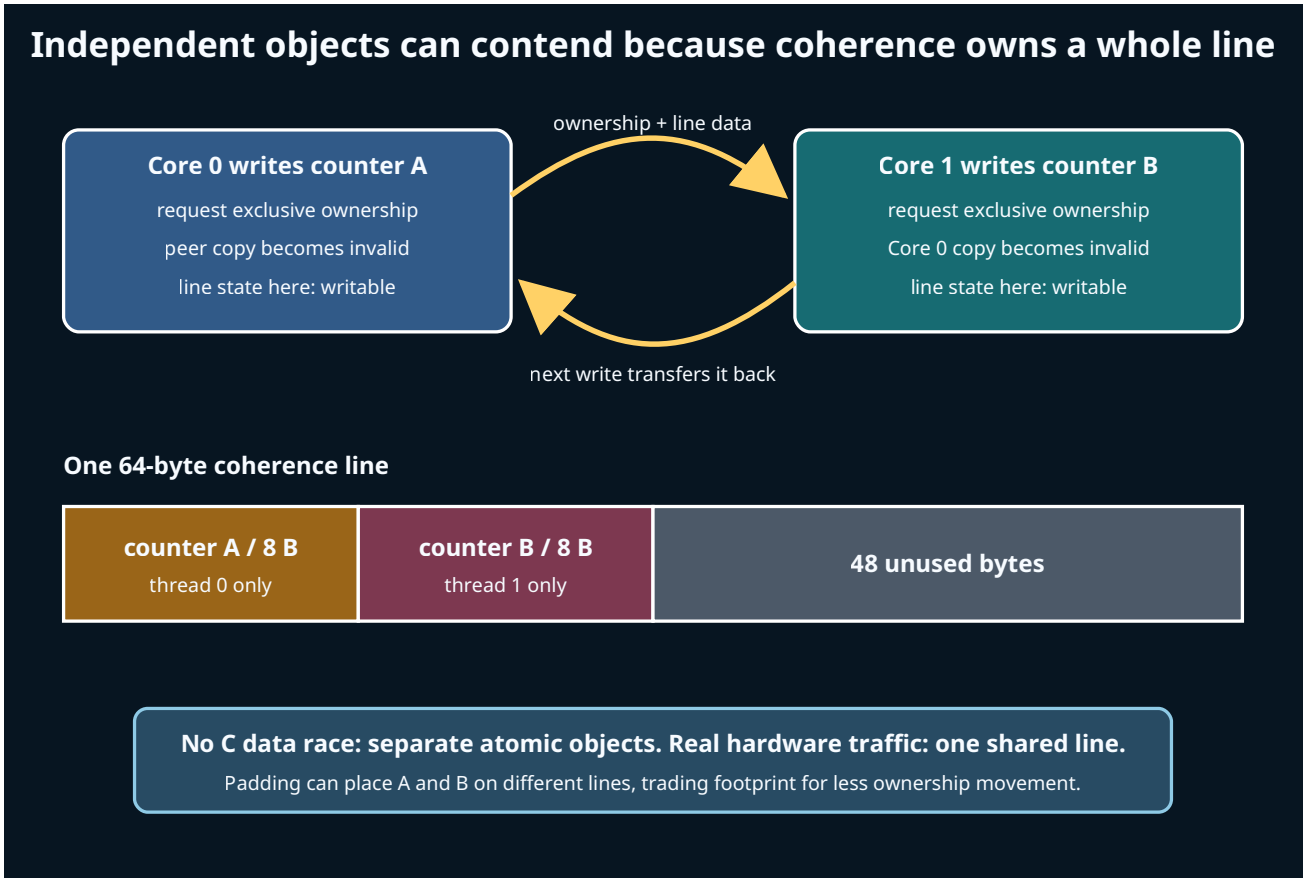
Multiple cores may hold copies of one physical line. A coherence protocol maintains a permitted view of those copies. To write, a core generally needs exclusive ownership; other cached copies must be invalidated or downgraded.

Now place two independent counters in one line:

```
64-byte cache line
┌──────────────────────┬──────────────────────┬──────────────┐
│ counter A (thread 0) │ counter B (thread 1) │ unused       │
└──────────────────────┴──────────────────────┴──────────────┘
```

The threads do not share a C object, but coherence works at line granularity. Each write can transfer ownership of the whole line. That is **false sharing**: logically independent writable data shares a coherence unit.

It is “false” only at the source-level sharing model; the hardware traffic is real. Padding or separating hot fields may help, but consumes memory and can harm locality elsewhere. Measure the actual workload.



Two independent counters still move one ownership unit because coherence operates at cache-line granularity.

Run the safe C11-atomic experiment:

```
/tmp/day5-cache false-sharing 20000000
taskset -c 0,1 /tmp/day5-cache false-sharing 20000000  # if two CPUs allowed
```

The atomics prevent a C data race; each thread owns a different atomic object. The compact and padded layouts change line placement, not language correctness.

On a one-vCPU VM, a container with restricted affinity, or sibling SMT threads with unusual topology, the difference may be small or noisy. Repeat, record CPU placement, and do not "fix" results to match the lesson.

Linux itself fights false sharing in hot shared structures with field reorganization, read-mostly grouping, per-CPU data, and batched updates. Those techniques trade coherence traffic against memory, complexity, and eventual aggregation cost.

---

7. Observe locality and counters

First run a software timing experiment:

```
/tmp/day5-cache locality 64 8
```

Predict before running:

- Why can a 1-byte stride have a different cost per *visited element* from a 64-byte stride?
- Why does a 4096-byte stride add TLB/page effects to the cache story?
- Why is elapsed time alone unable to identify one particular miss type?

Then ask `perf` which generic events it can expose:

```
perf list cache
perf stat -r 5 -e cycles,instructions,cache-references,cache-misses \
  /tmp/day5-cache locality 64 8
perf stat -r 5 -e cycles,instructions,cache-references,cache-misses \
  /tmp/day5-cache false-sharing 20000000
```

`perf` caveats are part of the lab

Generic `cache-misses` is not a portable synonym for "all L1 data misses." The kernel maps generic events to whatever the PMU supports. Events may be unsupported, multiplexed, counted speculatively, affected by skid, or denied by `perf_event_paranoid`. A hypervisor may hide the PMU entirely.

If `perf` reports `<not supported>` or permission denial:

1. keep the failure text as evidence;
2. run the timing and mapping modes;
3. inspect `perf list` and `/proc/sys/kernel/perf_event_paranoid`;
4. do not use `sudo` merely to force the course lab to work;
5. state that the environment cannot supply this evidence.

For serious false-sharing diagnosis on supported hardware, Linux provides `perf c2c`, which can attribute cache-to-cache activity and contended line offsets. It is more informative than comparing two wall-clock numbers, but it also depends on suitable PMU support and permissions.

---

8. CPU, Linux, kernel, and VM connections

```
C field/array layout
        ↓ determines addresses touched
virtual addresses
        ↓ translated by page tables and TLB
physical addresses
        ↓ select cache set/tag (exact stage is implementation-specific)
coherence domain
        ↓ coordinates line ownership among CPUs
memory controller/channel
        ↓ serves lower-level misses
Linux scheduler
        ↓ chooses where threads run
hypervisor
        ↓ may virtualize topology, time, and PMU access
```

Do not collapse cache indexing and virtual-memory translation into one universal formula. L1 caches are often virtually indexed and physically tagged; larger caches commonly use physical addressing, but exact designs are microarchitectural.

The scheduler can migrate a thread, changing which private caches are warm. Page allocation changes physical placement. Kernel per-CPU data avoids cross-core writes. A VM can make measured CPU time and reported topology less direct than bare metal. Cache behavior lives at the meeting point of all these layers.

---

9. Mastery evidence

Explain

Without notes:

- explain line, set, way, tag, index, and offset;
- distinguish direct-mapped, set-associative, and fully associative designs;
- distinguish compulsory, capacity, and conflict misses;
- separate write-through/write-back from write-allocate/no-write-allocate;
- explain false sharing without saying the two threads modify the same object;
- explain why `perf stat` output is evidence with scope, not ground truth.

Draw

Draw a four-set, two-way cache. Place memory lines 0, 4, and 8 into it. Label the common set, different tags, valid/dirty bits, and the replacement choice. Then draw two cores repeatedly acquiring one line that contains two counters.

Observe

Record:

- host-reported line size, set count, and associativity where available;
- one hand mapping checked by the challenge;
- repeated compact versus padded timings;
- `perf stat` output **or** the exact reason counters were unavailable;
- whether you were on bare metal, container, or VM as far as you can establish.

Build

1. Extend mapping mode to print binary fields for power-of-two geometry.
2. Generate five addresses with one offset/index and five tags.
3. Change padding from 64 to 128 bytes and predict whether correctness or only placement changes.
4. Add a single-thread mode to separate atomic instruction cost from inter-core coherence cost.

Why? notebook

1. Why fetch a whole line when the instruction asks for four bytes?
2. Why does associativity reduce conflict misses but not capacity misses?
3. Why is a tag required after the set has already been selected?
4. Why can two different addresses have the same index and offset?
5. Why can exact LRU become expensive?
6. Why are write-back and write-allocate separate decisions?
7. Why is false sharing a coherence problem rather than a C aliasing problem?
8. Why might padding make one workload faster and another worse?
9. Why can `cache-misses` differ across machines for the same binary?
10. Why can a VM hide the effect the source code was designed to expose?

Mental model at the end

```
address
  ├─ low bits choose byte offset
  ├─ next bits choose one set
  └─ remaining tag is compared across that set's ways
        ├─ match → hit
        └─ no match → miss, fill, perhaps evict/write back

multiple cores writing different words in one line
  → repeated ownership transfer
  → false sharing
```

References used selectively

Local:

- Bryant and O'Hallaron, *Computer Systems: A Programmer's Perspective*, 3rd ed., Chapter 6, especially §§6.3–6.6 on locality, cache memories, cache parameters, reads/writes, and performance. Local PDF: `references/library/books/01-foundations-and-architecture/computer-systems-programmers-perspective-bryant-ohallaron-3e-global.pdf`

Current official documentation consulted September 2026:

- Linux kernel, “False Sharing,” including `perf c2c`, `pahole`, mitigation tradeoffs, and kernel examples: https://docs.kernel.org/kernel-hacking/false-sharing.html
- `perf-stat(1)`, event selection, repetition, aggregation, and PMU caveats: https://man7.org/linux/man-pages/man1/perf-stat.1.html
- `lscpu(1)`, including cache and topology data sources: https://man7.org/linux/man-pages/man1/lscpu.1.html

Next bridge: cache lines already revealed that address and layout matter. Tomorrow asks who decides where an object begins, how bytes are ordered, which memory controller serves a physical page, and why “the RAM” may be nearer to one CPU than another.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Cache Lab

Format: C lab

What you will do: Day 5 cache lab: safe userspace experiments for mapping, locality, and

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
gcc -std=c17 -Wall -Wextra -O0 -g -pthread 02-assembly-memory-and-c/challenges/day-005-cache-lab.c
-lm -o /tmp/day-005-cache-lab && /tmp/day-005-cache-lab
```

- View lab source on GitHub

Day 6 — Alignment, Endianness, and NUMA

Curriculum alignment: Stage 4 — Memory Hierarchy and Performance · Alignment, Endianness, and NUMA

Target time: approximately 2 hours 45 minutes

- Alignment, padding, and ABI layout: 55 minutes
- Byte order and serialization: 40 minutes
- Controllers, channels, sockets, and NUMA: 45 minutes
- Layout and topology labs: 35 minutes
- Explain · Draw · Observe · Why: 10 minutes

> Yesterday, an address selected a cache line, set, and tag. Today we ask a prior question: who arranged the bytes at those addresses, and where in the machine will a cache miss eventually be served?

A C declaration looks logical: one character, one integer, one short, one double. Hardware sees byte addresses, naturally aligned transfers, cache lines, interconnects, memory controllers, and perhaps several unequal paths to DRAM. A network peer sees a byte sequence and cannot infer the sender's C ABI or native byte order.

The connecting idea is **layout**. Layout determines which addresses are touched. Those addresses influence whether an access is legal or efficient, whether two binaries agree, how bytes must be serialized, which cache lines are fetched, and—on a NUMA host—which memory controller may satisfy a miss.

Tracker objectives

By the end, you should be able to:

- **Explain alignment requirements** separately from alignment performance.
- **Recognize structure padding** and connect it to an ABI.
- **Understand byte order** and use an explicit wire format.
- **Describe NUMA implications**, including first touch and locality.

The tracker labs are present: inspect structure layouts, write and check endian conversions, and inspect the host with `lscpu`, `numactl`, `hwloc`, and safe fallbacks.

The mastery checkpoint is:

> **Explain how layout choices affect correctness, ABI compatibility, and performance.**

1. Alignment starts with addresses

An object is aligned to `A` bytes when its starting address is a multiple of `A`:

```
aligned(address, A)  ⇔  address mod A = 0
```

An eight-byte object at `0x1000` is eight-byte aligned. At `0x1004` it is four-byte aligned but not eight-byte aligned.

Why would a machine care? A naturally aligned access is easier to route through fixed-width hardware. A misaligned access may cross boundaries:

```
8-byte transfer lanes:

0x1000  [ 0 1 2 3 4 5 6 7 ] [ 8 9 a b c d e f ]
aligned load of 8 bytes:       └──────────────┘

0x1004  [ 0 1 2 3 4 5 6 7 ] [ 8 9 a b c d e f ]
misaligned load of 8 bytes:      └──────────────┘
                                    crosses a boundary
```

The real penalty depends on the microarchitecture and on which boundary is crossed. An access contained within one cache line may be cheap on a current x86-64 core. Crossing a cache-line boundary can require two line accesses. Crossing a page boundary may require two translations and can fault if the second page is absent. Atomicity guarantees can also be narrower for misaligned objects.

That gives us two different questions:

1. **Is the access valid under the language, ABI, and ISA?**
2. **If valid, how much does this machine charge for it?**

Never replace the first question with the second. “It worked quickly on my x86 laptop” does not prove that a C expression is defined or portable.

The C requirement

Every complete object type has an alignment requirement. C exposes it with `_Alignof(type)` in C11 and later (also spelled `alignof(type)` through `<stdalign.h>` in C11–C17). An object must be created at an address suitable for its type.

```
printf("%zu\n", _Alignof(uint32_t));
```

Allocators and declarations arrange this for ordinary objects. Trouble appears when code invents a typed pointer into a byte buffer:

```
unsigned char raw[8];
uint32_t value = *(uint32_t *)(raw + 1);  /* not a portable decoder */
```

The converted address may not satisfy `uint32_t` alignment. The expression can also run into C object/effective-type and representation rules. Some ISAs trap on some unaligned loads; others handle them in hardware; compilers may assume that a typed pointer is correctly aligned and optimize on that basis before the CPU ever executes an instruction.

For moving an object representation to or from bytes, `memcpy` is the portable tool:

```
uint32_t value;
memcpy(&value, raw + 1, sizeof value);
```

`memcpy` establishes the byte copy without dereferencing a misaligned `uint32_t *`. Compilers recognize fixed-size copies and commonly lower them to an efficient load when the target permits one.

ISA behavior is architecture-dependent

- x86 generally supports unaligned ordinary integer loads and stores, though boundary crossings can cost more and special atomic/vector rules matter.
- AArch64 supports many ordinary unaligned accesses to normal memory, but not every instruction or memory type behaves identically.
- Other architectures or modes may trap and let the operating system emulate an access, or may reject it.

The safe conclusion is not “architecture X always allows unaligned access.” It is: consult the language rule first, then the ISA instruction and memory type, then measure the actual processor.

Architecture lens — x86-64, AArch64, and RISC-V

x86-64 ordinary integer instructions generally tolerate unaligned addresses. AArch64 permits many unaligned accesses to normal memory but varies by access type and configuration. In RISC-V, naturally aligned accesses have the base guarantees, while the execution environment determines whether a misaligned access completes, traps for emulation, or fails. Even where an ISA permits the access, a particular core may charge extra for cache-line or page crossings.

Mainstream Linux deployments of all three are normally little-endian, but that is not a portable C or wire-format guarantee. The ISA defines available load/store behavior, an ABI fixes type alignment and aggregate layout for binary interoperability, and C still forbids an invalid typed access even when hardware could execute it. See Architecture Comparison.

---

2. Why structs contain bytes you did not declare

Consider:

```
struct mixed {
    char marker;
    uint32_t count;
    uint16_t code;
    double value;
};
```

Suppose this ABI gives `char`, `uint16_t`, `uint32_t`, and `double` alignments of 1, 2, 4, and 8 bytes. One plausible layout is:

```
offset   0       1 2 3       4 5 6 7       8 9      10 11 12 13 14 15
       ┌──────┬──────────┬──────────────┬─────────┬──────────────────────┐
       |marker|  padding |    count     |  code   |      padding         |
       └──────┴──────────┴──────────────┴─────────┴──────────────────────┘

offset  16 17 18 19 20 21 22 23
       ┌──────────────────────┐
       |         value        |
       └──────────────────────┘
```

The compiler inserts **internal padding** before a member when the next member needs a more aligned address. It may add **tail padding** after the last member so every element in an array of the struct has suitable alignment:

```
base of element i+1 = base of element i + sizeof(struct mixed)
```

`sizeof` includes padding. It must: pointer arithmetic on a struct array uses `sizeof` as the stride.

Reordering the fields from strictest to loosest alignment often reduces padding:

```
struct reordered {
    double value;
    uint32_t count;
    uint16_t code;
    char marker;
};
```

But “largest first” is a heuristic, not a language rule or always the best design. Layout changes can:

- break a stable binary interface or on-disk assumption;
- put fields used together on different cache lines;
- increase false sharing between threads;
- improve one array scan by shrinking the stride;
- worsen another access pattern by separating hot fields.

Measure the workload and preserve interfaces deliberately.

Layout belongs to an ABI, not just the source

C guarantees declaration order and forbids overlap between distinct members, but implementation and ABI rules determine concrete offsets, padding, alignment, and argument/return conventions. Separately compiled code interoperates because it follows the same ABI.

Use the compiler's results instead of guessing:

```
sizeof(struct mixed)
_Alignof(struct mixed)
offsetof(struct mixed, count)
```

Kernel and systems tooling can inspect compiled layouts too:

```
pahole /tmp/day6-layout 2>/dev/null || true
readelf --debug-dump=info /tmp/day6-layout 2>/dev/null | less
```

`pahole` needs debug information, so build with `-g`. Its absence is not a lab failure.

Packed is a protocol tool, not free compression

Compiler extensions such as `__attribute__((packed))` reduce or remove padding. That may be necessary for an explicitly specified hardware or wire layout, but a packed member can live at an address unsuitable for its type. Taking its address and dereferencing it as an ordinary typed pointer can be unsafe or slow.

Packing also does **not** define:

- integer byte order;
- integer widths unless fixed-width types are chosen;
- floating-point representation;
- bit-field allocation;
- protocol versioning;
- validation of untrusted lengths.

For a wire format, explicit encode/decode routines are clearer and more portable than sending a packed struct.

Padding bytes are real but not payload

Padding participates in `sizeof`, but its values need not represent any field. Consequences:

- `memcmp(&a, &b, sizeof a)` is not a valid general test of struct value equality;
- hashing every byte may include irrelevant padding;
- writing a struct directly to disk or a socket leaks an ABI-specific format and can expose uninitialized padding;
- changing compiler, ABI, options, or architecture can change the format.

Compare fields. Serialize fields.

---

3. Endianness answers one narrow question

Memory is byte-addressed. A multi-byte integer must assign its significance to increasing byte addresses.

For the value `0x12345678`:

```
address             A      A+1    A+2    A+3

big-endian         0x12   0x34   0x56   0x78
little-endian      0x78   0x56   0x34   0x12
```

Big-endian stores the most significant byte at the lowest address. **Little-endian** stores the least significant byte there.

Endianness does not mean that bits run backward inside each byte. It does not by itself specify bit-field layout, instruction encoding, text encoding, or the order in which a CPU performs operations.



ABI padding places members at suitable addresses; endianness separately orders each multi-byte value's representation.

Observe an object's representation legally through `unsigned char` bytes or `memcpy`:

```
uint32_t word = UINT32_C(0x01020304);
unsigned char bytes[sizeof word];
memcpy(bytes, &word, sizeof bytes);
```

If `bytes[0]` is `0x04`, this run observed little-endian order for that object. If it is `0x01`, it observed big-endian order.

Network byte order

Internet protocols define integers in **network byte order**, which is big-endian. Socket interfaces provide conversions:

```
uint16_t port_wire = htons(port_host);
uint32_t addr_wire = htonl(addr_host);

uint16_t port_host = ntohs(port_wire);
uint32_t addr_host = ntohl(addr_wire);
```

On a little-endian host these commonly compile to a byte swap. On a big-endian host they may compile to no operation. Correct source uses the conversion either way because the source states the boundary between host representation and protocol representation.

The names handle 16- and 32-bit unsigned quantities. For 64-bit values or a file format, use a specified library/API or encode bytes explicitly; do not assume a nonstandard helper exists everywhere.

Serialization is more than byte swapping

A robust format specifies:

```
field order
field widths and signedness
byte order
length and bounds rules
optional/versioned fields
text encoding
floating-point representation, if any
error handling
```

For a four-byte big-endian field:

```
uint32_t decode_be32(const unsigned char p[4])
{
    return ((uint32_t)p[0] << 24) |
           ((uint32_t)p[1] << 16) |
           ((uint32_t)p[2] << 8)  |
           (uint32_t)p[3];
}
```

This describes the format directly. It does not depend on host endianness, struct padding, alignment, or a type-punned load.

Before decoding untrusted input, prove that all required bytes are present. Byte-order correctness does not prevent an out-of-bounds read.

---

4. Run the layout and byte-order lab

The existing lab avoids undefined behavior:

```
02-assembly-memory-and-c/challenges/day-006-layout-endian-lab.c
```

Build and run:

```
gcc -std=c17 -Wall -Wextra -Wpedantic -Werror -O2 -g \
  02-assembly-memory-and-c/challenges/day-006-layout-endian-lab.c \
  -o /tmp/day6-layout

/tmp/day6-layout
```

Predict first:

1. What are the offsets and total size of `struct mixed`?

2. Which padding disappears in `struct reordered`?
3. What four bytes represent `0x01020304` on this host?
4. Why does the `memcpy` from `unaligned + 1` remain safe when a `uint32_t *` dereference would not be portable?

Then preserve the output. Results from one build are evidence about this compiler/target ABI, not a universal C layout.

Inspect what the compiler emitted

```
objdump -drwC -Mintel /tmp/day6-layout | less
readelf -h /tmp/day6-layout | grep 'Data:'
file /tmp/day6-layout
```

Find the fixed-size `memcpy` operation in the disassembly. The compiler may inline it. That is the payoff of expressing the operation legally: the optimizer can still select an efficient target instruction.

Extend safely

Add a function that encodes a `uint32_t` into four explicitly big-endian bytes, then decode it with the existing expression. Check these values:

```
0x00000000
0x00000001
0x01020304
0x80000000
0xffffffff
```

Do not reveal correctness by printing only. Return nonzero when any round-trip fails, so the lab can be used in a script or test runner.

---

5. From a cache miss to a memory controller

“RAM” is not one uniform bucket. A simplified current server path is:

```
load instruction
      ↓ virtual-to-physical translation
private cache(s)
      ↓ miss
shared cache / coherence fabric
      ↓ miss or remote request
socket/package interconnect
      ↓
integrated memory controller
      ↓ selects channel/rank/bank/row
DRAM module
```

Exact organization varies. A **memory controller** schedules requests and speaks the DRAM protocol. A controller may expose multiple **channels** that can transfer in parallel. DIMMs, ranks, banks, rows, and columns add more levels.

More populated channels can increase available memory bandwidth, but only when the platform's population rules and workload permit parallel use. Channel count is not NUMA node count:

- channels are parallel paths below a controller;
- a NUMA node is an operating-system-visible locality domain containing CPUs and/or memory;
- one NUMA node can have several channels;
- firmware and platform design decide what Linux reports.

Commodity tools often report CPU and NUMA topology but not reliable channel population. Use firmware/vendor tools or hardware documentation when channel details matter; do not infer channels from `lscpu`'s NUMA output.

---

6. Packages, cores, threads, and nodes are different

Linux and hardware tools use several topology levels:

- **logical CPU / processing unit (PU):** one schedulable hardware thread;
- **core:** an execution core; it may expose one or more logical CPUs through simultaneous multithreading;
- **socket/package:** a physical processor package containing cores, caches, and often one or more memory controllers;
- **NUMA node:** CPUs and memory grouped by access locality.

These relationships are common, not universal:

```
Machine
├── Package 0
│   ├── NUMA node 0: cores + local memory
│   └── NUMA node 1: cores + local memory
└── Package 1
    ├── NUMA node 2: cores + local memory
    └── NUMA node 3: cores + local memory
```

A package can contain multiple NUMA nodes. A virtual machine may expose one virtual node over hardware with a different shape. A container normally sees the host kernel's topology filtered by CPU and memory constraints.

In a **uniform** memory model, access cost is treated as approximately equal for every CPU-memory pairing. In **non-uniform memory access (NUMA)**, latency and sustainable bandwidth depend on where the CPU runs and where the physical page resides:

```
CPU on node 0 → memory on node 0     local path
CPU on node 0 → memory on node 1     interconnect + remote controller
```

Remote memory is still ordinary coherent memory. NUMA affects cost, not whether a valid pointer works.

---

7. First touch: virtual reservation is not physical placement

An anonymous `mmap` or a large `malloc` can reserve virtual address space without immediately assigning a private physical page to every virtual page. Linux allocates pages on demand.

Under the default local policy, an anonymous page is generally allocated near the CPU that triggers allocation. For writable anonymous memory, the first write commonly causes that private allocation:

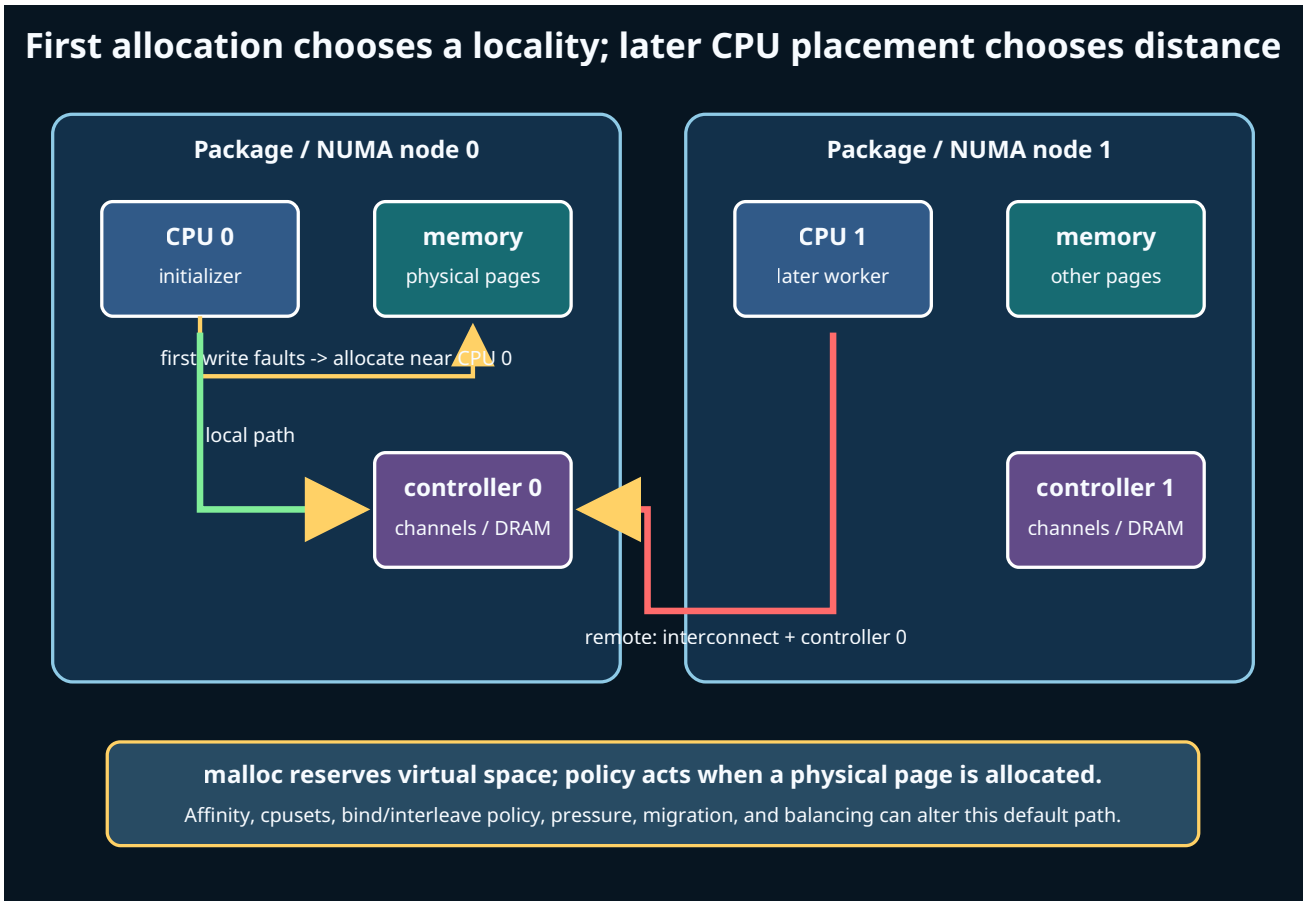
```
thread running on node 0
        ↓ first write faults on virtual page
kernel allocates a physical page under current memory policy
        ↓ usually local to node 0 when available and allowed
later thread runs on node 1
        ↓ accesses the same page remotely
```

This is the practical **first-touch** rule. State it with its conditions:

- placement follows the policy active when a new page is allocated;
- a first read of untouched anonymous memory may use the kernel's shared zero page rather than allocate the final private page;
- cpusets, cgroups, affinity, memory pressure, explicit policies, huge pages, migration, and automatic NUMA balancing can change the result;
- fallback may allocate from another allowed node when local memory is unavailable.

Therefore initialization strategy matters. If one thread initializes an entire large array on node 0 and workers later split across all nodes, many workers may use remote memory. Parallel initialization by the same workers that later process each region often establishes better locality.

First touch is not a source-level ownership rule, and it does not guarantee a page stays put forever.



Allocation-time CPU and policy choose an initial locality; execution-time CPU placement determines whether access is local or remote.

Linux NUMA policy

Linux supports task and address-range policies including:

- **local/default:** prefer allocation local to the allocating CPU;
- **bind:** allocate from a specified set of nodes, subject to policy rules;
- **preferred:** try preferred node(s), with fallback;
- **interleave:** distribute allocations across nodes, useful for bandwidth or avoiding one-node concentration.

`numactl` exposes these policies for a launched process. Binding CPUs and binding memory are separate actions:

```
numactl --cpunodebind=0 --membind=0 -- command
numactl --interleave=all -- command
```

Do not run these examples mechanically on node 0. First inspect which nodes exist and are allowed. Strict memory binding can fail when a node lacks available memory, and a container may prohibit it.

---

8. Inspect the host without assuming special tools

Run the read-only helper:

```
bash 02-assembly-memory-and-c/challenges/day-006-topology-probe.sh \
  | tee /tmp/day6-topology.txt
```

It tries richer tools when installed and falls back to `/proc` and `/sys`. It changes no affinity or memory policy.

You can also run the core commands individually:

```
lscpu
lscpu -e=CPU,NODE,SOCKET,CORE,ONLINE
lscpu --caches

numactl --hardware
numactl --show

lstopo-no-graphics
# or:
lstopo
```

Interpret failures as evidence:

- **command not found:** the optional package is absent;
- **one NUMA node:** this host or VM exposes uniform topology to the guest;
- **empty node distance files:** firmware or virtualization may not expose the data;
- **restricted CPU list:** a container/cgroup or affinity mask limits what this process can use;
- **policy operation not permitted:** the environment does not authorize it.

`lscpu` gathers from sysfs, `/proc/cpuinfo`, and architecture-specific libraries. `hwloc` builds a richer hierarchy and may reveal cache sharing and I/O locality. Neither tool turns a virtualized topology into a bare-metal fact.

Read the sysfs representation

```
cat /sys/devices/system/cpu/online 2>/dev/null
cat /sys/devices/system/node/online 2>/dev/null

for node in /sys/devices/system/node/node[0-9]*; do
    [ -d "$node" ] || continue
    printf '%s cpulist=' "${node##*/}"
    cat "$node/cpulist" 2>/dev/null || printf 'unavailable\n'
done
```

Node-distance matrices, when exposed:

```
for f in /sys/devices/system/node/node[0-9]*/distance; do
    [ -r "$f" ] || continue
    printf '%s: ' "$f"
    cat "$f"
done
```

The numbers are firmware-provided relative costs, not nanoseconds. Lower normally means nearer. Compare relationships; do not label a value “latency in ns.”

Optional NUMA behavior experiment

Only do this when at least two nodes are available, `numactl` is installed, and the allowed CPU/memory masks include them.

Choose a memory-bandwidth benchmark already installed, or write one that allocates a large buffer, writes every page, then repeatedly scans it. Record:

1. which node the initializer ran on;
2. which node the scanning thread ran on;
3. local and remote elapsed times over several repetitions;
4. buffer size, page size, CPU list, and node-distance matrix.

Launch variants using nodes that actually exist:

```
numactl --cpunodebind=NODE_A --membind=NODE_A -- ./scan
numactl --cpunodebind=NODE_B --membind=NODE_A -- ./scan
numactl --cpunodebind=NODE_A --interleave=all -- ./scan
```

Do not expect a dramatic result. Cache residency, hardware prefetch, thread migration, automatic NUMA balancing, VM topology, page migration, frequency changes, and workload size can obscure the difference. A null result is not permission to invent one.

---

9. One causal chain from declaration to DRAM

```
C member order and ABI
        ↓ choose offsets, padding, alignment, total stride
compiler-generated loads/stores
        ↓ touch particular virtual byte addresses
MMU and page tables
        ↓ select physical pages
cache line geometry
        ↓ combines nearby fields; may split or falsely share them
NUMA placement policy + first allocation
        ↓ place a physical page in a locality domain
CPU placement
        ↓ determines local or remote path
memory controller and channels
        ↓ schedule DRAM transfers
```

This is why a “small” source decision can be visible many layers below:

- reordering a struct changes array stride;
- stride changes how many objects fit in a cache line;
- cache-line use changes bandwidth and coherence traffic;
- page initialization changes physical placement;
- scheduler placement changes the distance to those pages.

None of those effects makes one universal best layout. The correct design starts with correctness and interface stability, then measures the relevant access pattern on the target system.

---

10. Mastery evidence

Explain

Without notes:

- separate C alignment validity from CPU unaligned-access performance;
- explain internal and tail padding, including why arrays need tail padding;
- explain why an ABI needs stable member offsets;
- explain why packed structs do not solve serialization;
- draw `0x12345678` in little- and big-endian memory;
- explain why network byte order conversions appear even on a big-endian host;
- distinguish logical CPU, core, package, NUMA node, controller, and channel;
- explain first touch without claiming physical memory is allocated at `malloc` time;
- explain why remote NUMA memory remains coherent but can cost more.

Draw

Draw:

1. `struct mixed` byte by byte, with member offsets and padding;
2. the same fields reordered, with `sizeof` and alignment;
3. one four-byte integer in both byte orders;
4. two packages, four NUMA nodes, local memory controllers, and one remote access crossing the interconnect;
5. the declaration-to-DRAM causal chain from section 9.

Observe

Keep:

- the C lab's `sizeof`, `_Alignof`, `offsetof`, and byte output;
- `file`/ELF byte-order evidence;
- `/tmp/day6-topology.txt`;
- online/allowed CPUs and nodes;
- a node-distance matrix or a statement that none was exposed;
- whether the environment appears to be bare metal, VM, or container;
- optional local/remote NUMA measurements, including null results.

Build

1. Add checked big-endian encode/decode round trips to the C lab.
2. Add a third struct order and predict its layout before compilation.
3. Place two `struct mixed` objects in an array and verify that the second address differs by exactly `sizeof(struct mixed)`.
4. Write a scanner whose initialization and read phases can be independently CPU-bound; run it only under safe, available NUMA policies.

Why? notebook

1. Why can a misaligned typed C access be wrong even when one CPU instruction would tolerate the address?
2. Why can crossing a page boundary be more serious than crossing an eight-byte boundary?
3. Why does a struct sometimes need padding after its final member?
4. Why can smaller structs improve bandwidth yet worsen false sharing?
5. Why is `memcmp` not general struct equality?
6. Why can a packed struct still fail as a network format?
7. Why is endianness about byte significance rather than bit reversal?
8. Why must serialization define lengths and versions as well as byte order?
9. Why are memory channels and NUMA nodes not interchangeable terms?
10. Why can one socket expose multiple NUMA nodes?
11. Why can single-threaded initialization create remote accesses later?
12. Why might the first read of anonymous memory not establish the final private-page placement?
13. Why can a VM report a topology that is internally consistent but not describe the physical host?
14. Why is a node-distance value not a latency in nanoseconds?

Mental model at the end

```
layout is a contract and a performance input

type alignment + ABI
    → member offsets + padding + array stride
    → addresses and cache-line placement

serialized format
    → explicit widths + field order + byte order + bounds
    → independent of native struct layout

page allocation policy + CPU at first allocation
    → physical NUMA placement
CPU running the access
    → local or remote controller path
```

The mastery connection is now complete: **data layout determines hardware access**. It does so through several contracts—C, ABI, serialization format, virtual memory policy, and hardware topology—not through one magical property of the source declaration.

References used selectively

Local:

- Bryant and O'Hallaron, *Computer Systems: A Programmer's Perspective*, 3rd ed., §3.9.3 on data alignment and Chapter 6 on the memory hierarchy. Local PDF: `references/library/`

`books/01-foundations-and-architecture/computer-systems-programmers-perspective-bryant-ohallaron-3e-global.pdf`

Current official documentation consulted September 2026:

- Linux kernel, “NUMA Memory Policy,” including policy scope, local allocation, range policies, cpuset interaction, and allocation-time behavior: https://docs.kernel.org/admin-guide/mm/numa_memory_policy.html
- `lscpu(1)`, including CPU, cache, socket, and NUMA topology data sources: https://man7.org/linux/man-pages/man1/lscpu.1.html
- `numactl(8)`, process CPU binding and NUMA memory-policy options: https://man7.org/linux/man-pages/man8/numactl.8.html
- hwloc current command-line tools, including `lstopo` and `hwloc-bind`: https://hwloc.readthedocs.io/en/master/doxygen/html/tools.html
- Linux kernel sysfs ABI, node topology and distance interfaces: https://docs.kernel.org/admin-guide/abi-stable-files.html
- POSIX byte-order conversion interfaces: https://pubs.opengroup.org/onlinepubs/9799919799/functions/htonl.html

Next bridge: layout showed that C source does not fully determine machine behavior. Tomorrow turns that observation into the C execution model: representations, integer conversions, promotions, scope, lifetime, and the implementation-defined choices a systems programmer must identify rather than guess.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Layout Endian Lab

Format: C lab

What you will do: Day 6: observe C layout and byte order without undefined behavior.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
gcc -std=c17 -Wall -Wextra -O0 -g -pthread 02-assembly-memory-and-c/challenges/day-006-layout-endian-lab.c -lm -o /tmp/day-006-layout-endian-lab && /tmp/day-006-layout-endian-lab
```

- View lab source on GitHub

Put it into practice — Topology Probe

Format: shell lab

What you will do: Day 6: read-only CPU, cache, and NUMA topology probe.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 02-assembly-memory-and-c/challenges/day-006-topology-probe.sh
```

- View lab source on GitHub

Day 7 — C's Execution Model and Data Types

Curriculum alignment: Stage 5 — C from the Machine's Perspective · C Execution Model and Data Types

Target time: approximately 2–3 hours

- C's abstract machine and environments: 30 minutes
- objects, values, types, bytes, and integers: 45 minutes
- conversions, expressions, and floating-point orientation: 35 minutes
- compile/run/disassemble lab: 45–60 minutes
- scope, lifetime preview, and mastery evidence: 20 minutes

> The processor executes instructions, not C statements. The C standard describes an abstract machine, not your particular x86-64 laptop. So what exactly did you ask for when you wrote `int x = -1;`, and which parts may a compiler choose?

This week moved from assembly instructions through caches, layout, and byte order. Today we climb back to C—but with enough machine knowledge to avoid a dangerous shortcut:

> A C program constrains observable behavior. It does not prescribe one instruction per source line, one register per variable, or even that every source object occupies memory throughout execution.

That distinction is the point of the lesson.

Tracker contract

By the end, you should be able to:

- connect C types to representation and generated machine operations;
- describe the C abstract machine and distinguish translation from execution;
- distinguish objects, values, types, representations, and identifiers;
- reason about integer ranges, rank, promotions, and common conversions at an introductory level;
- explain what C means by a byte and why `char` is special;
- recognize implementation-defined choices and avoid assuming every choice is universal;
- predict selected expression results without relying on accidental evaluation order;
- preview scope, storage duration, and lifetime;
- inspect generated assembly without expecting a line-by-line transcription.

The tracker mastery checkpoint is:

> **Explain why C source does not fully specify every machine-level detail.**

Today's sharper evidence question is:

> **Given a C expression, explain how its types affect the width, interpretation, extension, comparison, and instruction family that a compiler may choose.**

---

1. Why C needs a machine that does not physically exist

If C directly described one 1970s processor, programs would become obsolete with that processor. If it described only pure mathematics, it would be a poor language for device registers, packed bytes, operating systems, and memory allocators.

C instead defines an **abstract machine**. It is a specification model with:

- objects that hold values;
- types that constrain values and operations;
- expressions whose evaluations may compute values or produce side effects;
- control flow;
- rules for observable interactions such as input/output and access to volatile objects;
- areas where an implementation must choose and document behavior;
- areas where the program itself has no defined meaning.

A compiler's job is to preserve the behavior required by that model for the target implementation. The physical route may be radically different:

```
C source constraints
        ↓ front end checks syntax, types, and semantics
intermediate representation
        ↓ optimization preserves required observable behavior
target instructions, data, and metadata
        ↓ assembler + linker + loader
process executing on an ISA and operating system
```

The middle can delete variables, fold expressions at translation time, replace loops with another computation, keep values only in registers, or emit no instructions for a source statement whose result cannot matter.

This is the **as-if rule** in practical form: an implementation may transform the program however it likes *as if* the required observable behavior had occurred. It is not permission to change a defined result. It is freedom over how that result is obtained.

Three categories you must keep separate

Defined behavior has a meaning imposed by the language. Unsigned integer addition wraps modulo one more than the maximum value.

Implementation-defined behavior is a choice the implementation must make and document. Whether plain `char` behaves like `signed char` or `unsigned char` is one example.

Undefined behavior imposes no requirements on the implementation. Signed integer overflow is a classic example. It is not a portable wrapping request.

There is also **unspecified behavior**, where the implementation may choose among allowed possibilities without documenting which choice it made—for example, the order in which many function arguments are evaluated.

Those categories are language rules, not CPU exception classes. A processor may naturally wrap a signed `add`, yet the optimizer may reason that a well-defined C execution never overflows and transform surrounding code.

---

2. Translation environment and execution environment

The C model distinguishes two broad environments.

Translation environment

This is where source becomes an executable form:

```
source files
  → preprocessing directives and macro expansion
  → translation units
  → compiler analysis and code generation
  → assembler output / object files
  → linking
```

Some facts are resolved here:

- `_Static_assert` either succeeds or rejects the translation;
- `sizeof(int)` is known for the chosen target;
- constant expressions may be folded;
- diagnostics are issued for required constraint violations;
- implementation choices such as integer widths affect generated code.

This is not necessarily the same machine as the one that executes the program. Cross-compilers make that obvious: an x86-64 host can translate C for RISC-V.

Execution environment

This is where the translated program runs. The C standard discusses two broad forms:

- a **hosted** environment, normally with an operating system and a standard library, where program startup reaches `main`;
- a **freestanding** environment, common in kernels, boot code, and embedded systems, where startup and available library facilities are implementation-defined.

Linux userspace is hosted. The Linux kernel is not an ordinary hosted C program: it supplies its own entry paths, linker script, runtime support, and restricted dialect/build assumptions.

The translation/execution split prevents a common mistake: `sizeof(long)` is a property of the **target C implementation**, not necessarily the computer running the compiler process.

---

3. Object, value, type, identifier, and representation

These words are related but not interchangeable.

Consider:

```
int count = 5;
```

- `count` is an **identifier**, a source-level name.
- The declaration creates an **object** during its applicable lifetime.
- The object's **type** is `int`.
- Its stored **value** is initially `5`.
- Its **object representation** is the sequence of `unsigned char`-sized units that stores that value.

An object is a region of data storage whose contents can represent values. The compiler may optimize away any literal memory region when doing so preserves required behavior. The abstract object still helps define the program even if no stack slot survives into the assembly.

Values are interpretations, not free-floating bit meanings

The same bit pattern can participate in different interpretations:

```
0xffffffff
  as uint32_t on a common implementation → 4294967295
  as a 32-bit signed two's-complement value → -1
  as four character bytes                → encoding-dependent data
  as part of a float representation      → a floating encoding
```

Do not say “the bits are signed.” Bits are bits. A C type and an operation determine how those bits are interpreted and which result is required.

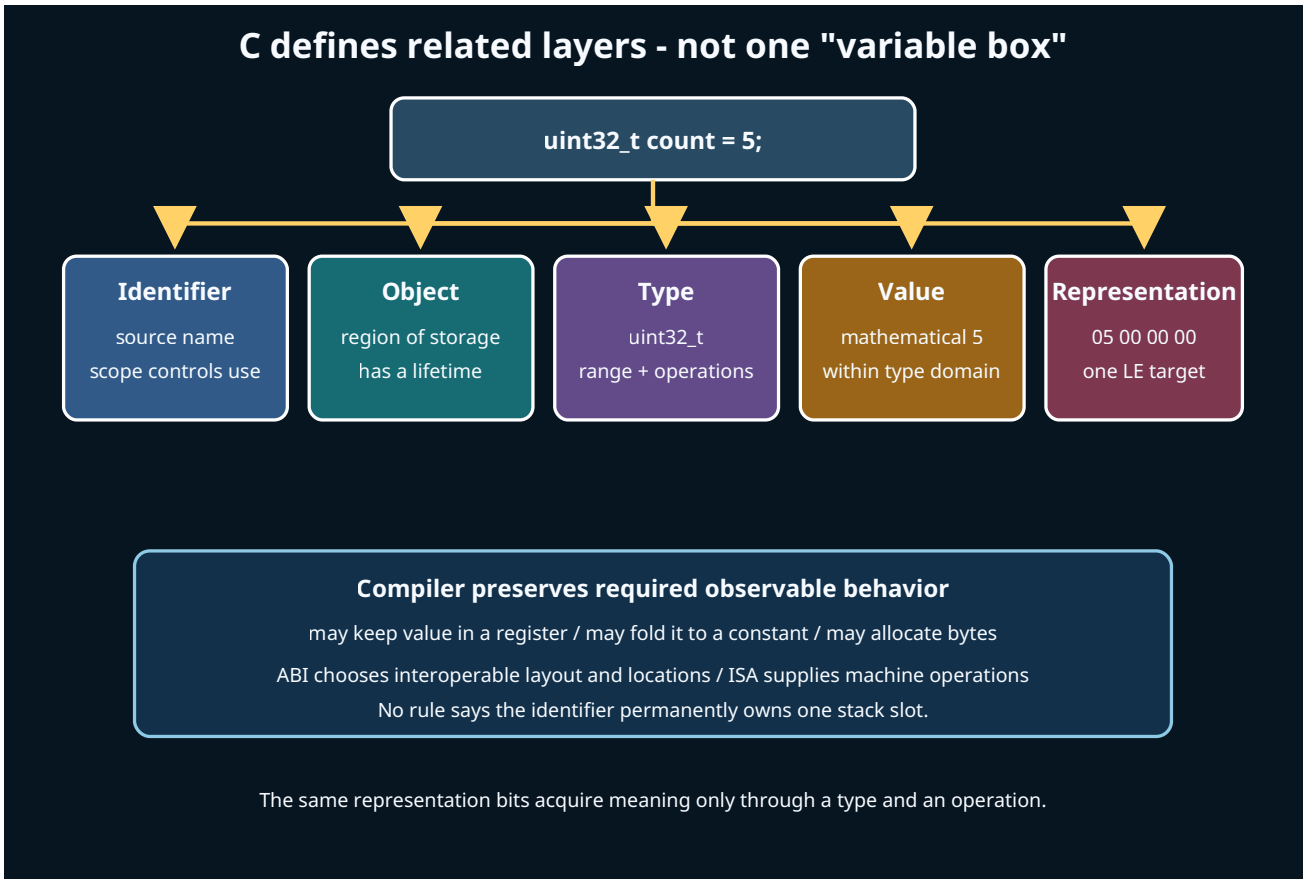
Not every type must use every possible bit pattern as an ordinary value, and padding bits or trap representations matter on some types and implementations. Today's lab avoids manufacturing typed values from arbitrary bytes. It observes existing objects through `unsigned char` and uses `memcpy` to copy representations safely.

Type affects more than storage size

A type helps determine:

- the set of values;
- representation and alignment requirements;
- which operations are permitted;
- how operands are converted;
- how arithmetic and comparisons are interpreted;
- pointer scaling and aliasing rules;
- often, but not uniquely, which machine instruction widths and families are useful.

This is why “`int` is four bytes” is not an adequate definition of `int`.



An identifier names an abstract object; its type, value, and bytes are related layers, not interchangeable terms.

---

4. C bytes and the character types

In C, `sizeof(char)`, `sizeof(signed char)`, and `sizeof(unsigned char)` are all exactly `1`. That does **not** say one C byte is always eight bits.

`CHAR_BIT`, from `<limits.h>`, tells you how many bits are in a byte for the implementation. It is at least 8. On the x86-64 Linux system used for this course it will almost certainly be 8, but the lesson asks you to measure rather than universalize.

```
printf("CHAR_BIT=%d\n", CHAR_BIT);
```

Consequences:

- `sizeof object` reports a count of C bytes;
- `sizeof(char)` cannot reveal the number of bits in a byte;
- an N-byte object contains `N * CHAR_BIT` bits in its object representation;
- `uint8_t` exists only when the implementation provides an unsigned integer type of exactly 8 bits with no padding.

There are three distinct character types:

- `char`;
- `signed char`;

- `unsigned char`.

Plain `char` has the same range/representation/behavior as one of the other two on a given implementation, but it remains a distinct type. Use `char` for character data, and use `unsigned char` when inspecting raw object representations byte by byte.

That last permission is crucial. Converting an object's address to `unsigned char *` lets code inspect each byte of its representation:

```
const unsigned char *bytes = object;
for (size_t i = 0; i < sizeof value; ++i)
    printf("%02x ", bytes[i]);
```

The resulting byte order is evidence about this implementation. It is not a portable serialization format.

---

5. Integer types: range first, encoding second

C's standard signed integer types, in increasing rank, are:

```
signed char < short < int < long < long long
```

Each has a corresponding unsigned type. The exact width of most is chosen by the implementation, subject to minimum ranges and ordering constraints. Common x86-64 Linux data models use:

```
char       8 bits
short     16 bits
int       32 bits
long      64 bits
long long 64 bits
pointer   64 bits
```

That is commonly called LP64. It is not universal: 64-bit Windows commonly uses 32-bit `long` in an LLP64 model.

Ask the implementation:

```
sizeof(short)
sizeof(int)
sizeof(long)
INT_MIN, INT_MAX, UINT_MAX
```

Use `<limits.h>` for fundamental integer limits and `<stdint.h>` when an exact or minimum width is genuinely part of the requirement:

- `int32_t`: exactly 32 bits, if provided;
- `uint32_t`: exactly 32 bits, if provided;
- `int_least32_t`: smallest available type with at least 32 value bits;
- `int_fast32_t`: implementation's fast type with at least 32 value bits;
- `intptr_t`: integer capable of round-tripping `void *`, if provided.

Exact-width names are useful for file formats, registers, and protocols, but do not automatically solve endianness, alignment, padding, or serialization.

Unsigned arithmetic is modular

For an unsigned type with maximum value `U_MAX`, arithmetic is reduced modulo `U_MAX + 1`:

```
uint32_t x = UINT32_MAX;
x = x + 1;                /* defined result: 0 */
```

That supports counters, masks, and low-level arithmetic. It does not mean every accidental wrap is logically correct.

Signed overflow is different

If a signed arithmetic result is not representable in its type, behavior is undefined. Do not reason:

```
the target add instruction wraps
therefore signed C overflow wraps
```

The compiler reasons from C semantics before choosing the target instruction. If modular behavior is required, express the relevant operation in a suitable unsigned type and convert deliberately.

---

6. Integer rank, promotions, and usual arithmetic conversions

Most surprising integer expressions become manageable if you ask two questions in order:

1. What type does each operand have *after promotion*?
2. What common type do the binary operands use?

Integer promotions

Types with rank below `int`—such as `unsigned char` and `short`—usually do not remain narrow during arithmetic.

If `int` can represent all values of the original type, the operand promotes to `int`; otherwise it promotes to `unsigned int`.

On the course machine:

```
unsigned char a = 250;
unsigned char b = 10;
int sum = a + b;            /* 260, not an 8-bit wrap */
```

Both operands promote to `int` before addition. Narrowing occurs only if the result is later converted back:

```
unsigned char narrowed = a + b;  /* conversion yields implementation's
                                    unsigned-char modulo result */
```

Integer promotions also apply to unary `+`, unary `-`, `~`, shift operands, and variadic arguments in relevant contexts. This is why generated code may load a byte and then compute with a 32-bit register.

Usual arithmetic conversions

For many binary arithmetic and comparison operators, C finds a common real type. At an introductory level:

1. floating operands may cause integer operands to convert to a floating type;
2. otherwise integer promotions happen;
3. if promoted integer types match, use that type;
4. same signedness generally selects the higher-rank type;
5. mixed signed/unsigned cases depend on rank and representable ranges.

The famous example:

```
int s = -1;
unsigned int u = 1;
printf("%d\n", s < u);
```

When `int` and `unsigned int` have the same rank, `s` converts to `unsigned int`. On the common 32-bit `unsigned int` implementation, `-1` becomes `UINT_MAX`, so the comparison is false.

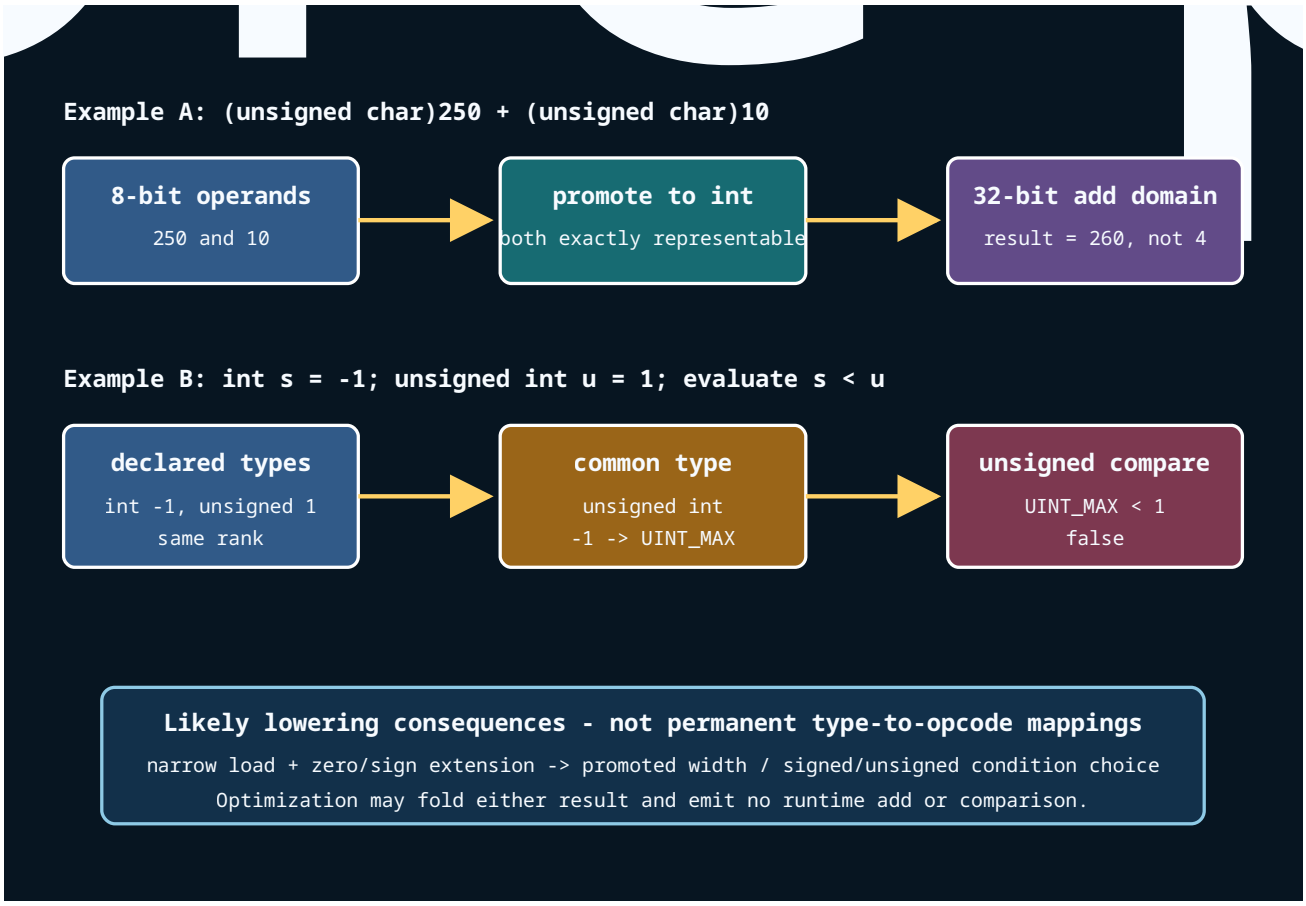
Nothing changed the stored object `s`. The conversion applies to the value used by this expression.

A disciplined prediction method

For every nontrivial expression, write:

```
declared operand types
  → promotions
  → common type
  → operation in that type
  → result type/value
  → assignment conversion, if any
```

Do this before running the Day 7 lab. It is slower than guessing once and much faster than debugging a signed/unsigned bug later.



Promotion and common-type rules determine the operation domain before a backend chooses machine width and condition codes.

---

7. How integer type appears in assembly

C does not mandate instruction selection, but type gives the compiler facts it must preserve.

Width

An access through `unsigned char *` reads one C byte. A common 32-bit `int` load reads four bytes. A 64-bit `long` operation on LP64 commonly uses a 64-bit instruction width.

Extension

When a narrow value becomes wider:

- an unsigned value needs zero extension;
- a signed value needs sign extension.

On x86-64, likely instruction families include `movzx` / `movzbl` for zero extension and `movsx` / `movsbl` for sign extension. Sometimes an ordinary 32-bit register write supplies the needed zeroing for free. The exact sequence is a compiler choice.

Comparison

The subtraction-like flag setting may be identical, but the branch condition depends on interpretation:

```
signed less-than     → condition based on sign/overflow relation
unsigned below       → condition based on carry/borrow
```

At higher optimization, the compiler may replace a branch with `setcc`, a conditional move, vector instructions, or a compile-time constant.

Division and shifts

Signed and unsigned division commonly use different instructions or setup. Right shift of unsigned values is logical; right shift of a negative signed value is implementation-defined in the C version used by this lab, even though x86 offers both arithmetic and logical right shifts.

This is the useful claim:

> Type constrains the required meaning, and that meaning influences generated operations.

This is the misleading claim:

> Every C type maps permanently to one assembly instruction.

Architecture lens — x86-64, AArch64, and RISC-V

The same defined C expression can lower differently across x86-64, AArch64, and RISC-V because their instructions, register sets, condition mechanisms, and available extensions differ. The target C implementation also chooses facts such as the data model, plain-`char` signedness, alignment, and endianness; common 64-bit Linux targets often share LP64 and little-endian choices, but C does not require that combination.

Keep the layers separate: C specifies language behavior, the platform ABI fixes binary layout and calling conventions, the ISA supplies architectural operations, and a compiler chooses a valid lowering. Pipeline and cache details may change cost but not the required C result. See Architecture Comparison.

---

8. Floating point: enough orientation to read the boundary

C supplies `float`, `double`, and `long double`. Their formats, precision, range, and evaluation details are properties of the implementation. Many current systems use IEC 60559 / IEEE 754 binary formats for `float` and `double`, but portable C code should query what it needs from `<float.h>`.

Useful macros include:

- `FLT_RADIX`: radix used for floating representation;
- `FLT_MANT_DIG`, `DBL_MANT_DIG`: precision in radix digits;
- `FLT_MIN`, `FLT_MAX`, `DBL_MIN`, `DBL_MAX`: range-related limits;
- `FLT_EVAL_METHOD`: possible wider evaluation behavior.

Binary floating point cannot represent most decimal fractions exactly. `0.1` is therefore usually a nearby binary value, not mathematical one tenth. That is a representation issue, not random compiler error.

Floating arithmetic also has rounding modes, infinities, NaNs, signed zero, exceptions, contraction, and optimization controls. Those deserve their own study. For today, retain four facts:

1. integer-to-floating conversion can lose precision;
2. floating-to-integer conversion requires careful range reasoning;
3. equality after arithmetic is often the wrong comparison model;
4. generated code commonly uses floating/vector registers and instruction families rather than integer arithmetic instructions.

The challenge prints precision characteristics and the bytes of `1.0f`. Treat those bytes as an observation. The program copies them with `memcpy` into `uint32_t`; it does not violate aliasing rules by pretending a `float *` is a `uint32_t *`.

---

9. Expressions, operators, and evaluation caveats

An **operator** such as `+`, `*`, `&&`, or `=` participates in an **expression**. Expressions may compute values and may also have side effects.

Three different ideas are often confused:

- **precedence** controls grouping;
- **associativity** resolves grouping among operators at the same precedence;
- **evaluation order** controls when subexpressions are evaluated.

For example:

```
a + b * c
```

groups as `a + (b * c)` because of precedence. That alone does not establish an execution order between every subexpression.

Likewise:

```
f() + g()
```

does not generally promise whether `f` or `g` is called first. If both depend on shared side effects, the program may become unspecified or undefined depending on what they do.

Avoid puzzle code such as:

```
i = i++ + 1;       /* undefined behavior */
printf("%d %d\n", next(), next()); /* argument order is not specified */
```

Write separate full expressions when order matters:

```
int first = next();
int second = next();
printf("%d %d\n", first, second);
```

Short-circuit operators do impose useful sequencing:

```
p != NULL && p->ready
```

The right operand of `&&` is evaluated only if the left operand compares nonzero. The conditional operator and comma operator also have sequencing rules; the comma separating function arguments is not the comma operator.

Week 3 returns to qualifiers, sequencing, optimization, atomics, and undefined behavior. Today's goal is simply to stop treating source order as a universal execution schedule.

---

10. Scope, storage duration, and lifetime are three questions

These terms are frequently collapsed into “where the variable lives.”

Scope: where can the name be used?

Scope is about identifiers in source:

- block scope for a local declaration;
- function scope for labels;
- function-prototype scope for parameter names in a prototype;
- file scope for declarations outside functions.

Storage duration: how long is storage reserved?

At this level:

- **automatic:** normally associated with block entry/exit;
- **static:** storage lasts for the entire program execution;
- **thread:** one instance per thread for the thread's execution;
- **allocated:** storage is controlled through allocation/deallocation.

`static` is context-sensitive. At block scope it gives static storage duration; at file scope it also affects linkage. Those are different properties.

Lifetime: when does an object exist?

Lifetime is the portion of execution during which an object exists and retains a valid stored value under the language rules. Accessing an object outside its lifetime is not repaired by the address bits still looking plausible.

```
int *bad(void)
{
    int local = 7;
    return &local;           /* pointer outlives the automatic object */
}
```

The compiler may place `local` on a stack, in a register, or nowhere materialized. The bug is defined at the C lifetime level, not by whether the old stack bytes happen to remain.

Pointers, allocation, ownership, and lifetime get full treatment next week. For now, practice asking separately:

```
Can this identifier be named here?          → scope
What rule provides its storage?             → storage duration
Does the referred-to object exist now?      → lifetime
Can another translation unit name it?       → linkage (a fourth property)
```

---

11. Lab — types, limits, conversions, bytes, and assembly

Use the existing challenge:

```
02-assembly-memory-and-c/challenges/day-007-c-machine-model-lab.c
```

Phase A: predict

Before compiling, record:

1. `CHAR_BIT` and each printed `sizeof`;
2. whether `-1 < 1u` is true;
3. the result of `UINT32_MAX + 1`;
4. the result of adding `(unsigned char)250` and `(unsigned char)10`;
5. the byte order expected for `0x11223344`;
6. the expected bytes of `1.0f`, clearly labelled as an implementation prediction rather than a C guarantee.

For items 2–4, write the promotion/conversion chain, not only the answer.

Phase B: compile and run

```
cd /home/jd/Desktop/masterclass

gcc -std=c17 -Wall -Wextra -Wpedantic -Wconversion -Wsign-conversion \
  -O0 -g 02-assembly-memory-and-c/challenges/day-007-c-machine-model-lab.c \
  -o /tmp/day7-c-model-O0

/tmp/day7-c-model-O0
```

Record compiler version and target:

```
gcc --version | sed -n '1p'
gcc -dumpmachine
```

Warnings are teaching evidence. Do not silence one with a cast until you can state which conversion the cast requests and why it is valid.

Phase C: compare source with generated assembly

Generate readable compiler output:

```
gcc -std=c17 -Wall -Wextra -O0 -fno-asynchronous-unwind-tables \
  -S -masm=intel \
  02-assembly-memory-and-c/challenges/day-007-c-machine-model-lab.c \
  -o /tmp/day7-c-model-O0.s

gcc -std=c17 -Wall -Wextra -O2 -fno-asynchronous-unwind-tables \
  -S -masm=intel \
  02-assembly-memory-and-c/challenges/day-007-c-machine-model-lab.c \
  -o /tmp/day7-c-model-O2.s

diff -u /tmp/day7-c-model-O0.s /tmp/day7-c-model-O2.s || true
```

Find `mixed_compare`, `wrap_add`, and `promoted_sum` at `-O0`. Then inspect `main` in both files.

At `-O2`, static helpers may be inlined, constant-propagated, or removed. That is not a failed lab. It is direct evidence that function boundaries and source statements do not each require a surviving instruction sequence.

For each operation you do find, annotate:

- operand C types after promotions;
- operation width;
- whether extension is signed or zero-filling;
- signed or unsigned condition used by a comparison;
- where the result appears according to the ABI;
- whether the compiler computed the result during translation.

Phase D: inspect instructions in the executable

```
gcc -std=c17 -Wall -Wextra -O2 -g \
  02-assembly-memory-and-c/challenges/day-007-c-machine-model-lab.c \
  -o /tmp/day7-c-model-O2

objdump -d -Mintel --disassemble=main /tmp/day7-c-model-O2
```

Compare this with the `.s` file. The executable contains final addresses and encodings; compiler-generated `.s` is still assembly source for the assembler.

Phase E: change one type at a time

Make a temporary copy outside the repository:

```
cp 02-assembly-memory-and-c/challenges/day-007-c-machine-model-lab.c \
  /tmp/day7-c-model-experiment.c
```

Try controlled changes in `/tmp`:

1. change `mixed_compare`'s second parameter from `unsigned int` to `int`;
2. change `promoted_sum` parameters from `unsigned char` to `signed char` and choose safely representable inputs;
3. add functions that widen `signed char` and `unsigned char` to `int`;

4. add `int32_t` and `int64_t` addition functions;
5. add one `float` addition and one `double` addition.

For every change:

```
predict C result
  → predict assembly-level consequence
  → compile at -O0 and -O2
  → annotate what changed
  → explain which difference came from type and which from optimization
```

Do not add signed-overflow or out-of-range floating-to-integer experiments and then infer portable behavior from one run. A single observed result does not define undefined behavior.

---

12. Reading the results without overclaiming

Suppose the compiler emits no addition for `wrap_add(UINT32_MAX)` in optimized `main`, only materializing zero for `printf`. The wrong conclusion is:

> Unsigned addition uses no CPU instruction.

The defensible conclusion is:

> For this whole program, compiler, target, flags, and visible call site, the required unsigned result was known during translation, so no runtime addition was needed.

Likewise, seeing four bytes for `int` does not prove C defines `int` as four bytes. Seeing little-endian order does not prove every C machine is little-endian. Seeing IEEE-like `float` bytes does not make that encoding the definition of C `float`.

Attach scope to evidence:

```
language guarantee
implementation documentation
compiler/target/flag observation
one runtime measurement
```

That habit is one of the dividing lines between systems reasoning and machine folklore.

---

13. Connections across the week

```
C type and expression
    ↓ language defines required values and side effects
compiler chooses widths, extensions, comparisons, and instruction families
    ↓ ABI places arguments/results in registers or memory
ISA instructions transform architectural state
    ↓ loads/stores use addresses and object representations
alignment and byte order shape layout/interpretation
    ↓ caches move lines rather than C objects
memory hierarchy affects cost without changing defined C results
```

The compiler is not replacing C types with hardware types one-for-one. It is proving that some sequence—or no runtime sequence at all—implements the required behavior.

This also explains why kernel code is careful about types:

- fixed-width integer types communicate hardware/protocol widths;
- unsigned masks express modular bit operations;
- signed/unsigned comparisons can silently change domains;
- object lifetime constrains valid pointer use even when addresses remain;
- compiler flags and kernel-specific rules form part of the implementation contract.

---

14. Mastery — Explain · Draw · Observe · Build

Explain

Without notes:

- why C uses an abstract machine;
- translation environment versus execution environment;
- hosted versus freestanding execution;
- object versus identifier versus value versus type versus representation;
- why a C byte is not defined as eight bits;
- integer range and rank;
- integer promotions and the usual arithmetic conversions in `-1 < 1u`;
- unsigned wrap versus signed overflow;
- precedence versus evaluation order;
- scope versus storage duration versus lifetime;
- why one C line need not correspond to one instruction.

Draw

Draw both:

```
source → preprocessing → translation unit → compiler → assembly/object
       → linker → executable → loader → executing process
```

and:

```
declared types → promotions → common type → required operation/result
              → optimizer → target operation width/condition/extension
```

Mark which arrows belong to the language model, implementation, ABI, ISA, OS, and microarchitecture.

Observe

Preserve:

- compiler version and target triple;
- type sizes and limits;
- predicted and observed conversion results;
- raw bytes with a note that they are implementation observations;
- annotated `-O0` and `-O2` snippets;
- one helper that disappears or changes shape under optimization.

Build

Create a one-page conversion sheet based on rules, not memorized examples:

1. list integer ranks;
2. state the integer-promotion test;
3. sketch the signed/unsigned common-type decision;
4. include one safe example of each;
5. attach one observed assembly consequence: width, extension, or condition.

Then write two tiny functions that differ only in parameter signedness. Predict the comparison condition, compile them, and explain the generated operations.

Why? notebook

1. Why can the compiler remove an object that C says exists?
2. Why is `sizeof(char)` always one while `CHAR_BIT` can vary?
3. Why do two `unsigned char` operands often produce an `int` result?
4. Why can `-1 < 1u` be false without changing the stored value of `-1`?
5. Why is unsigned overflow defined while signed overflow is not?
6. Why can signed and unsigned addition use the same machine instruction but comparisons use different conditions?
7. Why is examining a `float` through `unsigned char` or `memcpy` safer than pointer-casting it to `uint32_t *`?
8. Why can `-O2` erase the function you planned to inspect?
9. Why does source order not always specify evaluation order?
10. Why is an old stack address not evidence that an automatic object's lifetime continues?
11. Why does a cross-compiler's `sizeof(long)` describe the target rather than the host?
12. Why can two different assembly sequences both correctly implement the same C expression?

End model

```
C specifies a typed abstract execution
  → implementation choices fill in widths, ranges, and representations
  → promotions/conversions determine the domain of each operation
  → the optimizer preserves required observable behavior
  → the backend chooses target operations and the ABI carries values
  → the CPU executes instructions, not source lines
```

If you can take `-1 < 1u`, derive its C result, identify the unsigned comparison in unoptimized assembly, and explain why optimized assembly may contain no comparison at all, you are reasoning across the right boundaries.

References used selectively

Primary language reference:

- ISO/IEC 9899:2018 (C17), especially clauses 5.1 (conceptual models), 5.2.4.2 (numerical limits), 6.2.1–6.2.6 (scope, linkage, storage duration, types, representation), 6.3 (conversions), and 6.5 (expressions).

Local companions:

- Bryant and O'Hallaron, *Computer Systems: A Programmer's Perspective*, 3rd ed., §§2.1–2.4 and Chapter 3 for representations, arithmetic, and the source-to-machine connection. Local PDF: `references/library/books/01-foundations-and-architecture/computer-systems-programmers-perspective-bryant-ohallaron-3e-global.pdf`
- Kerrisk, *The Linux Programming Interface*, Chapter 6 for a Linux process memory-layout perspective, kept distinct from C's language-level storage duration and lifetime rules.

Current implementation references:

- GCC manual, “C Implementation-Defined Behavior” and “Options Controlling C Dialect”: https://gcc.gnu.org/onlinedocs/gcc/C-Implementation.html
- cppreference C language pages on arithmetic types, conversions, object, and lifetime, used as a navigational companion rather than the normative standard: https://en.cppreference.com/w/c/language

Next bridge: types tell the compiler what values and operations mean. Next week, pointers, arrays, strings, allocation, and ownership ask when an address designates a live object—and what happens when it does not.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check

- View quiz source on GitHub

Put it into practice — C Machine Model Lab

Format: C lab

What you will do: Day 7: defined experiments for types, conversions, representation, and

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
gcc -std=c17 -Wall -Wextra -O0 -g -pthread 02-assembly-memory-and-c/challenges/day-007-c-machine-model-lab.c -lm -o /tmp/day-007-c-machine-model-lab && /tmp/day-007-c-machine-model-lab
```

- View lab source on GitHub